The Network Layer (Unit-III-1)

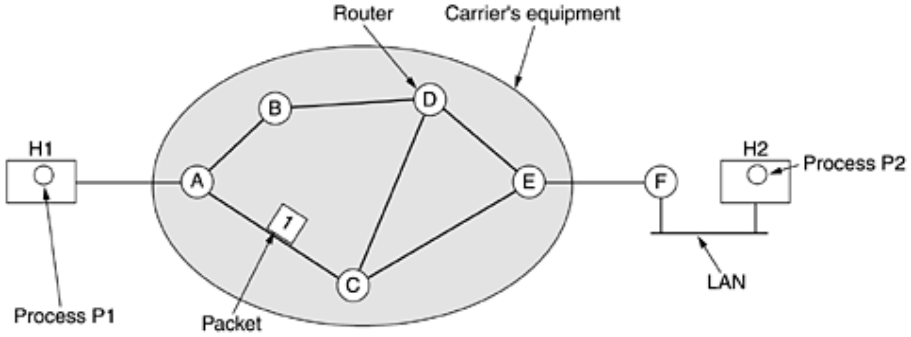
❖ Network Layer Design Issues:

While designing the network layer we have to consider some of the design issues. These issues include the service provided to the transport layer and the internal design of the subnet.

Store-and-Forward Packet Switching:

The major components of the system are the carrier's equipment, shown inside the shaded oval, and the customers' equipment, shown outside the oval. Host *H1* is directly connected to one of the carrier's routers, *A*, by a leased line. In contrast, *H2* is on a LAN with a router, *F*, owned and operated by the customer. This router also has a leased line to the carrier's equipment. We have shown *F* as being outside the oval because it does not belong to the carrier, but in terms of construction, software, and protocols, it is probably no different from the carrier's routers.

Figure 5-1. The environment of the network layer protocols.



Here, a host with a packet to send transmits it to the nearest router, either on its own LAN or over a point-to-point link to the carrier. The packet is ***stored*** there until it has fully arrived so the checksum can be verified. Then it is ***forwarded*** to the next router along the path until it reaches the destination host, where it is delivered. This mechanism is ***store-and-forward*** packet switching.

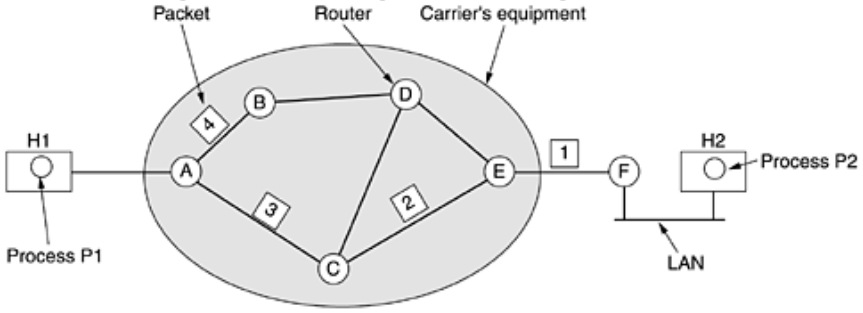
Services Provided to the Transport Layer:

The network layer provides services to the transport layer at the ***network layer/transport layer interface***. The network layer services have been designed with the following *goals* in mind.

1. The services should be independent of the router technology.
2. The transport layer should be shielded from the number, type, and topology of the routers present.
3. The network addresses made available to the transport layer should use a uniform numbering plan, even across LANs and WANs.

Implementation of Connectionless Service:

Figure 5-2. Routing within a datagram subnet.



A's table

Dest.	initially	later
A	–	–
B	B	B
C	C	C
D	B	B
E	C	B
F	C	B

Dest. Line

C's table

A	A
B	A
C	–
D	D
E	E
F	E

E's table

A	C
B	D
C	C
D	D
E	–
F	F

In connectionless service, packets are injected into the subnet individually and routed independently of each other. No advance setup is needed. In this context, the packets are frequently called **datagrams** and the subnet is called a **datagram subnet**.

Every router has an internal table telling it *where to send packets* for each possible destination. Each table entry is a pair consisting of a ***destination*** and ***the outgoing line*** to use for that destination. Only directly-connected lines can be used.

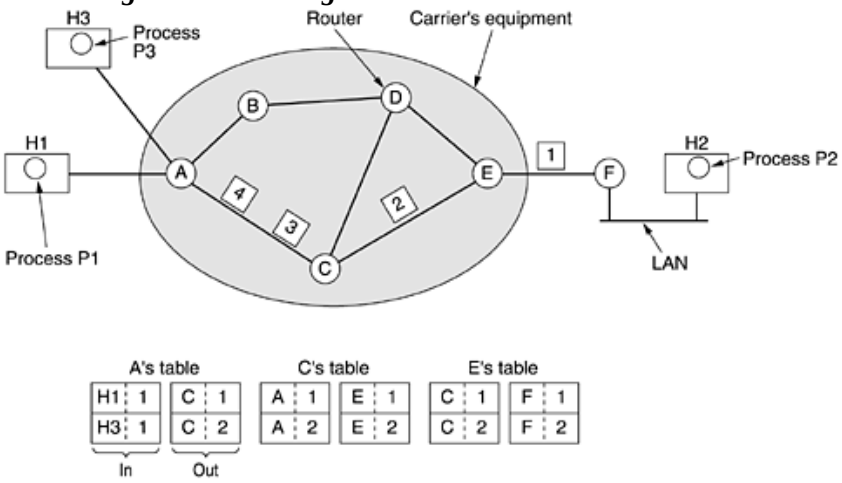
As they arrived at *A*, packets 1, 2, and 3 were stored briefly (to verify their checksums). Then each was forwarded to *C* according to *A*'s table. Packet 1 was then forwarded to *E* and then to *F*. When it got to *F*, it was encapsulated in a data link layer frame and sent to *H2* over the LAN. Packets 2 and 3 follow the same route However, something different happened to packet 4. When it got to *A* it was sent to router *B*, even though it is also destined for *F*. The algorithm that manages the tables and makes the routing decisions is called the **routing algorithm**.

Implementation of Connection-Oriented Service:

If *connection-oriented service* is used, a path from the source router to the destination router must be established before any data packets can be sent. This connection is called a **VC** (**virtual circuit**), in analogy with the physical circuits set up by the telephone system, and the subnet is called a **virtual-circuit subnet**.

As an example, consider the situation of Fig. 5-3. Here, host *H1* has established connection 1 with host *H2*. It is remembered as the first entry in each of the routing tables. The first line of *A*'s table says that if a packet bearing connection identifier 1 comes in from *H1*, it is to be sent to router *C* and given connection identifier 1. Similarly, the first entry at *C* routes the packet to *E*, also with connection identifier 1.

Figure 5-3. Routing within a virtual-circuit subnet.



Comparison of Virtual-Circuit and Datagram Subnets:

Issue	Datagram subnet	Virtual-circuit subnet
Circuit setup	Not needed	Required
Addressing	Each packet contains the full source and destination address	Each packet contains a short VC number
State information	Routers do not hold state information about connections	Each VC requires router table space per connection
Routing	Each packet is routed independently	Route chosen when VC is set up; all packets follow it
Effect of router failures	None, except for packets lost during the crash	All VCs that passed through the failed router are terminated
Quality of service	Difficult	Easy if enough resources can be allocated in advance for each VC
Congestion control	Difficult	Easy if enough resources can be allocated in advance for each VC

❖ **Routing Algorithms:**

Characteristics of routing algorithms:

- **Correctness:** it could able to deliver packets from source to destination without failure or without other nodes.
- **Simplicity:** the function should be simple in operation.
- **Robustness:** if the network is delivering packets via some route, if any failures or overloads occur, the function should react to such contingencies without the loss of packets or the breaking of virtual circuits.
- **Stability:** The outing function should react to contingencies slowly that are neither fast nor too slow. Why means, for example, if the network may react to congestion in one area by shifting most of load to second area. Now the second area is overloaded and the first is under-utilized, causing a second shift. During these shifts, packets may travel in loops through the network.
- **Fairness and Optimality:** some performance criteria may give higher priority to the exchange of packets between neighbor stations compared to an exchange between distant stations. This policy may maximize average throughput but will appear unfair to the station that primarily needs to communicate with distant stations.
- **Efficiency:** The efficiency routing function involves the processing overhead at each node and often a transmission overhead.

Classification of routing algorithms:

Routing algorithms can be grouped into two major classes:

- **Nonadaptive algorithms** do not base their *routing decisions* on measurements or estimates of the *current traffic* and *topology*. Instead, the choice of the route to use to get from *I* to *J* (for all *I* and *J*) is computed in advance, off-line, and downloaded to the routers when the network is booted. This procedure is sometimes called ***"static routing"***.

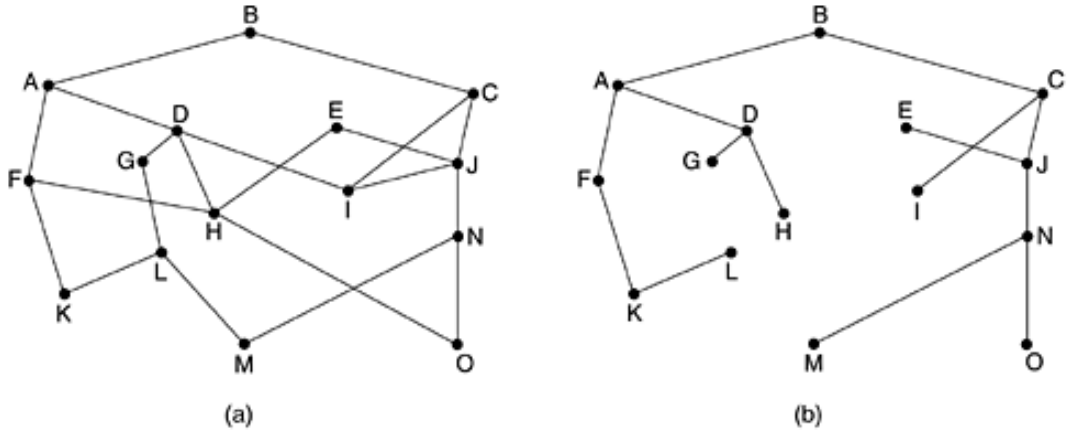
- **Adaptive algorithms** on the contrary are ***dynamic*** and ***online***. They collect their information about the state of the network and make routing decisions based on the latest information, for example, Distance vector routing and link state routing.

The Optimality Principle:

It states that one can make a general statement about *optimal routes* without regard to network topology or traffic. This statement is known as the **optimality principle**. It *states that* if router *J* is on the optimal path from router *I* to router *K*, then the optimal path from *J* to *K* also falls along the same route. To see this, call the part of the route from *I* to *J* r_1 and the rest of the route r_2. If a route better than r_2 existed from *J* to *K*, it could be concatenated with r_1 to improve the route from *I* to *K*, *contradicting* our statement that $r1r2$ is optimal.

As the result of optimality principle the set of optimal routes from all sources to a given destination form a tree rooted at the destination. Such a tree is called a **sink tree** and is illustrated in Fig. 5-6, where the distance metric is the number of hops. The goal of all routing algorithms is to discover and use the sink trees for all routers. Since a sink tree is indeed a tree, it ***does not contain any loops***, so each packet will be delivered within a finite and bounded number of hops.

***Figure 5-6. (a) A subnet. (b) A sink tree for router* B.**

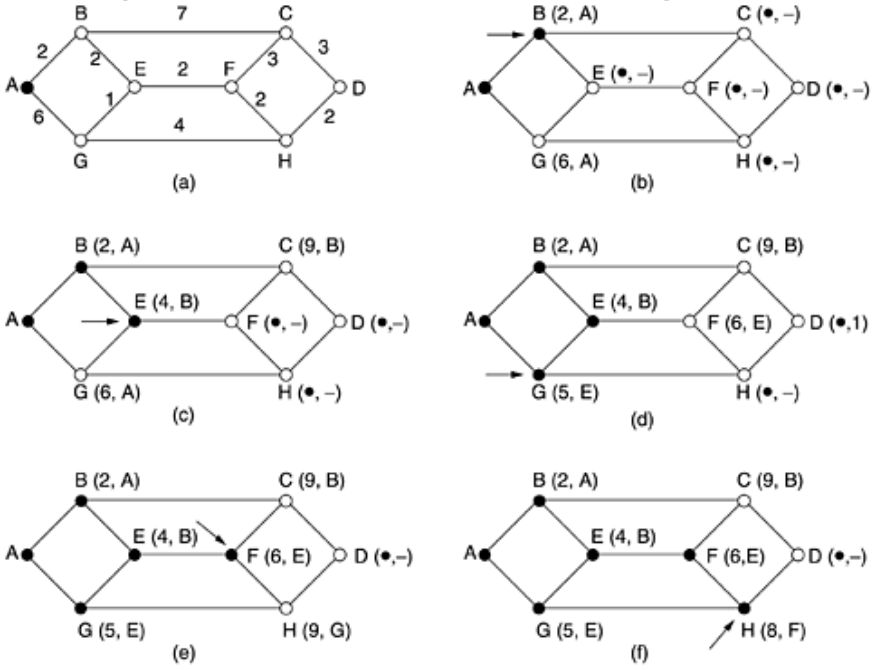


Shortest Path Routing:

The idea behind routing algorithms is to build a ***graph*** of the subnet, with each *node* of the graph representing a *router* and each *arc* of the graph representing a *communication line*. To choose a route between a given pair of routers, the *algorithm just finds the shortest path* between them on the graph.

The concept of a **shortest path** deserves some explanation. One way of measuring path length is the *number of hops*. Using this metric, the paths *ABC* and *ABE* in Fig. 5-7 are equally long. Another metric is the *geographic distance in kilometers*, in which case *ABC* is clearly much longer than *ABE.* Another metric is may be *time delay* etc., like so many metrics can be used for shortest path routing.

Figure 5-7. The first five steps used in computing the shortest path from* A *to* D. *The arrows indicate the working node.



Several algorithms for computing the shortest path between two nodes of a graph are known. This one is due to ***Dijkstra*** (1959). Each *node is labeled* (in parentheses) with its *distance from the source node along the best known path*. Initially, no paths are known, so all nodes are labeled with *infinity*. As the algorithm proceeds and paths are found, the labels may change, reflecting better paths. A label may be either ***tentative*** *or* ***permanent***.

⇨ Initially, all labels are tentative. When it is discovered that a label represents the shortest possible path from the source to that node, it is made permanent and never changed thereafter.

To illustrate how the labeling algorithm works, look at the weighted, undirected graph of Fig. 5-7(a), where the weights represent, for example, distance. We want to find the shortest path from *A* to *D*. We start out by marking node *A* as permanent, indicated by a filled-in circle. Then we examine, in turn, each of the nodes adjacent to *A* (the working node), relabeling each one with the distance to *A*. Whenever a node is relabeled, we also label it with the node from which the probe was made so that we can reconstruct the final path later. Having examined each of the nodes adjacent to *A*, we examine all the tentatively labeled nodes in the whole graph and make the one with the smallest label permanent, as shown in Fig. 5-7(b). This one becomes the new working node.

We now start at *B* and examine all nodes adjacent to it. If the sum of the label on *B* and the distance from *B* to the node being considered is less than the label on that node, we have a shorter path, so the node is relabeled.

After all the nodes adjacent to the working node have been inspected and the tentative labels changed if possible, the entire graph is searched for the *tentatively-labeled node* with the smallest value. This node is made *permanent* and becomes the working node for the next round. Figure 5-7 shows the first five steps of the algorithm.

Flooding:

Flooding is a static algorithm, in which in which *"every incoming packet is sent out on every outgoing line except the one it arrived on"*. Flooding obviously generates vast numbers of *duplicate packets*, in fact, an infinite number unless some measures are taken to damp (Discourage) the process.

- ✓ One such measure is to have a *hop counter* contained in the header of each packet, which is decremented at each hop, with the packet being discarded when the counter reaches zero.
  - Ideally, the hop counter should be initialized to the *length of the path* from source to destination. If the sender does not know how long the path is, it can initialize the counter to the worst case, namely, the full diameter of the subnet.
- ✓ An alternative technique for damming the flood is to keep track of which packets have been flooded, to avoid sending them out a second time. Achieve this goal is to have the source router put a sequence number in each packet it receives from its hosts. Each router then needs a ***list*** per source router telling which sequence numbers originating at that source have already been seen. If an incoming packet is on the list, it is not flooded.

Applications and Advantages:

- ⇨ Flooding is very effective routing approach, when, the information in the routing tables is not available, such as during ***system start up***.
- ⇨ Flooding is also effective when the source needs to send a packet to all hosts connected to the network for example in ***military applications***.
- ⇨ In ***distributed data base applications***, it is sometimes necessary to update the entire database concurrently; in such cases flooding is used.
- ⇨ Flooding always chooses the ***shortest path***, because it chooses every possible path in parallel.
- ⇨ In ***wireless networks***, all messages transmitted by a station can be received by all other stations within its radio range, which is, in fact, flooding, and some algorithms utilize this property.

Distance Vector Routing:

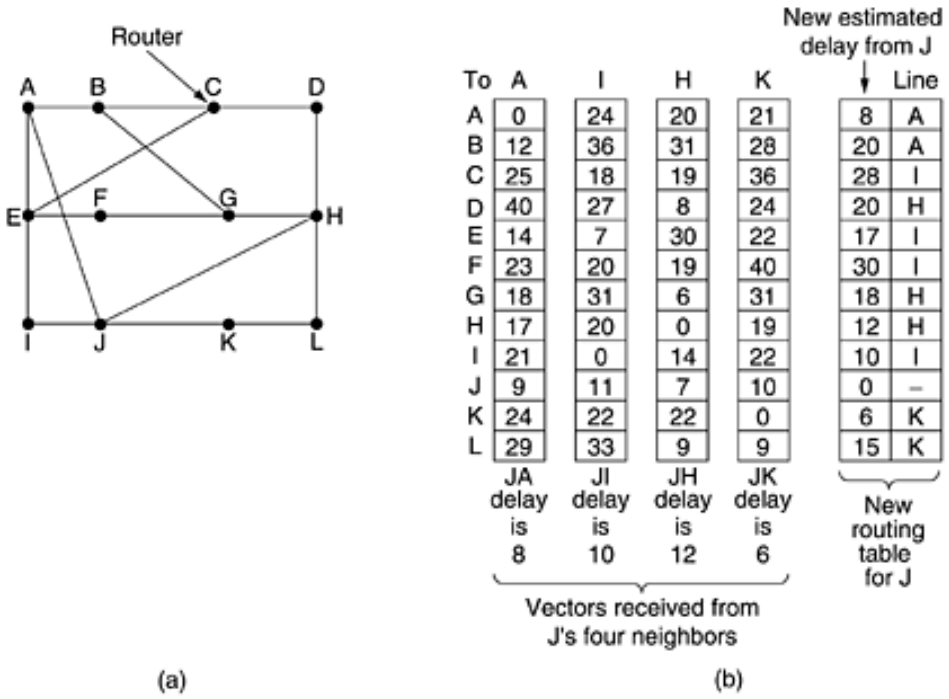
Modern computer networks generally use *dynamic routing algorithms* rather than the *static* ones because static algorithms *do not take the current network load into account*.

Distance vector routing algorithms operate by having each router maintain a ***table*** (i.e, a vector) giving the best known distance to each destination and which line to use to get there. These tables are updated by *exchanging information with the neighbors*.

The distance vector routing algorithm is also called by other names as, distributed **Bellman-Ford routing algorithm** and the **Ford-Fulkerson algorithm**.

This can be explained in Fig. 5-9. Part (a) shows a subnet. The first four columns of part (b) show the delay vectors received from the neighbors of router *J*. Suppose that *J* has measured or estimated its delay to its neighbors, *A*, *I*, *H*, and *K* as 8, 10, 12, and 6 msec, respectively.

***Figure 5-9. (a) A subnet. (b) Input from* A, I, H, K, *and the new routing table for* J.**

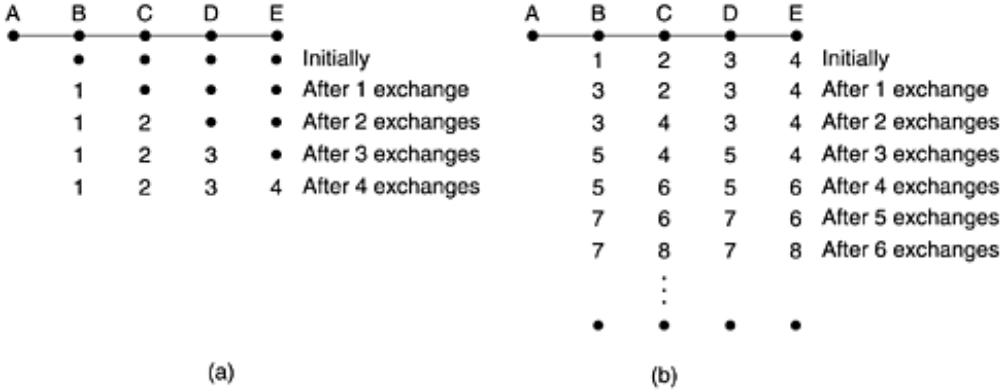


Consider how *J* computes its new route to router *G*. It knows that it can get to *A* in 8 msec, and *A* claims to be able to get to *G* in 18 msec, so *J* knows it can count on a delay of 26 msec to *G* if it forwards packets bound for *G* to *A*. Similarly, it computes the delay to *G* via *I*, *H*, and *K* as 41 (31 + 10), 18 (6 + 12), and 37 (31 + 6) msec, respectively. The best of these values is 18, so it makes an entry in its routing table that the delay to *G* is 18 msec and that the route to use is via '*H'*.

The Count-to-Infinity Problem:

consider the five-node (linear) subnet of Fig. 5-10, where the delay metric is the number of *hops*. Suppose *A* is down initially and all the other routers know this. In other words, they have all recorded the delay to *A* as infinity.

Figure 5-10. The count-to-infinity problem.



When *A* comes up, the other routers learn about it via the vector exchanges. At the time of the first exchange, *B* learns that its left neighbor has zero delay to *A*. *B* now makes an entry in its routing table that *A* is one hop away to the left. All the other routers still think that *A* is down. At this point, the routing table entries for *A* are as shown in the second row of Fig. 5-10(a). Clearly, the good news is spreading at the rate of one hop per exchange, then, C,D,E routers are updated as 2,3,4 respectively.

Now let us consider the situation of Fig. 5-10(b), in which all the lines and routers are initially up. Routers *B*, *C*, *D*, and *E* have distances to *A* of 1, 2, 3, and 4, respectively. Suddenly *A* goes down, or alternatively, the line between *A* and *B* is cut, which is effectively the same thing from *B*'s point of view.

- When **A** goes down, line between **A** and **B** is out.
- **B** does not hear anything from **A.**
- **C** informs **B**, I Have Path to **A**, of length 2.
- If metric used is Time Delay, there is no well-defined upper bound. So, high value is needed to prevent a path with a long delay from being treated as down. This problem is known as **count-to-infinity**.

Link State Routing:

Distance vector routing was replaced by ***link state routing***. Two primary problems caused its demise.

- ✶ First, since the delay metric was *queue length*, it did not take line bandwidth into account when choosing routes.
- ✶ Second problem is the count-to-infinity problem.

The idea behind link state routing is simple and can be stated as five parts. Each router must do the following:

1) Discover its neighbors and learn their network addresses.
2) Measure the delay or cost to each of its neighbors.
3) Construct a packet telling all it has just learned.
4) Send this packet to all other routers.
5) Compute the shortest path to every other router.

Learning about the Neighbors:

When a router is booted, its first task is to *learn* who its neighbors are. It accomplishes this goal by sending a special **HELLO** packet on each point-to-point line.

Measuring Line Cost:

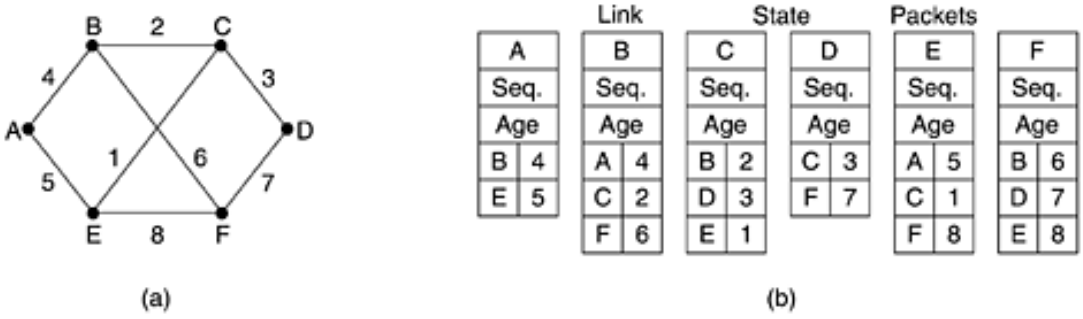
The link state routing algorithm requires each router to know, or at least have a reasonable *estimate* of, the *delay* to each of its neighbors. The most direct way to determine this delay is to send over the line a ***special ECHO packet*** that the other side is required to send back immediately. By measuring the *round-trip time* and *dividing it by two*, the sending router can get a reasonable estimate of the delay.

Building Link State Packets:

Once the information needed for the exchange has been collected, the next step is for each router to *build a packet* containing all the data. The packet starts with the ***identity of the sender***, followed by a ***sequence number*** and ***age***, and a ***list of neighbors***. For each neighbor, the *delay* to that neighbor is given.

An example subnet is given in Fig. 5-13(a) with delays shown as labels on the lines. The corresponding link state packets for all six routers are shown in Fig. 5-13(b).

Figure 5-13. (a) A subnet. (b) The link state packets for this subnet.



Distributing the Link State Packets:

The next step after building link state packets is to distribute them across the network. Flooding is used as the basic algorithm for distributing link state packets. To avoid flooding the same packet, each new packet is given a sequence number. When a packet arrives at a router for flooding then it checks whether this packet is already seen by using a pair (*source router*, *sequence number*) that each router have.

When a new link state packet comes in, it is checked against the list of packets already seen. If it is new, it is forwarded on all lines except the one it arrived on. If it is a duplicate, it is discarded. If a packet with a sequence number lower than the highest one seen so far ever arrives, it is rejected as being obsolete (**outdated**) since the router has more recent data. The ***age*** of each packet *decrement* it once per second. When the *age hits zero*, the information from that router is discarded.

The ***data structure*** used by router 'B' for the subnet is depicted in Fig. 5- 14. Each **row** here corresponds to a recently-arrived, but as yet not fully-processed, link state packet. The table records where the *packet originated*, its *sequence number* and *age*, and the data. In addition, there are send and acknowledgement flags for each of B's three lines (to A, C, and F, respectively). The ***send flags*** mean that the packet must be sent on the indicated line. The ***acknowledgement flags*** mean that it must be acknowledged there.

Figure 5-14. The packet buffer for router B in Fig. 5-13.

			Send flags			ACK flags			
Source	Seq.	Age	A	C	F	A	C	F	Data
A	21	60	0	1	1	1	0	0	
F	21	60	1	1	0	0	0	1	
E	21	59	0	1	0	1	0	1	
C	20	60	1	0	1	0	1	0	
D	21	59	1	0	0	0	1	1	

In above table, the link state packet from *A* arrives directly, so it must be sent to *C* and *F* and acknowledged to *A*, as indicated by the flag bits. Similarly, the packet from *F* has to be forwarded to *A* and *C* and acknowledged to *F*.

***Computing the New Routes*:**

Once a router has accumulated a full set of link state packets, router builds the entire subnet graph because every link is represented. Every link is, in fact, represented twice, once for each direction. The two values can be averaged or used separately.

Now ***Dijkstra's algorithm*** can be run locally to construct the shortest path to all possible destinations. The results of this algorithm can be installed in the routing tables, and normal operation resumed (continued).

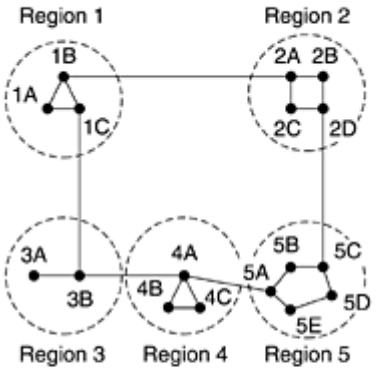
Hierarchical Routing:

Hierarchical routing is an algorithm for routing packets hierarchically. It is used due to the following reasons.

- As networks grow in size, the router routing tables grow proportionally.
- Router memory consumed by ever-increasing tables.
- More CPU time is needed to scan them and more bandwidth is needed to send status reports about them.
- At a certain point the network may grow to the point where it is no longer feasible for every router to have an entry for every other router.

When hierarchical routing is used, the routers are divided into **regions**, with each router knowing all the details about *how to route packets to destinations within its own region*, but knowing *nothing about the internal structure of other regions*. When different networks are interconnected, then each one can be treated as a separate region in order to free the routers in one network from having to know the topological structure of the other ones.

Figure 5-15. Hierarchical routing.



(a)

Full table for 1A

Dest.	Line	Hops
1A	–	–
1B	1B	1
1C	1C	1
2A	1B	2
2B	1B	3
2C	1B	3
2D	1B	4
3A	1C	3
3B	1C	2
4A	1C	3
4B	1C	4
4C	1C	4
5A	1C	4
5B	1C	5
5C	1B	5
5D	1C	6
5E	1C	5

(b)

Hierarchical table for 1A

Dest.	Line	Hops
1A	–	–
1B	1B	1
1C	1C	1
2	1B	2
3	1C	2
4	1C	3
5	1C	4

(c)

Figure 5-15 gives a quantitative example of routing in a two-level hierarchy with five regions. The full routing table for router 1*A* has 17 entries, as shown in Fig. 5-15(b). When routing is done hierarchically, as in Fig. 5-15(c), there are entries for all the local routers as before, but all other regions have been condensed into a single router, so all traffic for region 2 goes via the "**1*B* -2*A*"** line, but the rest of the remote traffic goes via the **"1*C* -3*B*"** line. Hierarchical routing has reduced the table from 17 to 7 entries. "As the ratio of the number of regions to the number of routers per region grows, the savings in table space increase".

Broadcast Routing:

Sending a packet to all destinations simultaneously is called ***broadcasting***; the algorithms used for broadcasting are called ***broadcast routing***. Various methods have been proposed for doing it. They are,

1) Distinct point-to-point routing
2) Flooding
3) Multi-destination routing
4) Use of spanning tree
5) Reverse path forwarding

Distinct point-to-point routing:

This is the simplest method for broadcasting.in this method '*sender simply sends a distinct packet to each destination or to all the nodes in the network'*. Thus it takes no special features of the subnet. This method is not desirable due to two reasons. First, Wasteful of bandwidth, Second it requires the source to have a complete list of all destinations.

Flooding:

Flooding is another obvious candidate. This algorithm sends a packet on every outgoing line except the line on which it arrived. The problem with flooding as a broadcast technique is the same problem it has as a point-to-point routing algorithm: "it generates too many packets and consumes too much bandwidth".

Multi-destination routing:

A third algorithm is **multi-destination routing**. If this method is used, each packet contains either a list of destinations or a bit map indicating the desired destinations. When a packet arrives at a router, the router checks all the destinations to determine the set of output lines that will be needed. The router generates a new copy of the packet for each output line to be used and includes in each packet only those destinations that are to use the line. In effect, the destination set is partitioned among the output lines. After a sufficient number of hops, each packet will carry only one destination and can be treated as a normal packet.

Use of spanning tree:

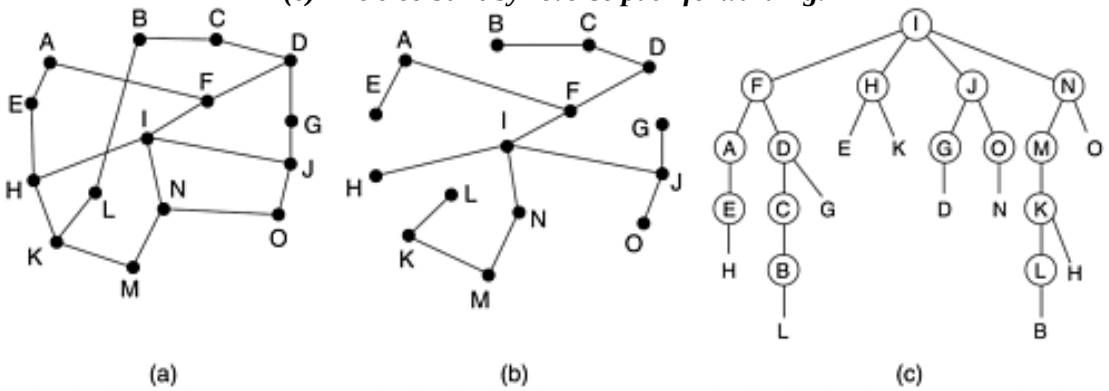
A **spanning tree** is a subset of the subnet that includes all the routers but contains ***no loops***.

If each router knows which of its lines belong to the spanning tree, it can copy an incoming broadcast packet onto all the spanning tree lines except the one it arrived on. This method makes *excellent use of bandwidth*, generating the *absolute minimum number of packets* necessary to do the job.

Reverse path forwarding:

An example of reverse path forwarding is shown in Fig. 5-16. Part (a) shows a *subnet*, part (b) shows a *sink tree for router I* of that subnet, and part (c) shows how the reverse path algorithm works. On the first hop, *I* send packets to *F*, *H*, *J*, and *N*, as indicated by the second row of the tree. Each of these packets arrives on the preferred path to *I* (assuming that the preferred path falls along the sink tree) and is so indicated by a *circle* around the letter. On the second hop, eight packets are generated, two by each of the routers that received a packet on the first hop. As it turns out, all eight of these arrive at previously unvisited routers, and five of these arrive along the preferred line. Of the six packets generated on the third hop, only three arrive on the preferred path (at *C*, *E*, and *K*); the others are duplicates. After five hops and 24 packets, the broadcasting terminates, compared with four hops and 14 packets had the sink tree been followed exactly.

Figure 5-16. Reverse path forwarding. (a) A subnet. (b) A sink tree. (c) The tree built by reverse path forwarding.



Advantages:

- The reverse path forwarding is that it is both reasonably efficient and easy to implement.
- It does not require routers to know about spanning trees.
- It does not have the overhead of a destination list or bit map in each broadcast packet as does multi-destination addressing.
- It does not require any special mechanism to stop the process.

Multicast Routing:

For some applications such as tele conferencing, a source may want to send packets to multiple destinations simultaneously or a group of processes implementing a distributed database systems. It is frequently necessary for one process to send a message to all the other members of the group.

⇨ If the group is small, it can just send each other member a point-to-point message.

⇨ If the group is large, this strategy is expensive.

Thus, we need a way to send messages to well defined groups that are numerically large in size but small compared to the network as a whole.

Sending a message to such a group is called **multicasting**, and its routing algorithm is called **multicast routing**.

Multicasting requires *group management*.

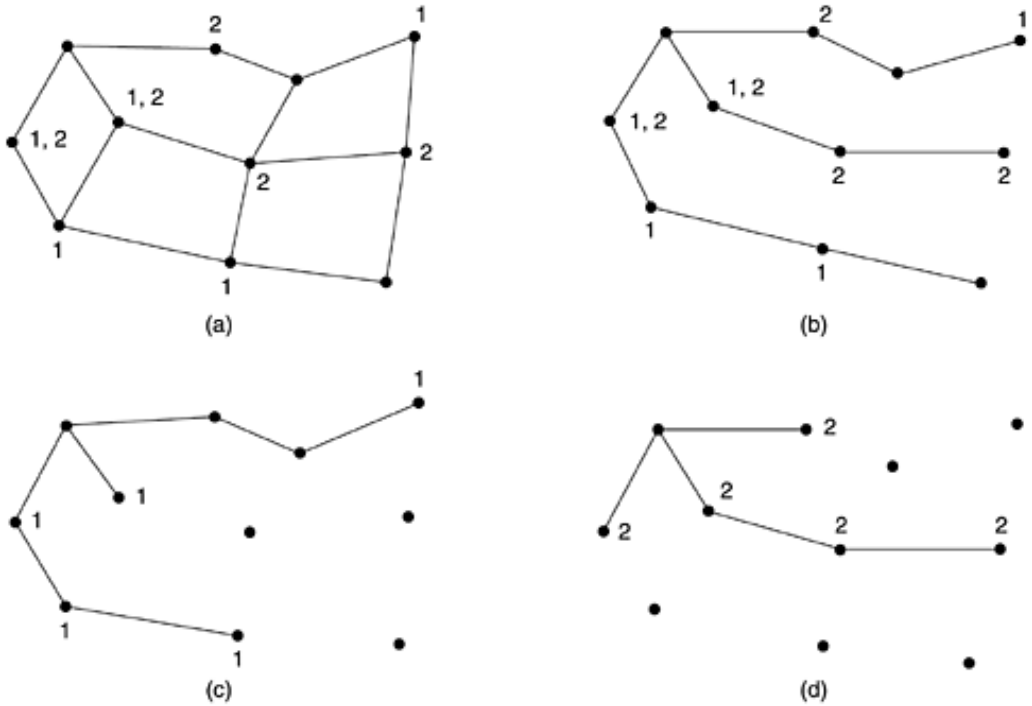
✓ To *create* and *destroy groups* and

✓ To allow processes to *join* and *leave* groups.

The routing algorithm does not know how these tasks are accomplished but when a process joins a group; it informs its host of this fact. It is important that routers know which of their hosts belong to which groups. Either host must inform their routers about changes in group membership, or routers must query their hosts periodically. Either way, routers learn about which of their hosts are in which groups. Routers tell their neighbors, so the information propagates through the subnet.

To do multicast routing, each router computes a *spanning tree* covering all other routers. For example, in Fig. 5-17(a) we have two groups, 1 and 2. Some routers are attached to hosts that belong to one or both of these groups, as indicated in the figure. A spanning tree for the leftmost router is shown in Fig. 5-17(b).

Figure 5-17. (a) A network. (b) A spanning tree for the leftmost router. (c) A multicast tree for group 1. (d) A multicast tree for group 2

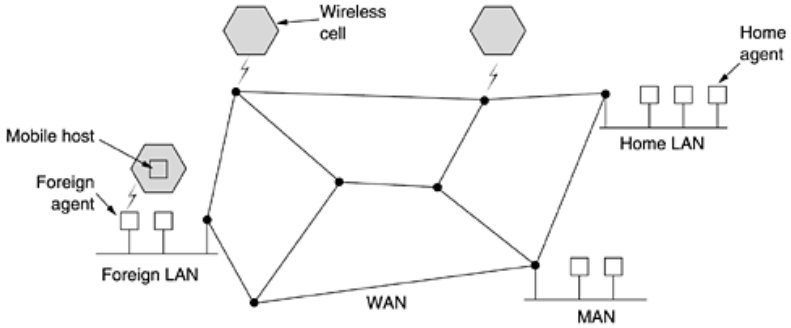


When a process sends a multicast packet to a group, the **first** router examines its spanning tree and prunes it, removing all lines that do not lead to hosts that are members of the group. In our example, Fig. 5-17(c) shows the pruned spanning tree for group 1. Similarly, Fig. 5- 17(d) shows the pruned spanning tree for group 2. Multicast packets are forwarded only along the appropriate spanning tree.

Various ways of pruning the spanning tree are possible. The simplest one can be used if *link state routing* is used and each router is aware of the complete topology, including which hosts belong to which groups. Then the spanning tree can be pruned, starting at the end of each path, working toward the root, and removing all routers that do not belong to the group in question.

Routing for Mobile Hosts:

Figure 5-18. A WAN to which LANs, MANs, and wireless cells are attached.



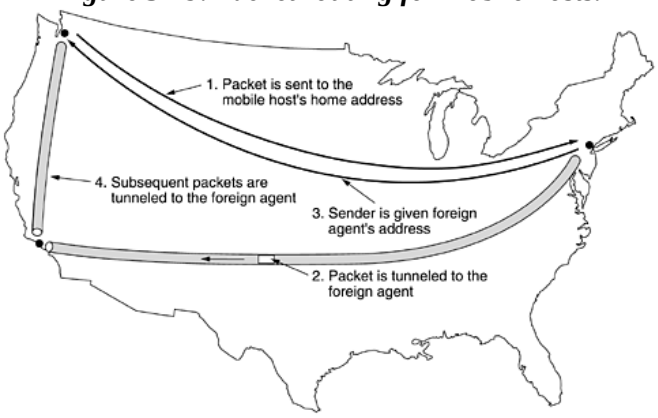
In the model of Fig. 5-18, the world is divided up (geographically) into small units. Let us call them areas, where an area is typically a LAN or wireless cell. Each area has one or more **foreign agents**, which are processes that keep track of all mobile hosts visiting the area. In addition, each area has a **home agent**, which keeps track of hosts whose home is in the area, but who are currently visiting another area.

When a new host enters an area, either by connecting to it (e.g., plugging into the LAN) or just wandering (travelling) into the cell, his computer must register itself with the foreign agent there. The registration procedure typically works like this:

1) Periodically, each foreign agent broadcasts a packet announcing its existence and address. A newly-arrived mobile host may wait for one of these messages, but if none arrives quickly enough, the mobile host can broadcast a packet saying: Are there any foreign agents around?
2) The mobile host registers with the foreign agent, giving its home address, current data link layer address, and some security information.

3) The foreign agent contacts the mobile host's home agent and says: One of your hosts is over here. The message from the foreign agent to the home agent contains the foreign agent's network address. It also includes the security information to convince the home agent that the mobile host is really there.
4) The home agent examines the security information, which contains a timestamp, to prove that it was generated within the past few seconds. If it is happy, it tells the foreign agent to proceed.
5) When the foreign agent gets the acknowledgement from the home agent, it makes an entry in its tables and informs the mobile host that it is now registered.

Figure 5-19. Packet routing for mobile hosts.



Ideally, when a host leaves an area, that, too, should be announced to allow deregistration, but many users abruptly turn off their computers when done.

When a packet is sent to a mobile host, it is routed to the host's home LAN because that is what the address says should be done, as illustrated in step 1 of Fig. 5-19.

The home agent then does two things.

- **First**, it encapsulates the packet in the payload field of an outer packet and sends the latter to the foreign agent (step 2 in Fig. 5-19). This mechanism is called ***tunneling***;
- **Second**, the home agent tells the sender to henceforth send packets to the mobile host by encapsulating them in the payload of packets explicitly addressed to the foreign agent instead of just sending them to the mobile host's home address (step 3). Subsequent packets can now be routed directly to the host via the foreign agent (step 4), bypassing the home location entirely.

Routing in Ad Hoc Networks:

Here routing can be done when the hosts are mobile and also routers themselves are mobile. Among the possibilities are:

1. Military vehicles on a battlefield with no existing infrastructure.
2. A fleet of ships at sea.
3. Emergency workers at an earthquake that destroyed the infrastructure.
4. A gathering of people with notebook computers in an area lacking 802.11.

In all these cases, and others, each node consists of a *router* and a *host*, usually on the same computer. *Networks of nodes* that just happen to be near each other are called **ad hoc networks** or **MANETs** (**Mobile Ad hoc NETworks**).

A variety of routing algorithms for ad hoc networks have been proposed. One of the more interesting ones is the ***AODV (Ad hoc On-demand Distance Vector) routing algorithm***. It is a distant relative of the Bellman-Ford distance vector algorithm but adapted to work in a mobile environment and takes into account the *limited bandwidth* and *low battery life* found in this environment.

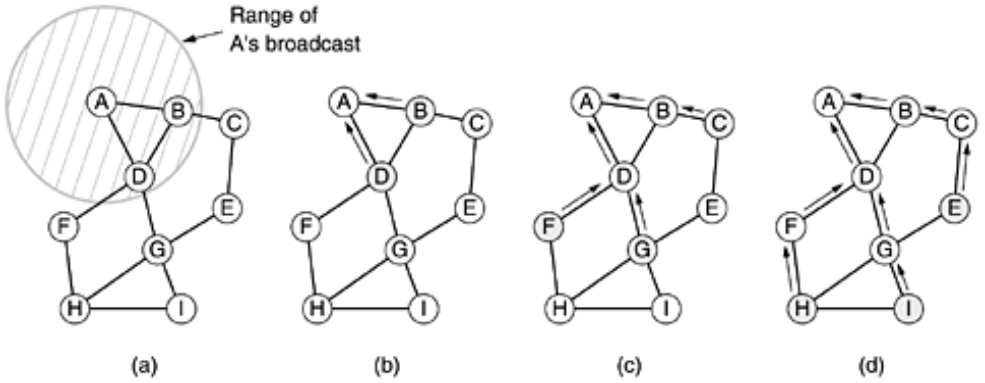
- It is an ***on-demand algorithm***, that is, it determines a route to some destination only when somebody wants to send a packet to that destination.

Route Discovery:

Consider the ad hoc network of Fig. 5-20, in which a process at node *A* wants to send a packet to node *I*. The AODV algorithm maintains a table at each node, keyed (enetered) by destination, giving information about that destination, including which neighbor to send packets to in order to reach the destination. Suppose that *A* looks in its table and does not find an entry for *I*. It now has to discover a route to *I*. This property of discovering routes only when they are needed is what makes this algorithm **"on demand."**

Figure 5-20. (a) Range of* A*'s broadcast. (b) After* B *and* D *have received* A*'s broadcast

(c) After* C, F, *and* G *have received* A*'s broadcast.
(d) After* E, H, *and* I *have received* A*'s broadcast.
The shaded nodes are new recipients. The arrows show the possible reverse routes.



To locate *I*, *A* constructs a special ROUTE REQUEST packet and broadcasts it. The packet reaches *B* and *D*, as illustrated in Fig. 5-20(a). In fact, the reason *B* and *D* are connected to *A* in the graph is that they can receive communication from *A*. *F*, for example, is not shown with an arc to *A* because it cannot receive *A*'s radio signal. Thus, *F* is not connected to *A*.

Route Maintenance:

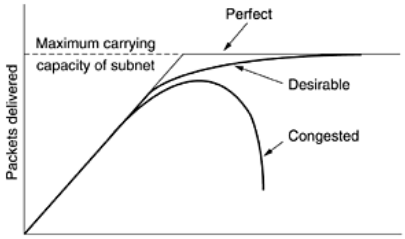
Because nodes can move or be switched off, the topology can change spontaneously. Periodically, each node broadcasts a ***Hello*** message. Each of its neighbors is expected to respond to it. If no response is forthcoming, the broadcaster knows that that neighbor has moved out of range and is no longer connected to it. Similarly, if it tries to send a packet to a neighbor that does not respond, it learns that the neighbor is no longer available.

This information is used to purge ($_{\textbf{remove}}$) routes that no longer work. For each possible destination, each node, *N*, keeps track of its neighbors that have fed it a packet for that destination during the last ΔT seconds. These are called *N*'s **active neighbors** for that destination.

❖ Congestion Control Algorithms:

When too many packets are present in (a part of) the subnet, performance degrades. This situation is called **congestion**.

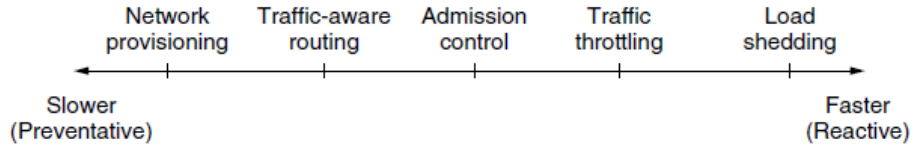
Figure depicts the onset of congestion. When the number of packets hosts send into the network is well within its carrying capacity, the number delivered is proportional to the number sent. If twice as many are sent, twice as many are delivered. However, as the offered load approaches the carrying capacity, bursts of traffic occasionally fill up the buffers inside routers and some packets are lost. These lost packets consume some of the capacity, so the number of delivered packets falls below the ideal curve. The network is now congested.



If all of a sudden, streams of packets begi... arriving on three of four input lines and all need the same output line, a queue will build up. If there is insufficient memory to hold all of them, packets will be lost. Adding more memory may help up to a point that if routers have an infinite amount of memory, congestion gets worse, not better. This is because by the time packets get to the front of the queue, they have already timed out (repeatedly) and duplicates have been sent. This makes matters worse, not better—it leads to congestion collapse.

Low-bandwidth links or routers that process packets more slowly than the line rate can also become congested. In this case, the situation can be improved by directing some of the traffic away from the bottleneck to other parts of the network. Eventually, however, all regions of the network will be congested.

Approaches to Congestion Control



The most basic way to avoid congestion is to build a network that is well matched to the traffic that it carries. If there is a low-bandwidth link on the path along which most traffic is directed, congestion is likely. Sometimes resources can be added dynamically when there is serious congestion, for example, turning on spare routers or enabling lines that are normally used only as backups (to make the system fault tolerant) or purchasing bandwidth on the open market. This is called **provisioning** and happens on a time scale of months, driven by long-term traffic trends.

Some local radio stations have helicopters flying around their cities to report on road congestion to make it possible for their mobile listeners to route their packets (cars) around hotspots. This is called **traffic-aware routing**. Splitting traffic across multiple paths is also helpful.

Sometimes it is not possible to increase capacity. The only way then to beat back the congestion is to decrease the load. In a virtual-circuit network, new connections can be refused if they would cause the network to become congested. This is called **admission control**.

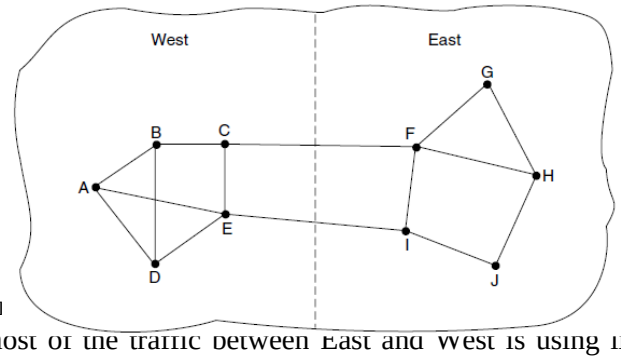
At a finer granularity, when congestion is imminent the network can deliver feedback to the sources whose traffic flows are responsible for the problem. The network can request these sources to throttle their traffic, or it can slow down the traffic itself. Two difficulties with this approach are how to identify the onset of congestion, and how to inform the source that needs to slow down.

To tackle the first issue, routers can monitor the average load, queueing delay, or packet loss. In all cases, rising numbers indicate growing congestion. To tackle the second issue, routers must participate in a feedback loop with the sources.

Finally, when all else fails, the network is forced to discard packets that it cannot deliver. The general name for this is **load shedding**. A good policy for choosing which packets to discard can help to prevent congestion collapse.

Traffic-Aware Routing

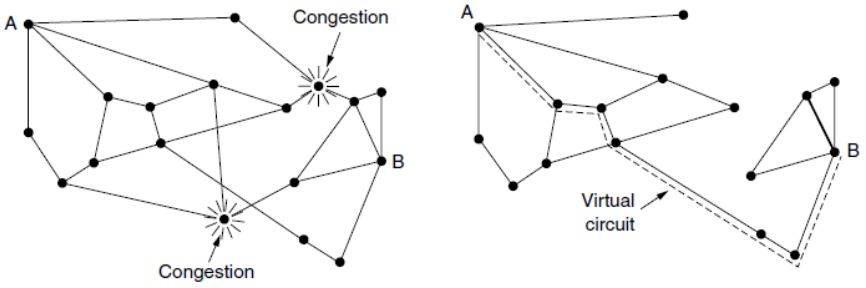
The goal in taking load into account when computing routes is to shift traffic away from hotspots that will be the first places in the network to experience congestion. The most direct way to do this is to set the link weight to be a function of the (fixed) link bandwidth and propagation delay plus the (variable) measured load or average queuing delay. Least-weight paths will then favour paths that are more lightly loaded, all else being equal.



Consider the netwo[illegible]Vest, connected by two links, *CF* and *EI*. Suppose that most of the traffic between East and West is using link *CF*, and, as a result, this link is heavily loaded with long delays. Including queuing delay in the weight used for the shortest path calculation will make *EI* more attractive. After the new routing tables have been installed, most of the East-West traffic will now go over *EI*, loading this link. Consequently, in the next update, *CF* will appear to be the shortest path. As a result, the routing tables may oscillate wildly, leading to erratic routing and many potential problems.

Admission Control

By analogy, in the telephone system, when a switch gets overloaded it practices admission control by not giving dial tones. The task is straightforward in the telephone network because of the fixed bandwidth of calls (64 kbps for uncompressed audio). However, virtual circuits in computer networks come in all shapes and sizes. Thus, the circuit must come with some characterization of its traffic if we are to apply admission control.



For example, consider the network illustrated in Fig(a). in which two routers are congested, as indicated. Suppose that a host attached to router *A* wants to set up a connection to a host attached to router *B*. Normally, this connection would pass through one of the congested routers. To avoid this situation, we can redraw the network as shown in Fig(b). omitting the congested routers and all of their lines. The dashed line shows a possible route for the virtual circuit that avoids the congested routers.

Traffic Throttling

When congestion is imminent, it must tell the senders to throttle back their transmissions and slow down. The term **congestion avoidance** is sometimes used to contrast this operating point with the one in which the network has become (overly) congested.

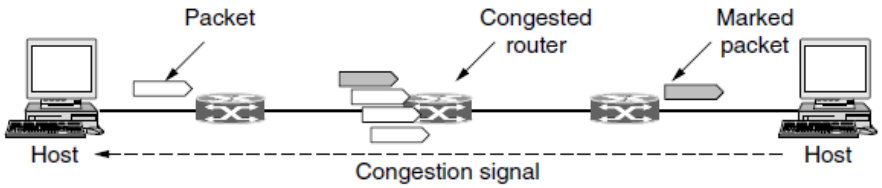
Choke Packets

The most direct way to notify a sender of congestion is to tell it directly. In this approach, the router selects a congested packet and sends a **choke packet** back to the source host, giving it the destination found in the packet. The original packet may be tagged (a header bit is turned on) so that it will not generate any more choke packets farther along the path and then forwarded in the usual way. To avoid increasing load on the network during a time of congestion, the router may only send choke packets at a low rate.

When the source host gets the choke packet, it is required to reduce the traffic sent to the specified destination, for example, by 50%. In a datagram network, choke packets to be sent to fast senders, because they will have the most packets in the queue. The host should ignore these additional chokes for the fixed time interval until its reduction in traffic takes effect. After that period, further choke packets indicate that the network is still congested.

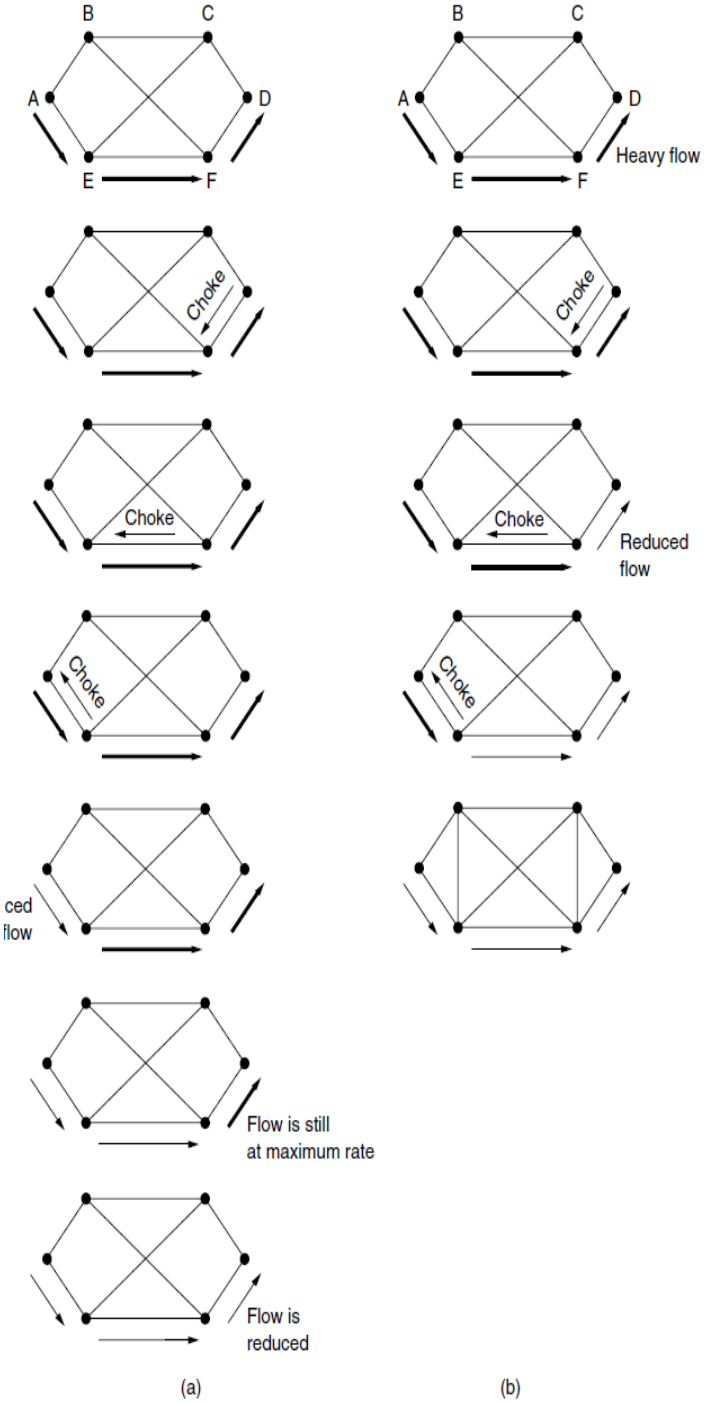
Explicit Congestion Notification

Instead of generating additional packets to warn of congestion, a router can tag any packet it forwards (by setting a bit in the packet's header) to signal that it is experiencing congestion. When the network delivers the packet, the destination can note that there is congestion and inform the sender when it sends a reply packet. The sender can then throttle its transmissions as before. This design is called **ECN** (**Explicit Congestion Notification**). Packets are unmarked when they are sent, as illustrated in Fig. If any of the routers they pass through is congested, that router will then mark the packet as having experienced congestion as it is forwarded. The destination will then echo any marks back to the sender as an explicit congestion signal in its next reply packet.



Hop-by-Hop Backpressure

At high speeds or over long distances, many new packets may be transmitted after congestion has been signalled because of the delay before the signal takes effect. Consider, for example, a host in San Francisco (router *A* in Fig.) that is sending traffic to a host in New York (router *D* in Fig.) at the speed of 155 Mbps. If the New York host begins to run out of buffers, it will take about 40 msec for a choke packet to get back to San Francisco to tell it to slow down. An ECN indication will take even longer because it is delivered via the destination.



Choke packet propagation is illustrated as the second, third, and fourth steps in Fig(a). In those 40 msec, another 6.2 megabits will have been sent. Even if the host in San Francisco completely shuts down immediately, the 6.2 megabits in the pipe will continue to pour in and have to be dealt with. Only in the seventh diagram in Fig(a) will the New York router notice a slower flow. An alternative approach is to have the choke packet take effect at every hop it passes through, as shown in the sequence of Fig.(b). Here, as soon as the choke packet reaches *F*, *F* is required to reduce the flow to *D*. Doing so will require *F* to devote more buffers to the connection, since the source is still sending away at full blast, but it gives *D* immediate relief. In the next step, the choke packet reaches *E*, which tells *E* to reduce the flow to *F*. This action puts a greater demand on *E*'s buffers but gives *F* immediate relief. Finally, the choke packet reaches *A* and the flow genuinely slows down. The net effect of this hop-by-hop scheme is to provide quick relief at the point of congestion, at the price of using up more buffers upstream. In this way, congestion can be nipped in the bud without losing any packets.

Load Shedding

Load shedding is a fancy way of saying that when routers are being inundated by packets that they cannot handle, they just throw them away. The key question for a router drowning in packets is which packets to drop. The preferred choice may depend on the type of applications that use the network. For a file transfer, an old packet is worth more than a new one. This is because dropping packet 6 and keeping packets 7 through 10, for example, will only force the receiver to do more work to buffer data that it cannot yet use. In

contrast, for real-time media, a new packet is worth more than an old one. This is because packets become useless if they are delayed and miss the time at which they must be played out to the user.

Random Early Detection

A popular algorithm for doing this is called **RED** (**Random Early Detection**). To determine when to start discarding, routers maintain a running average of their queue lengths. When the average queue length on some link exceeds a threshold, the link is said to be congested and a small fraction of the packets are dropped at random. Picking packets at random makes it more likely that the fastest senders will see a packet drop; this is the best option since the router cannot tell which source is causing the most trouble in a datagram network. The affected sender will notice the loss when there is no acknowledgement, and then the transport protocol will slow down. The lost packet is thus delivering the same message as a choke packet, but implicitly, without the router sending any explicit signal. RED routers improve performance compared to routers that drop packets only when their buffers are full, though they may require tuning to work well. For example, the ideal number of packets to drop depends on how many senders need to be notified of congestion. However, ECN is the preferred option if it is available. It works in exactly the same manner, but delivers a congestion signal explicitly rather than as a loss; RED is used when hosts cannot receive explicit signals.

QUALITY OF SERVICE:

An easy solution to provide good quality of service is to build a network with enough capacity for whatever traffic will be thrown at it. The name for this solution is **over provisioning**. The resulting network will carry application traffic without significant loss and, assuming a decent routing scheme, will deliver packets with low latency. Performance doesn't get any better than this.

To some extent, the telephone system is over provisioned because it is rare to pick up a telephone and not get a dial tone instantly. There is simply so much capacity available that demand can almost always be met. The trouble with this solution is that it is expensive.

Four issues must be addressed to ensure quality of service:

1. What applications need from the network?
2. How to regulate the traffic that enters the network.
3. How to reserve resources at routers to guarantee performance.
4. Whether the network can safely accept more traffic.

No single technique deals efficiently with all these issues. Instead, a variety of techniques have been developed for use at the network (and transport) layer. Practical quality-of-service solutions combine multiple techniques. To this end, we will describe two versions of quality of service for the Internet called Integrated Services and Differentiated Services.

APPLICATION REQUIREMENTS:

A stream of packets from a source to a destination is called a **flow.** A flow might be all the packets of a connection in a connection-oriented network, or all the packets sent from one process to another process in a connectionless network. The needs of each flow can be characterized by four primary parameters: ***bandwidth***, ***delay***, ***jitter***, and ***loss***. Together, these determine the **QoS** (**Quality of Service**) the flow requires.

Several common applications and the stringency (meaning toughness/flexibility) of their network requirements are listed in Fig. 3.11. The applications differ in their bandwidth needs, with email, audio in all forms, and remote login not needing much, but file sharing and video in all forms needing a great deal.

More interesting are the delay requirements. File transfer applications, including email and video, are not delay sensitive. If all packets are delayed uniformly by a few seconds, no harm is done.

Interactive applications, such as Web surfing and remote login, are more delay sensitive. Real-time applications, such as telephony and videoconferencing, have strict delay requirements. If all the words in a telephone call are each delayed by too long, the users will find the connection unacceptable. On the other hand, playing audio or video files from a server does not require low delay.

The variation (i.e., standard deviation) in the delay or packet arrival times is called **jitter**. The first three applications in Fig. 3.11 are not sensitive to the packets arriving with irregular time intervals between them. Remote login is somewhat sensitive to that, since updates on the screen will appear in little bursts if the connection suffers much jitter.

Video and especially audio are extremely sensitive to jitter. If a user is watching a video over the network and the frames are all delayed by exactly 2.000 seconds, no harm is done. But if the transmission time varies randomly between 1 and 2 seconds, the result will be terrible unless the application hides the jitter. For audio, a jitter of even a few milliseconds is clearly audible.

Application	Bandwidth	Delay	Jitter	Loss
Email	Low	Low	Low	Medium
File sharing	High	Low	Low	Medium
Web access	Medium	Medium	Low	Medium
Remote login	Low	Medium	Medium	Medium
Audio on demand	Low	Low	High	Low
Video on demand	High	Low	High	Low
Telephony	Low	High	High	Low
Videoconferencing	High	High	High	Low

FIGURE 3.11: STRINGENCY OF APPLICATIONS' QUALITY-OF-SERVICE REQUIREMENTS

To accommodate a variety of applications, networks may support different categories of QoS. An influential example comes from ATM networks. They support:

1. Constant bit rate (e.g., telephony).
2. Real-time variable bit rate (e.g., compressed videoconferencing).
3. Non-real-time variable bit rate (e.g., watching a movie on demand).
4. Available bit rate (e.g., file transfer).

These categories are also useful for other purposes and other networks.

TRAFFIC SHAPING: Before the network can make QoS guarantees, it must know what traffic is being guaranteed. In the telephone network, this characterization is simple. For example, a voice call (in uncompressed format) needs 64 kbps and consists of one 8-bit sample every 125 μsec.

However, traffic in data networks is **bursty**. It typically arrives at nonuniform rates as the traffic rate varies (e.g., videoconferencing with compression), users interact with applications (e.g., browsing a new Web page), and computers switch between tasks. Bursts of traffic are more difficult to handle than constant-rate traffic because they can fill buffers and cause packets to be lost.

Traffic shaping is a technique for regulating the average rate and burstiness of a flow of data that enters the network. The goal is to allow applications to transmit a wide variety of traffic that suits their needs, including some bursts, yet have a simple and useful way to describe the possible traffic patterns to the network.

When a flow is set up, the user and the network (i.e., the customer and the provider) agree on a certain traffic pattern (i.e., shape) for that flow. In effect, the customer says to the provider ''my transmission pattern will look like this; can you handle it?''

Sometimes this agreement is called an **SLA** (**Service Level Agreement**), especially when it is made over aggregate flows and long periods of time, such as all of the traffic for a given customer. As long as the customer fulfills her part of the bargain and only sends packets according to the agreed-on contract, the provider promises to deliver them all in a timely fashion.

Traffic shaping reduces congestion and thus helps the network live up to its promise. However, to make it work, there is also the issue of how the provider can tell if the customer is following the agreement and what to do if the customer is not. Packets in excess of the agreed pattern might be dropped by the network, or they might be marked as having lower priority. Monitoring a traffic flow is called **traffic policing**.

PACKET SCHEDULING:

Being able to regulate the shape of the offered traffic is a good start. However, to provide a performance guarantee, we must reserve sufficient resources along the route that the packets take through the network. To do this, we are assuming that the packets of a flow follow the same route. Spraying them over routers at random makes it hard to guarantee anything. As a consequence, something similar to a virtual circuit has to be set up from the source to the destination, and all the packets that belong to the flow must follow this route.

Algorithms that allocate router resources among the packets of a flow and between competing flows are called **packet scheduling algorithms**. Three different kinds of resources can potentially be reserved for different flows:

1. Bandwidth.
2. Buffer space.
3. CPU cycles.

The first one, bandwidth, is the most obvious. If a flow requires 1 Mbps and the outgoing line has a capacity of 2 Mbps, trying to direct three flows through that line is not going to work. Thus, reserving bandwidth means not oversubscribing any output line.

A second resource that is often in short supply is buffer space. When a packet arrives, it is buffered inside the router until it can be transmitted on the chosen outgoing line. The purpose of the buffer is to absorb small bursts of traffic as the flows contend with each other.
If no buffer is available, the packet has to be discarded since there is no place to put it. For good quality of service, some buffers might be reserved for a specific flow so that flow does not have to compete for buffers with other flows. Up to some maximum value, there will always be a buffer available when the flow needs one.
Finally, CPU cycles may also be a scarce resource. It takes router CPU time to process a packet, so a router can process only a certain number of packets per second. While modern routers are able to process most packets quickly, some kinds of packets require greater CPU processing, such as the ICMP packets. Making sure that the CPU is not overloaded is needed to ensure timely processing of these packets.

How Networks Can Be Connected

Networks can be interconnected by different devices.

- In the ***physical layer***, networks can be connected by *repeaters* or *hubs*, which just move the bits from one network to an identical network. These are mostly analog devices and do not understand anything about digital protocols.
- In the ***Data Link Layer***, networks are connected by *bridges* and *switches.* They can accept frames, examine the MAC addresses, and forward the frames to a different network while doing minor protocol translation in the process.
- In the ***network layer***, we have *routers* that can connect two networks. If two networks have dissimilar network layers, the router may be able to translate between the packet formats, although packet translation is now increasingly rare. A router that can handle multiple protocols is called a **multiprotocol router**.
- In the ***transport layer*** we find transport gateways, which can interface between two transport connections. Transport gateway has a different transport protocol, by essentially gluing one connection to another connection.
- Finally, in the application layer, application gateways translate message semantics.

Here, we will focus on internetworking in the network layer. To see how that differs from switching in the data link layer, examine Fig. 5-44. In Fig. 5-44(a), the source machine, *S*, wants to send a packet to the destination machine, *D*. These machines are on different Ethernets, connected by a switch. *S* encapsulates the packet in a frame and sends it on its way. The frame arrives at the switch, which then determines that the frame has to go to LAN 2 by looking at its MAC address. The switch just removes the frame from LAN 1 and deposits it on LAN 2.

INTERNETWORKING:

HOW NETWORKS DIFFER:

Networks can differ in many ways. Some of the differences, such as different modulation techniques or frame formats, are internal to the physical and data link layers. These differences will not concern us here. Instead, in Fig. 3.12 we list some of the differences that can be exposed to the network layer. It is papering over these differences that makes internetworking more difficult than operating within a single network.

When packets sent by a source on one network must transit one or more foreign networks before reaching the destination network, many problems can occur at the interfaces between networks. To start with, the source needs to be able to address the destination.

What do we do if the source is on an Ethernet network and the destination is on a WiMAX network? Assuming we can even specify a WiMAX destination from an Ethernet network, packets would cross from a connectionless network to a connection-oriented one.

This may require that a new connection be set up on short notice, which injects a delay, and much overhead if the connection is not used for many more packets. Many specific differences may have to be accommodated as well. How do we multicast a packet to a group with some members on a network that does not support multicast?

The differing max packet sizes used by different networks can be a major nuisance, too. How do you pass an 8000-byte packet through a network whose maximum size is 1500 bytes? If packets on a connection-oriented network transit a connectionless network, they may arrive in a different order than they were sent. That is something the sender likely did not expect, and it might come as an (unpleasant) surprise to the receiver as well.

Item	Some Possibilities
Service offered	Connectionless versus connection oriented
Addressing	Different sizes, flat or hierarchical
Broadcasting	Present or absent (also multicast)
Packet size	Every network has its own maximum
Ordering	Ordered and unordered delivery
Quality of service	Present or absent; many different kinds
Reliability	Different levels of loss
Security	Privacy rules, encryption, etc.
Parameters	Different timeouts, flow specifications, etc.
Accounting	By connect time, packet, byte, or not at all

FIGURE 3.12: SOME OF THE MANY WAYS NETWORKS CAN DIFFER.

How Networks Can Be Connected

There are two basic choices for connecting different networks: we can build devices that translate or convert packets from each kind of network into packets for each other network, or, like good computer scientists, we can try to solve the problem by adding a layer of indirection and building a common layer on top of the different networks. In either case, the devices are placed at the boundaries between networks.

Internetworking has been very successful at building large networks, but it only works when there is a common network layer. There have, in fact, been many network protocols over time. Getting everybody to agree on a single format is difficult when companies perceive it to their commercial advantage to have a proprietary format that they control.

A router that can handle multiple network protocols is called a **multiprotocol router**. It must either translate the protocols, or leave connection for a higher protocol layer. Neither approach is entirely satisfactory. Connection at a higher layer, say, by using TCP, requires that all the networks implement TCP (which may not be the case). Then, it limits usage across the networks to applications that use TCP (which does not include many real-time applications).

TUNNELING:

Handling the general case of making two different networks interwork is exceedingly difficult. However, there is a common special case that is manageable even for different network protocols. This case is where the source and destination hosts are on the same type of network, but there is a different network in between. As an example, think of an international bank with an IPv6 network in Paris, an IPv6 network in London and connectivity between the offices via the IPv4 Internet. This situation is shown in Fig. 3.13.

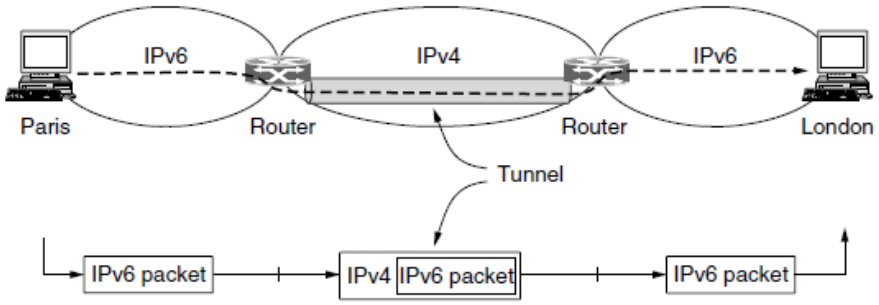


FIGURE 3.13: TUNNELING A PACKET FROM PARIS TO LONDON

The solution to this problem is a technique called **tunneling**. To send an IP packet to a host in the London office, a host in the Paris office constructs the packet

containing an IPv6 address in London, and sends it to the multiprotocol router that connects the Paris IPv6 network to the IPv4 Internet.

When this router gets the IPv6 packet, it encapsulates the packet with an IPv4 header addressed to the IPv4 side of the multiprotocol router that connects to the London IPv6 network.

That is, the router puts a (IPv6) packet inside a (IPv4) packet. When this wrapped packet arrives, the London router removes the original IPv6 packet and sends it onward to the destination host. The path through the IPv4 Internet can be seen as a big tunnel extending from one multiprotocol router to the other.

The IPv6 packet just travels from one end of the tunnel to the other, snug in its nice box. It does not have to worry about dealing with IPv4 at all. Neither do the hosts in Paris or London. Only the multiprotocol routers have to understand both IPv4 and IPv6 packets.

In effect, the entire trip from one multiprotocol router to the other is like a hop over a single link. Tunneling is widely used to connect isolated hosts and networks using other networks.

INTERNETWORK ROUTING:

Routing through an internet poses the same basic problem as routing within a single network, but with some added complications. To start, the networks may internally use different routing algorithms. For example, one network may use link state routing and another distance vector routing. Since link state algorithms need to know the topology but distance vector algorithms do not, this difference alone would make it unclear how to find the shortest paths across the internet.

Networks run by different operators lead to bigger problems. First, the operators may have different ideas about what is a good path through the network. One operator may want the route with the least delay, while another may want the most inexpensive route. This will lead the operators to use different quantities to set the shortest-path costs.

Finally, the internet may be much larger than any of the networks that comprise it. It may therefore require routing algorithms that scale well by using a hierarchy, even if none of the individual networks need to use a hierarchy.

All of these considerations lead to a two-level routing algorithm. Within each network, an **intradomain** or **interior gateway protocol** is used for routing. (''Gateway'' is an older term for ''router.'') It might be a link state protocol of the Kind.

Across the networks that make up the internet, an **interdomain** or **exterior gateway protocol** is used. The networks may all use different intradomain protocols, but they must use the same interdomain protocol.

*In the Internet, the interdomain routing protocol is called **BGP** (**Border Gateway Protocol**).*

There is one more important term to introduce. Since each network is operated independently of all the others, it is often referred to as an **AS** (**Autonomous System**). A good mental model for an AS is an ISP network. In fact, an ISP network may be comprised of more than one AS, if it is managed, or, has been acquired, as multiple networks. But the difference is usually not significant.

PACKET FRAGMENTATION: Each network or link imposes some maximum size on its packets. These limits have various causes, among them:

1. Hardware (e.g., the size of an Ethernet frame).
2. Operating system (e.g., all buffers are 512 bytes).
3. Protocols (e.g., the number of bits in the packet length field).
4. Compliance with some (inter)national standard.
5. Desire to reduce error-induced retransmissions to some level.
6. Desire to prevent one packet from occupying the channel too long.

The result of all these factors is that the network designers are not free to choose any old maximum packet size they wish. Maximum payloads for some common technologies are 1500 bytes for Ethernet and 2272 bytes for 802.11. IP is more generous, allows for packets as big as 65,515 bytes.

Hosts usually prefer to transmit large packets because this reduces packet overheads such as bandwidth wasted on header bytes. An obvious internetworking problem appears when a large packet wants to travel through a network whose maximum packet size is too small. This nuisance has been a persistent issue, and solutions to it have evolved along with much experience gained on the Internet.

One solution is to make sure the problem does not occur in the first place. However, this is easier said than done. A source does not usually know the path a packet will take through the network to a destination, so it certainly does not know how small packets must be to get there. This packet size is called the **Path MTU** (**Path Maximum Transmission Unit**).

The alternative solution to the problem is to allow routers to break up packets into **fragments**, sending each fragment as a separate network layer

packet. However, as every parent of a small child knows, converting a large object into small fragments is considerably easier than the reverse process.

THE NETWORK LAYER IN THE INTERNET

THE IP VERSION 4 PROTOCOL:

An appropriate place to start our study of the network layer in the Internet is with the format of the IP datagrams themselves. An IPv4 datagram consists of a header part and a body or payload part. The header has a 20-byte fixed part and a variable-length optional part. The header format is shown in Fig. 3.14. The bits are transmitted from left to right and top to bottom, with the high-order bit of the *Version* field going first. (This is a ‘‘big-endian’’ network byte order.

On little-endian machines, such as Intel x86 computers, a software conversion is required on both transmission and reception.) In retrospect, little-endian would have been a better choice, but at the time IP was designed, no one knew it would come to dominate computing.

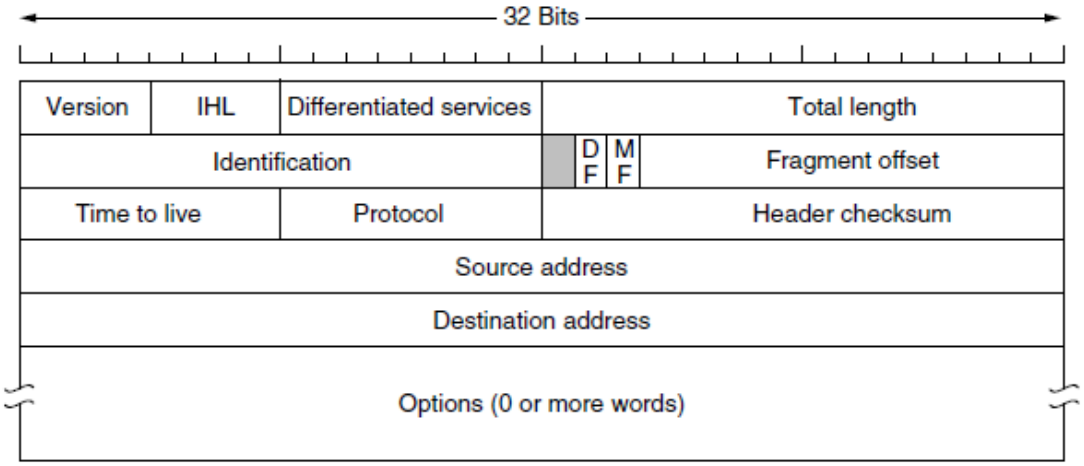


FIGURE 3.14: THE IPV4 (INTERNET PROTOCOL) HEADER

The *Version* field keeps track of which version of the protocol the datagram belongs to.

Since the header length is not constant, a field in the header, *IHL*, is provided to tell how long the header is, in 32-bit words. The minimum value is 5, which applies when no options are present. The maximum value of this 4-bit field is 15, which limits the header to 60 bytes, and thus the *Options* field to 40 bytes.

The *Differentiated services* field is one of the few fields that have changed its meaning (slightly) over the years. Originally, it was called the *Type of service* field. Various combinations of reliability and speed are possible. For digitized voice, fast delivery beats accurate delivery.

For file transfer, error-free transmission is more important than fast transmission. The *Type of service* field provided 3 bits to signal priority and 3 bits to signal whether a host cared more about delay, throughput, or reliability.

The *Total length* includes everything in the datagram—both header and data. The maximum length is 65,535 bytes. At present, this upper limit is tolerable, but with future networks, larger datagrams may be needed.

The *Identification* field is needed to allow the destination host to determine which packet a newly arrived fragment belongs to. All the fragments of a packet contain the same *Identification* value.

DF stands for Don't Fragment. It is an order to the routers not to fragment the packet. Originally, it was intended to support hosts incapable of putting the pieces back together again.

MF stands for More Fragments. All fragments except the last one have this bit set. It is needed to know when all fragments of a datagram have arrived.

The *Fragment offset* tells where in the current packet this fragment belongs. All fragments except the last one in a datagram must be a multiple of 8 bytes, the elementary fragment unit. Since 13 bits are provided, there is a maximum of 8192 fragments per datagram, supporting a maximum packet length up to the limit of the *Total length* field. Working together, the *Identification*, *MF*, and *Fragment offset* fields are used to implement fragmentation.

The *TtL (Time to live)* field is a counter used to limit packet lifetimes. It was originally supposed to count time in seconds, allowing a maximum lifetime of 255 sec.

When the network layer has assembled a complete packet, it needs to know what to do with it. The *Protocol* field tells it which transport process to give the packet to. TCP is one possibility, but so are UDP and some others.

Since the header carries vital information such as addresses, it rates its own checksum for protection, the *Header checksum*. The algorithm is to add up all the 16-bit halfwords of the header as they arrive, using one's complement arithmetic, and then take the one's complement of the result. For purposes of this algorithm, the *Header checksum* is assumed to be zero upon arrival. Such a checksum is useful for detecting errors while the packet travels through the network.

The *Source address* and *Destination address* indicate the IP address of the source and destination network interfaces.

The *Options* field was designed to provide an escape to allow subsequent versions of the protocol to include information not present in the original design, to permit experimenters to try out new ideas, and to avoid allocating header bits to

information that is rarely needed. The options are of variable length. The *Options* field is padded out to a multiple of 4 bytes. Originally, the five options listed in Fig. 3.15.

Option	Description
Security	Specifies how secret the datagram is
Strict source routing	Gives the complete path to be followed
Loose source routing	Gives a list of routers not to be missed
Record route	Makes each router append its IP address
Timestamp	Makes each router append its address and timestamp

FIGURE 3.15: SOME OF THE IP OPTIONS

IPV4 ADDRESSES:

The identifier used in the IP layer of the TCP/IP protocol suite to identify the connection of each device to the Internet is called the Internet address or IP address. An IPv4 address is a 32-bit address that uniquely and universally defines the connection of a host or a router to the Internet. The IP address is the address of the connection, not the host or the router, because if the device is moved to another network, the IP address may be changed.

IPv4 addresses are unique in the sense that each address defines one, and only one, connection to the Internet. If a device has two connections to the Internet, via two networks, it has two IPv4 addresses. IPv4 addresses are universal in the sense that the addressing system must be accepted by any host that wants to be connected to the Internet.

Address Space

A protocol like IPv4 that defines addresses has an address space. An **address space** is the total number of addresses used by the protocol. If a protocol uses *b* bits to define an address, the address space is 2^b because each bit can have two different values (0 or 1). IPv4 uses 32-bit addresses, which means that the address space is 232 or 4,294,967,296 (more than four billion). If there were no restrictions, more than 4 billion devices could be connected to the Internet.

Notation

There are three common notations to show an IPv4 address: binary notation (base 2), dotted-decimal notation (base 256), and hexadecimal notation (base 16). In *binary notation,* an IPv4 address is displayed as 32 bits. To make the address more readable, one or more spaces are usually inserted between each octet (8 bits). Each octet is often referred to as a byte. To make the IPv4 address more compact and easier to read, it is usually written in decimal form with a decimal point (dot) separating the bytes.

This format is referred to as *dotted-decimal notation*. Note that because each byte (octet) is only 8 bits, each number in the dotted-decimal notation is between 0 and 255. We sometimes see an IPv4 address in hexadecimal notation. Each hexadecimal digit is equivalent to four bits. This means that a 32-bit address has 8 hexadecimal digits. This notation is often used in network programming. Figure 3.16 shows an IP address in the three discussed notations.

HIERARCHY IN ADDRESSING: A 32-bit IPv4 address is also hierarchical, but divided only into two parts. The first part of the address, called the *prefix*, defines the network; the second part of the address, called the *suffix*, defines the node (connection of a device to the Internet).

Figure 3.17 shows the prefix and suffix of a 32-bit IPv4 address. The prefix length is *n* bits and the suffix length is (32 − *n*) bits.

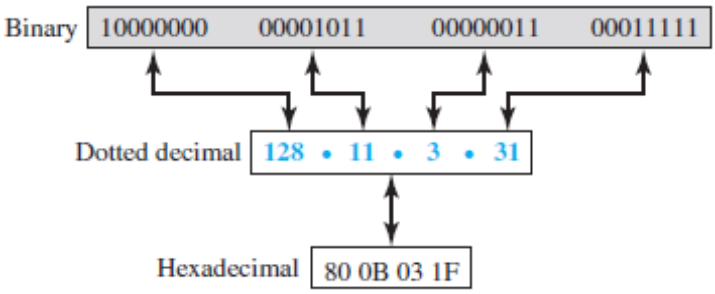


FIGURE 3.16: THREE DIFFERENT NOTATIONS IN IPV4 ADDRESSING

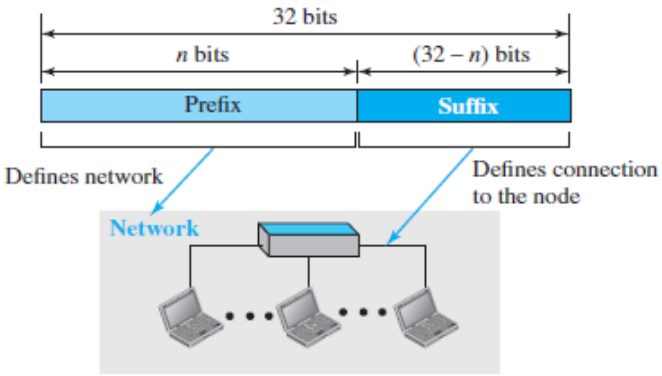


FIGURE 3.17: HIERARCHY IN ADDRESSING

A prefix can be fixed length or variable length. The network identifier in the IPv4 was first designed as a fixed-length prefix. This scheme, which is now obsolete, is referred to as classful addressing. The new scheme, which is referred to as classless addressing, uses a variable-length network prefix. First, we briefly discuss Classful addressing; then we concentrate on classless addressing.

Classful Addressing:

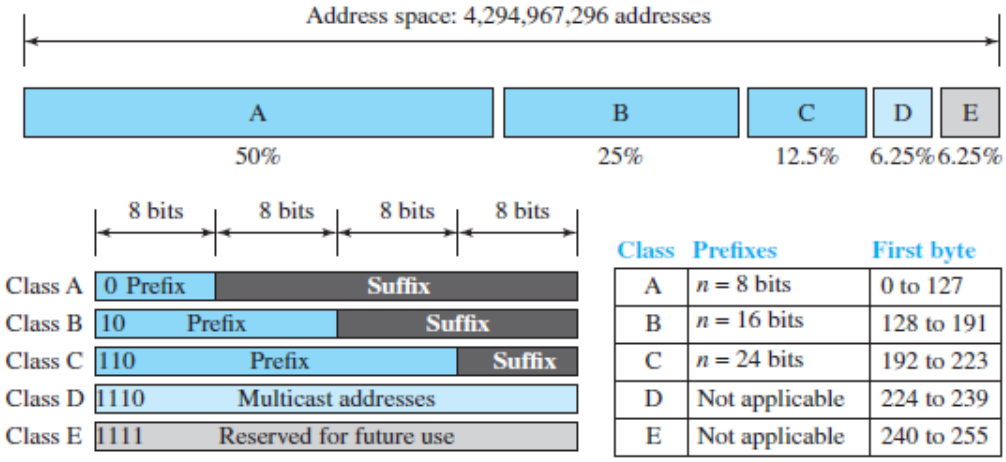
When the Internet started, an IPv4 address was designed with a fixed-length prefix, but to accommodate both small and large networks, three fixed-length

prefixes were designed instead of one (n = 8, n = 16, and n = 24). The whole address space was divided into five classes (class A, B, C, D, and E), as shown in Figure 3.18. This scheme is referred to as **classful addressing.**

In class A, the network length is 8 bits, but since the first bit, which is 0, defines the class, we can have only seven bits as the network identifier. This means there are only 27 = 128 networks in the world that can have a class A address.

In class B, the network length is 16 bits, but since the first two bits, which are (10)2, define the class, we can have only 14 bits as the network identifier. This means there are only 214 = 16,384 networks in the world that can have a class B address.

All addresses that start with (110)2 belong to class C. In class C, the network length is 24 bits, but since three bits define the class, we can have only 21 bits as the network identifier. This means there are 221 = 2,097,152 networks in the world that can have a class C address.



Class	Prefixes	First byte
A	n = 8 bits	0 to 127
B	n = 16 bits	128 to 191
C	n = 24 bits	192 to 223
D	Not applicable	224 to 239
E	Not applicable	240 to 255

FIGURE 3.18: OCCUPATION OF THE ADDRESS SPACE IN CLASSFUL ADDRESSING

Class D is not divided into prefix and suffix. It is used for multicast addresses. All addresses that start with 1111 in binary belong to class E. As in Class D, Class E is not divided into prefix and suffix and is used as reserve.

Advantage of Classful Addressing:

Although classful addressing had several problems and became obsolete, it had one advantage: Given an address, we can easily find the class of the address and, since the prefix length for each class is fixed, we can find the prefix length immediately. In other words, the prefix length in classful addressing is inherent in the address; no extra information is needed to extract the prefix and the suffix.

Address Depletion: The reason that classful addressing has become obsolete is address depletion. Since the addresses were not distributed properly,

the Internet was faced with the problem of the addresses being rapidly used up, resulting in no more addresses available for organizations and individuals that needed to be connected to the Internet.

Subnetting and Supernetting: To alleviate address depletion, two strategies were proposed and, to some extent, implemented: subnetting and Supernetting. In subnetting, a class A or class B block is divided into several subnets.

Each subnet has a larger prefix length than the original network. While subnetting was devised to divide a large block into smaller ones, Supernetting was devised to combine several class C blocks into a larger block to be attractive to organizations that need more than the 256 addresses available in a class C block. This idea did not work either because it makes the routing of packets more difficult.

Classless Addressing:

Subnetting and Supernetting in classful addressing did not really solve the address depletion problem. With the growth of the Internet, it was clear that a larger address space was needed as a long-term solution. The larger address space, however, requires that the length of IP addresses also be increased, which means the format of the IP packets needs to be changed.

Although the long-range solution has already been devised and is called IPv6, a short-term solution was also devised to use the same address space but to change the distribution of addresses to provide a fair share to each organization. The short-term solution still uses IPv4 addresses, but it is called *classless addressing*. In other words, the class privilege was removed from the distribution to compensate for the address depletion.

In classless addressing, the whole address space is divided into variable length blocks. The prefix in an address defines the block (network); the suffix defines the node (device). Theoretically, we can have a block of 20, 21, 22, . . . , 232 addresses. One of the restrictions, as we discuss later, is that the number of addresses in a block needs to be a power of 2. An organization can be granted one block of addresses. Figure 3.19 shows the division of the whole address space into nonoverlapping blocks.

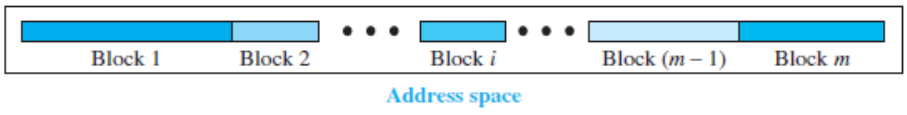


FIGURE 3.19: VARIABLE-LENGTH BLOCKS IN CLASSLESS ADDRESSING

Unlike classful addressing, the prefix length in classless addressing is variable. We can have a prefix length that ranges from 0 to 32. The size of the

network is inversely proportional to the length of the prefix. A small prefix means a larger network; a large prefix means a smaller network.

We need to emphasize that the idea of classless addressing can be easily applied to classful addressing. An address in class A can be thought of as a classless address in which the prefix length is 8. An address in class B can be thought of as a classless address in which the prefix is 16, and so on. In other words, classful addressing is a special case of classless addressing.

Prefix Length: Slash Notation:

The first question that we need to answer in classless addressing is how to find the prefix length if an address is given. Since the prefix length is not inherent in the address, we need to separately give the length of the prefix. In this case, the prefix length, *n*, is added to the address, separated by a slash. The notation is informally referred to as *slash notation* and formally as ***classless interdomain routing*** or ***CIDR*** (pronounced cider) strategy. An address in classless addressing can then be represented as shown in Figure 3.20.

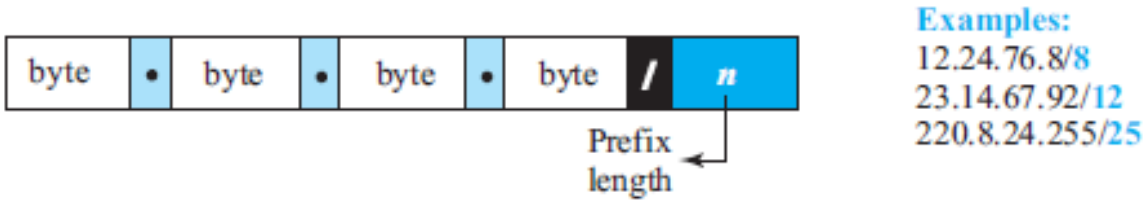


FIGURE 3.20: SLASH NOTATION (CIDR)

Extracting Information from an Address:

Given any address in the block, we normally like to know three pieces of information about the block to which the address belongs: the number of addresses, the first address in the block, and the last address. Since the value of prefix length, *n*, is given, we can easily find these three pieces of information, as shown in Figure 3.21.

1. The number of addresses in the block is found as $N = 2^{32-n}$.
2. To find the first address, we keep the *n* leftmost bits and set the (32 − *n*) rightmost bits all to 0s.
3. To find the last address, we keep the *n* leftmost bits and set the (32 − *n*) rightmost bits all to 1s.

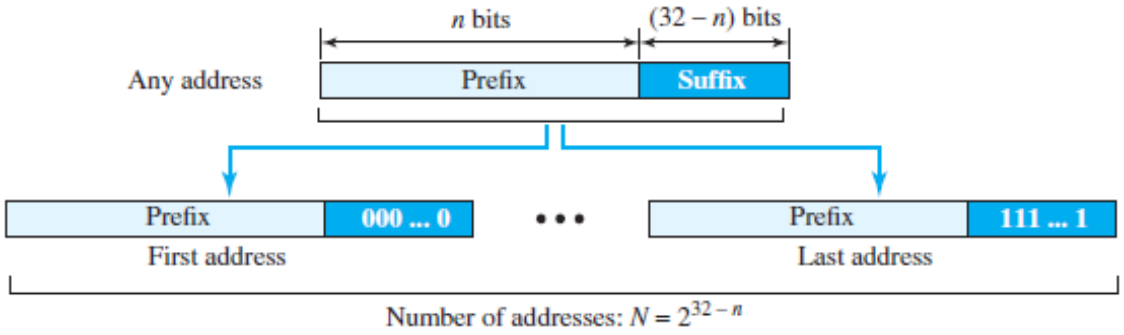


FIGURE 3.21: INFORMATION EXTRACTION IN CLASSLESS ADDRESSING

Example:

A classless address is given as 167.199.170.82/**27**. We can find the above three pieces of information as follows. The number of addresses in the network is 2^{32-n} = 25 = 32 addresses.

The first address can be found by keeping the first 27 bits and changing the rest of the bits to 0s.

Address: 167.199.170.82/**27** **10100111 11000111 10101010 01010010**

First address: 167.199.170.64/**27** **10100111 11000111 10101010 01000000**

The last address can be found by keeping the first 27 bits and changing the rest of the bits to 1s.

Address: 167.199.170.82/**27** **10100111 11000111 10101010 01011111**

Last address: 167.199.170.95/**27** **10100111 11000111 10101010 01011111**

IP VERSION 6:

IP has been in heavy use for decades. It has worked extremely well, as demonstrated by the exponential growth of the Internet. Unfortunately, IP has become a victim of its own popularity: it is close to running out of addresses. Even with CIDR and NAT using addresses more sparingly, the last IPv4 addresses are expected to be assigned by ICANN before the end of 2012.

IPv6 (**IP version 6**) is a replacement design that does just that. It uses 128-bit addresses; a shortage of these addresses is not likely any time in the foreseeable future. However, IPv6 has proved very difficult to deploy. It is a different network layer protocol that does not really interwork with IPv4, despite

many similarities. Also, companies and users are not really sure why they should want IPv6 in any case.

In 1990 IETF started work on a new version of IP, one that would never run out of addresses, would solve a variety of other problems, and be more flexible and efficient as well. Its major goals were:

1. Support billions of hosts, even with inefficient address allocation.
2. Reduce the size of the routing tables.
3. Simplify the protocol, to allow routers to process packets faster.
4. Provide better security (authentication and privacy).
5. Pay more attention to the type of service, particularly for real-time data.
6. Aid multicasting by allowing scopes to be specified.
7. Make it possible for a host to roam without changing its address.
8. Allow the protocol to evolve in the future.
9. Permit the old and new protocols to coexist for years.

The design of IPv6 presented a major opportunity to improve all of the features in IPv4 that fall short of what is now wanted. One proposal was to run TCP over CLNP, the network layer protocol designed for OSI. With its 160-bit addresses, CLNP would have provided enough address space forever.

IPv6 meets IETF's goals fairly well. It maintains the good features of IP, discards or deemphasizes the bad ones, and adds new ones where needed. In general, IPv6 is not compatible with IPv4, but it is compatible with the other auxiliary Internet protocols, including TCP, UDP, ICMP, IGMP, OSPF, BGP, and DNS, with small modifications being required to deal with longer addresses.

The main features of IPv6 are discussed below.

- First and foremost, IPv6 has longer addresses than IPv4. They are 128 bits long, which solves the problem that IPv6 set out to solve: providing an effectively unlimited supply of Internet addresses.
- The second major improvement of IPv6 is the simplification of the header. It contains only seven fields (versus 13 in IPv4). This change allows routers to process packets faster and thus improves throughput and delay.
- The third major improvement is better support for options. This change was essential with the new header because fields that

previously were required are now optional (because they are not used so often).

- In addition, the way options are represented is different, making it simple for routers to skip over options not intended for them. This feature speeds up packet processing time.

- A fourth area in which IPv6 represents a big advance is in security.
- Finally, more attention has been paid to quality of service.

The Main IPv6 Header:

The IPv6 header is shown in Fig. 3.22. The *Version* field is always 6 for IPv6 (and 4 for IPv4). During the transition period from IPv4, which has already taken more than a decade, routers will be able to examine this field to tell what kind of packet they have.

As an aside, making this test wastes a few instructions in the critical path, given that the data link header usually indicates the network protocol for demultiplexing, so some routers may skip the check.

The *Differentiated services* field (originally called *Traffic class*) is used to distinguish the class of service for packets with different real-time delivery requirements.

The *Flow label* field provides a way for a source and destination to mark groups of packets that have the same requirements and should be treated in the same way by the network, forming a pseudo connection.

The *Payload length* field tells how many bytes follow the 40-byte header of Fig. 3.22. The name was changed from the IPv4 *Total length* field because the meaning was changed slightly: the 40 header bytes are no longer counted as part of the length (as they used to be). This change means the payload can now be 65,535 bytes instead of a mere 65,515 bytes.

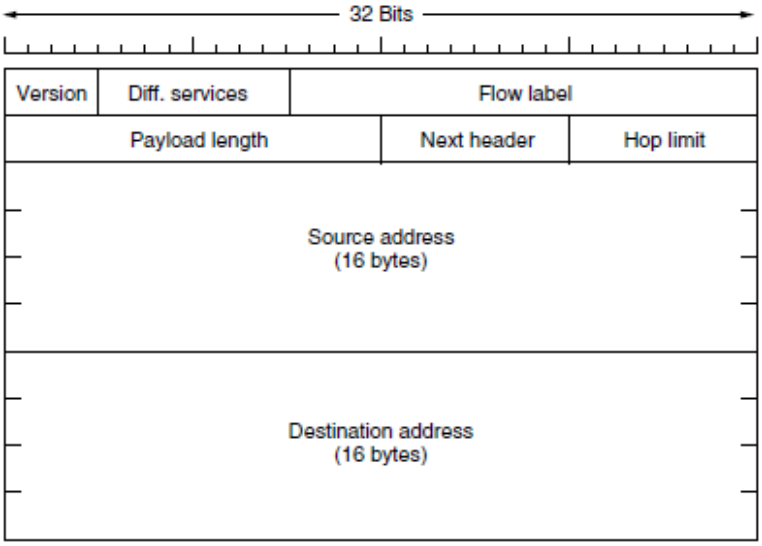


FIGURE 3.22: THE IPV6 FIXED HEADER (REQUIRED)

The *Next header* field tells which transport protocol handler (e.g., TCP, UDP) to pass the packet to.

The *Hop limit* field is used to keep packets from living forever. It is, in practice, the same as the *Time to live* field in IPv4, namely, a field that is decremented on each hop. In

Next come the *Source address* and *Destination address* fields. A new notation has been devised for writing 16-byte addresses. They are written as eight groups of four hexadecimal digits with colons between the groups, like this:

8000:0000:0000:0000:0123:4567:89AB:CDEF

Since many addresses will have many zeros inside them, three optimizations have been authorized. First, leading zeros within a group can be omitted, so 0123 can be written as 123. Second, one or more groups of 16 zero bits can be replaced by a pair of colons. Thus, the above address now becomes

8000::123:4567:89AB:CDEF

INTERNET CONTROL PROTOCOLS:

In addition to IP, which is used for data transfer, the Internet has several companion control protocols that are used in the network layer. They include ICMP, ARP, and DHCP.

ICMP—The Internet Control Message Protocol:

The operation of the Internet is monitored closely by the routers. When something unexpected occurs during packet processing at a router, the event is reported to the sender by the **ICMP** (**Internet Control Message Protocol**). ICMP is also used to test the Internet. About a dozen types of ICMP messages are

defined. Each ICMP message type is carried encapsulated in an IP packet. The most important ones are listed in Fig. 3.23.

Message type	Description
Destination unreachable	Packet could not be delivered
Time exceeded	Time to live field hit 0
Parameter problem	Invalid header field
Source quench	Choke packet
Redirect	Teach a router about geography
Echo and echo reply	Check if a machine is alive
Timestamp request/reply	Same as Echo, but with timestamp
Router advertisement/solicitation	Find a nearby router

FIGURE 3.23: THE PRINCIPAL ICMP MESSAGE TYPES

The DESTINATION UNREACHABLE message is used when the router cannot locate the destination or when a packet with the *DF* bit cannot be delivered because a ‘‘small-packet’’ network stands in the way.

The TIME EXCEEDED message is sent when a packet is dropped because its *TtL (Time to live)* counter has reached zero. This event is a symptom that packets are looping, or that the counter values are being set too low.

The PARAMETER PROBLEM message indicates that an illegal value has been detected in a header field. This problem indicates a bug in the sending host’s IP software or possibly in the software of a router transited.

The SOURCE QUENCH message was long ago used to throttle hosts that were sending too many packets. When a host received this message, it was expected to slow down.

The REDIRECT message is used when a router notices that a packet seems to be routed incorrectly. It is used by the router to tell the sending host to update to a better route.

The TIMESTAMP REQUEST and TIMESTAMP REPLY messages are similar, except that the arrival time of the message and the departure time of the reply are recorded in the reply. This facility can be used to measure network performance.

OSPF—AN INTERIOR GATEWAY ROUTING PROTOCOL:

The Internet is made up of a large number of independent networks or **ASes** (**Autonomous Systems**) that are operated by different organizations, usually a company, university, or ISP. Inside of its own network, an organization can use its own algorithm for internal routing, or **intradomain routing**, as it is more commonly known. Nevertheless, there are only a handful of standard protocols that are popular.

An intradomain routing protocol is also called an **interior gateway protocol**. We will study the problem of routing between independently operated networks, or **interdomain routing**. For that case, all networks must use the same interdomain routing protocol or **exterior gateway protocol**. The protocol that is used in the Internet is BGP (Border Gateway Protocol).

Early intradomain routing protocols used a distance vector design, based on the distributed Bellman-Ford algorithm inherited from the ARPANET. It works well in small systems, but less well as networks get larger. It also suffers from the count-to-infinity problem and generally slow convergence.

The ARPANET switched over to a link state protocol in May 1979 because of these problems, and in 1988 IETF began work on a link state protocol for intradomain routing. That protocol, called **OSPF** (**Open Shortest Path First**), became a standard in 1990. It drew on a protocol called **IS-IS** (**Intermediate-System to Intermediate-System**), which became an ISO standard.

Given the long experience with other routing protocols, the group designing OSPF had a long list of requirements that had to be met. First, the algorithm had to be published in the open literature, hence the ''O'' in OSPF.

Second, the new protocol had to support a variety of distance metrics, including physical distance, delay, and so on. Third, it had to be a dynamic algorithm, one that adapted to changes in the topology automatically and quickly.

Fourth, and new for OSPF, it had to support routing based on type of service. The new protocol had to be able to route real-time traffic one way and other traffic a different way. At the time, IP had a *Type of service* field, but no existing routing protocol used it. This field was included in OSPF but still nobody used it, and it was eventually removed.

Fifth, and related to the above, OSPF had to do load balancing, splitting the load over multiple lines. Most previous protocols sent all packets over a single best route, even if there were two routes that were equally good. The other route was not used at all. In many cases, splitting the load over multiple routes gives better performance.

Sixth, support for hierarchical systems was needed. By 1988, some networks had grown so large that no router could be expected to know the entire topology. OSPF had to be designed so that no router would have to.

OSPF supports both point-to-point links (e.g., SONET) and broadcast networks (e.g., most LANs). Actually, it is able to support networks with multiple routers, each of which can communicate directly with the others (called **multi-access networks**) even if they do not have broadcast capability. Earlier protocols did not handle this case well.

OSPF works by exchanging information between adjacent routers, which is not the same as between neighboring routers. In particular, it is inefficient to have every router on a LAN talk to every other router on the LAN. To avoid this situation, one router is elected as the **designated router**. It is said to be **adjacent** to all the other routers on its LAN, and exchanges information with them.

In effect, it is acting as the single node that represents the LAN. Neighboring routers that are not adjacent do not exchange information with each other. A backup designated router is always kept up to date to ease the transition should the primary designated router crash and need to be replaced immediately.

During normal operation, each router periodically floods LINK STATE UPDATE messages to each of its adjacent routers. These messages gives its state and provide the costs used in the topological database. The flooding messages are acknowledged, to make them reliable.

Each message has a sequence number, so a router can see whether an incoming LINK STATE UPDATE is older or newer than what it currently has. Routers also send these messages when a link goes up or down or its cost changes.

DATABASE DESCRIPTION messages give the sequence numbers of all the link state entries currently held by the sender. By comparing its own values with those of the sender, the receiver can determine who has the most recent values. These messages are used when a link is brought up.

All these messages are sent directly in IP packets. The five kinds of messages are summarized in Fig. 3.24.

Message type	Description
Hello	Used to discover who the neighbors are
Link state update	Provides the sender's costs to its neighbors
Link state ack	Acknowledges link state update
Database description	Announces which updates the sender has
Link state request	Requests information from the partner

FIGURE 3.24: THE FIVE TYPES OF OSPF MESSAGES

BGP—THE EXTERIOR GATEWAY ROUTING PROTOCOL:

Within a single AS, OSPF and IS-IS are the protocols that are commonly used. Between ASes, a different protocol, called **BGP** (**Border Gateway Protocol**), is used. A different protocol is needed because the goals of an intradomain protocol and an interdomain protocol are not the same. All an intradomain protocol has to do is move packets as efficiently as possible from the source to the destination.

BGP is a form of distance vector protocol, but it is quite unlike intradomain distance vector protocols such as RIP. We have already seen that policy, instead of minimum distance, is used to pick which routes to use. Another large difference is that instead of maintaining just the cost of the route to each destination, each BGP router keeps track of the path used. This approach is called a **path vector protocol**.

The path consists of the next hop router (which may be on the other side of the ISP, not adjacent) and the sequence of ASes, or **AS path**, that the route has followed (given in reverse order). Finally, pairs of BGP routers communicate with each other by establishing TCP connections. Operating this way provides reliable communication and also hides all the details of the network being passed through.

An example of how BGP routes are advertised is shown in Fig. 3.25. There are three ASes and the middle one is providing transit to the left and right ISPs. A route advertisement to prefix *C* starts in *AS3*. When it is propagated across the link to *R2c* at the top of the figure, it has the AS path of simply *AS3* and the next hop router of *R3a*.

At the bottom, it has the same AS path but a different next hop because it came across a different link. This advertisement continues to propagate and crosses the boundary into *AS1*. At router *R1a*, at the top of the figure, the AS path is *AS2*, *AS3* and the next hop is *R2a*.

Carrying the complete path with the route makes it easy for the receiving router to detect and break routing loops. The rule is that each router that sends a route outside of the AS prepends its own AS number to the route. (This is why the list is in reverse order.)

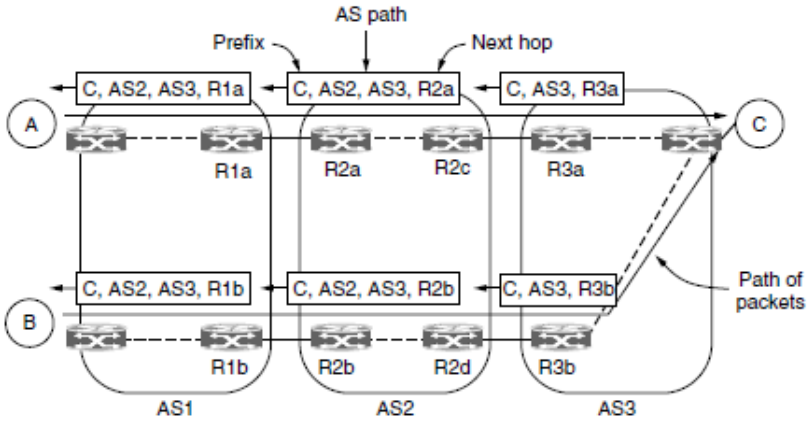


FIGURE 3.25: PROPAGATION OF BGP ROUTE ADVERTISEMENTS

When a router receives a route, it checks to see if its own AS number is already in the AS path. If it is, a loop has been detected and the advertisement is discarded.

INTERNET PROTOCOL (IP):

The network layer in version 4 can be thought of as one main protocol and three auxiliary ones. The main protocol, Internet Protocol version 4 (IPv4), is responsible for packetizing, forwarding, and delivery of a packet at the network layer.

The Internet Control Message Protocol version 4 (ICMPv4) helps IPv4 to handle some errors that may occur in the network-layer delivery. The Internet Group Management Protocol (IGMP) is used to help IPv4 in multicasting. The Address Resolution Protocol (ARP) is used to glue the network and data-link layers in mapping network-layer addresses to link-layer addresses. Figure 3.26 shows the positions of these four protocols in the TCP/IP protocol suite.

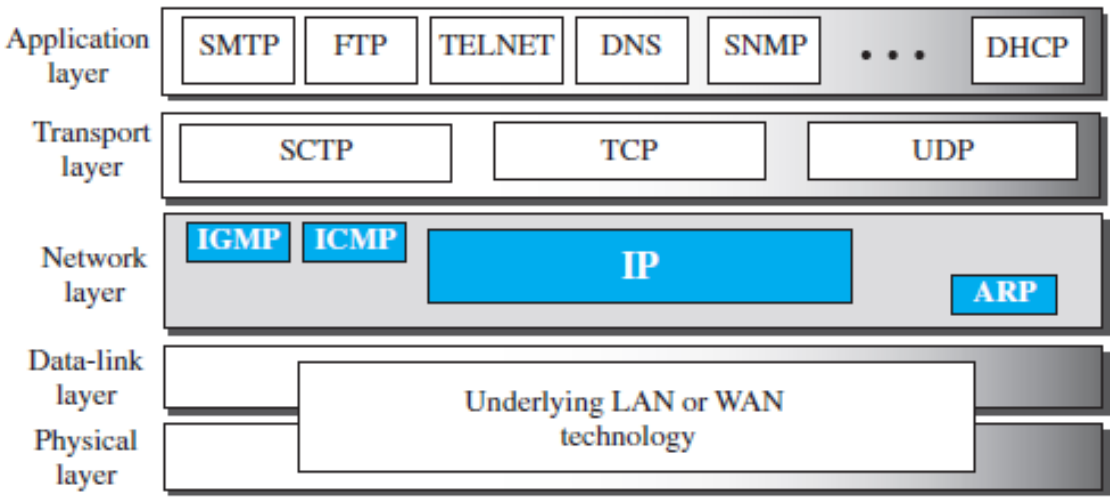


FIGURE 3.26: POSITION OF IP & OTHER NETWORK-LAYER PROTOCOLS IN TCP/IP PROTOCOL SUITE

IPv4 is an unreliable datagram protocol—a best-effort delivery service. The term *best-effort* means that IPv4 packets can be corrupted, be lost, arrive out of order, or be delayed, and may create congestion for the network. If reliability is important, IPv4 must be paired with a reliable transport-layer protocol such as TCP.

IPv4 is also a connectionless protocol that uses the datagram approach. This means that each datagram is handled independently, and each datagram can follow a different route to the destination. This implies that datagrams sent by the same source to the same destination could arrive out of order. Again, IPv4 relies on a higher-level protocol to take care of all these problems.

DATAGRAM FORMAT:

Packets used by the IP are called *datagrams*. Figure 3.27 shows the IPv4 datagram format. A datagram is a variable-length packet consisting of two parts:

header and payload (data). The header is 20 to 60 bytes in length and contains information essential to routing and delivery. It is customary in TCP/IP to show the header in 4-byte sections.

- ***Version Number.*** The 4-bit version number (VER) field defines the version of the IPv4 protocol, which, obviously, has the value of 4.
- ***Header Length.*** The 4-bit header length (HLEN) field defines the total length of the datagram header in 4-byte words. The IPv4 datagram has a variable-length header.
- ***Service Type.*** In the original design of the IP header, this field was referred to as type of service (TOS), which defined how the datagram should be handled.

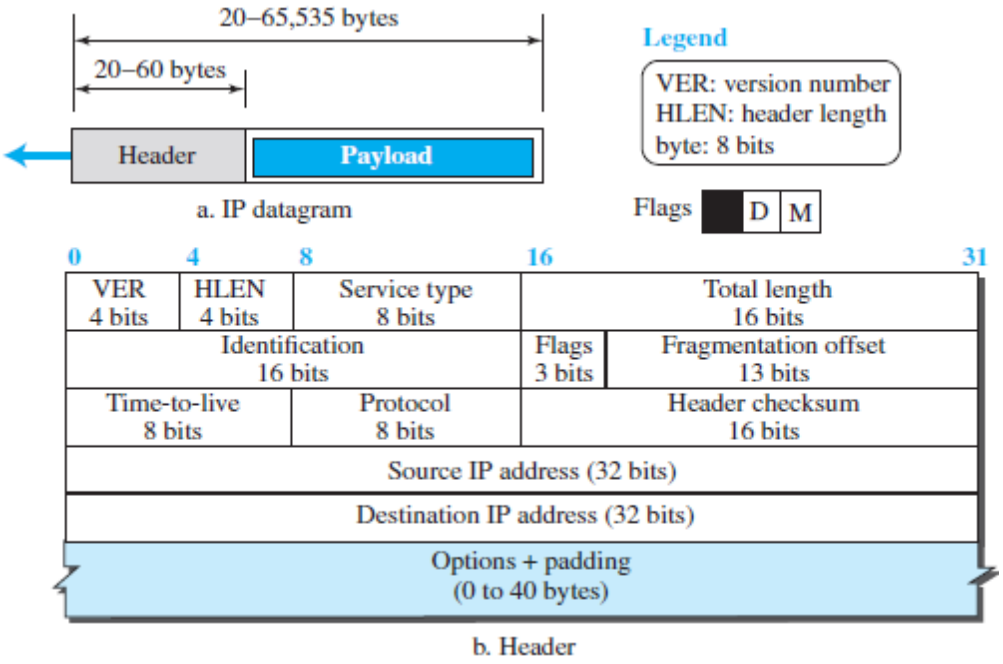


FIGURE 3.27: IP DATAGRAM

- ***Total Length.*** This 16-bit field defines the total length (header plus data) of the IP datagram in bytes. A 16-bit number can define a total length of up to 65,535 (when all bits are 1s).
- ***Identification, Flags, and Fragmentation Offset.*** These three fields are related to the fragmentation of the IP datagram when the size of the datagram is larger than the underlying network can carry.
- ***Time-to-live.*** The time-to-live (TTL) field is used to control the maximum number of hops (routers) visited by the datagram. When a source host sends the datagram, it stores a number in this field. This value is approximately two times the maximum number of routers between any two hosts. Each router that processes the datagram decrements this number by one. If this value, after being decremented, is zero, the router discards the datagram.

- ***Protocol*.** In TCP/IP, the data section of a packet, called the *payload,* carries the whole packet from another protocol. A datagram, for example, can carry a packet belonging to any transport-layer protocol such as UDP or TCP. A datagram can also carry a packet from other protocols that directly use the service of the IP, such as some routing protocols or some auxiliary protocols.

- ***Header checksum*.** IP is not a reliable protocol; it does not check whether the payload carried by a datagram is corrupted during the transmission. IP puts the burden of error checking of the payload on the protocol that owns the payload, such as UDP or TCP. The datagram header, however, is added by IP, and its error-checking is the responsibility of IP.

- ***Source and Destination Addresses*.** These 32-bit source and destination address fields define the IP address of the source and destination respectively. The source host should know its IP address. The destination IP address is either known by the protocol that uses the service of IP or is provided by the DNS.

- ***Options.*** A datagram header can have up to 40 bytes of options. Options can be used for network testing and debugging. Although options are not a required part of the IP header, option processing is required of the IP software.

- ***Payload*.** Payload, or data, is the main reason for creating a datagram. Payload is the packet coming from other protocols that use the service of IP. Comparing a datagram to a postal package, payload is the content of the package; the header is only the information written on the package.

ICMPv4:

The IPv4 has no error-reporting or error-correcting mechanism. The IP protocol also lacks a mechanism for host and management queries. A host sometimes needs to determine if a router or another host is alive. And sometimes a network manager needs information from another host or router.

The **Internet Control Message Protocol version 4 (ICMPv4)** has been designed to compensate for the above two deficiencies. It is a companion to the IP protocol. ICMP itself is a network-layer protocol.

However, its messages are not passed directly to the data-link layer as would be expected. Instead, the messages are first encapsulated inside IP datagrams before going to the lower layer. When an IP datagram encapsulates an ICMP message, the value of the protocol field in the IP datagram is set to 1 to indicate that the IP payroll is an ICMP message.

MESSAGES:

ICMP messages are divided into two broad categories: ***error-reporting messages*** and ***query messages***.

The ***error-reporting messages*** report problems that a router or a host (destination) may encounter when it processes an IP packet.

The ***query messages***, which occur in pairs, help a host or a network manager get specific information from a router or another host. For example, nodes can discover their neighbors. Also, hosts can discover and learn about routers on their network and routers can help a node redirect its messages.

An ICMP message has an 8-byte header and a variable-size data section. Although the general format of the header is different for each message type, the first 4 bytes are common to all.

As Figure 3.28 shows, the first field, ICMP type, defines the type of the message. The code field specifies the reason for the particular message type. The last common field is the checksum field (to be discussed later in the chapter). The rest of the header is specific for each message type.

The data section in error messages carries information for finding the original packet that had the error. In query messages, the data section carries extra information based on the type of query.

Error Reporting Messages: Since IP is an unreliable protocol, one of the main responsibilities of ICMP is to report some errors that may occur during the processing of the IP datagram. ICMP does not correct errors, it simply reports them.

Error correction is left to the higher-level protocols. Error messages are always sent to the original source because the only information available in the datagram about the route is the source and destination IP addresses.

ICMP uses the source IP address to send the error message to the source (originator) of the datagram. To make the error-reporting process simple, ICMP follows some rules in reporting messages:

- First, no error message will be generated for a datagram having a multicast address or special address (such as *this host* or *loopback*).
- Second, no ICMP error message will be generated in response to a datagram carrying an ICMP error message.
- Third, no ICMP error message will be generated for a fragmented datagram that is not the first fragment.

Destination Unreachable: The most widely used error message is the destination unreachable (type 3). This message uses different codes (0 to 15) to define the type of error message and the reason why a datagram has not reached its final destination.

Source Quench: Another error message is called the *source quench* (type 4) message, which informs the sender that the network has encountered congestion and the datagram has been dropped; the source needs to slow down sending more datagrams.

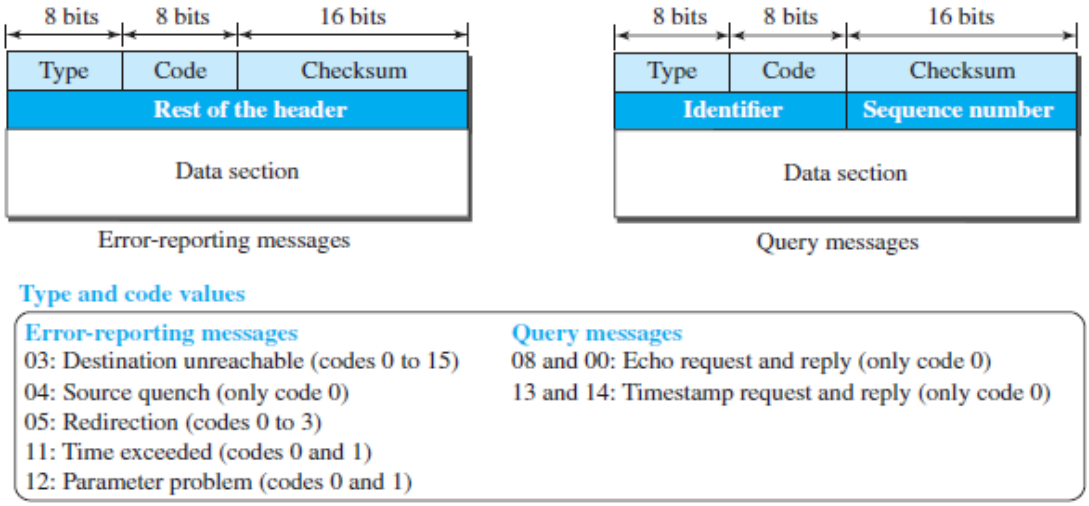


FIGURE 3.28: GENERAL FORMAT OF ICMP MESSAGES

Redirection Message: The *redirection message* (type 5) is used when the source uses a wrong router to send out its message. The router redirects the message to the appropriate router, but informs the source that it needs to change its default router in the future. The IP address of the default router is sent in the message.

Parameter Problem: A *parameter problem message* (type 12) can be sent when either there is a problem in the header of a datagram (code 0) or some options are missing or cannot be interpreted (code 1).

Query Messages: Query messages in ICMP can be used independently without relation to an IP datagram. Of course, a query message needs to be encapsulated in a datagram, as a carrier.

Query messages are used to probe or test the liveliness of hosts or routers in the Internet, find the one-way or the round-trip time for an IP datagram between two devices, or even find out whether the clocks in two devices are synchronized. Naturally, query messages come in pairs: request and reply.

IGMP:

The protocol that is used today for collecting information about group membership is the **Internet Group Management Protocol (IGMP).** IGMP is a

protocol defined at the network layer; it is one of the auxiliary protocols, like ICMP, which is considered part of the IP. IGMP messages, like ICMP messages, are encapsulated in an IP datagram.

Messages:

There are only two types of messages in IGMP version 3, query and report messages, as shown in Figure 3.29. A query message is periodically sent by a router to all hosts attached to it to ask them to report their interests about membership in groups. A report message is sent by a host as a response to a query message.



FIGURE 3.29: IGMP OPERATION

Query Message:

The query message is sent by a router to all hosts in each interface to collect information about their membership. There are three versions of query messages, as described below:

a. A ***general* query** message is sent about membership in any group. It is encapsulated in a datagram with the destination address 224.0.0.1 (all hosts and routers). Note that all routers attached to the same network receive this message to inform them that this message is already sent and that they should refrain from resending it.

b. A ***group-specific*** query message is sent from a router to ask about the membership related to a specific group. This is sent when a router does not receive a response about a specific group and wants to be sure that there is no active member of that group in the network. The group identifier (multicast address) is mentioned in the message. The message is encapsulated in a datagram with the destination address set to the corresponding multicast address. Although all hosts receive this message, those not interested drop it.

c. A ***source-and-group-specific*** query message is sent from a router to ask about the membership related to a specific group when the message comes from a specific source or sources. Again the message is sent when the router does not hear about a specific group related to a specific host

or hosts. The message is encapsulated in a datagram with the destination address set to the corresponding multicast address. Although all hosts receive this message, those not interested drop it.

Report Message

A report message is sent by a host as a response to a query message. The message contains a list of records in which each record gives the identifier of the corresponding group (multicast address) and the addresses of all sources that the host is interested in receiving messages from (inclusion).

The record can also mention the source addresses from which the host does not desire to receive a group message (exclusion). The message is encapsulated in a datagram with the multicast address 224.0.0.22 (multicast address assigned to IGMPv3).

In IGMPv3, if a host needs to join a group, it waits until it receives a query message and then sends a report message. If a host needs to leave a group, it does not respond to a query message. If no other host responds to the corresponding message, the group is purged from the router database.

Propagation of Membership Information:

After a router has collected membership information from the hosts and other routers at its own level in the tree, it can propagate it to the router located in a higher level of the tree. Finally, the router at the tree root can get the membership information to build the multicast tree. The process, however, is more complex than what we can explain in one paragraph. Interested readers can check the book website for the complete description of this protocol.

Encapsulation: The IGMP message is encapsulated in an IP datagram with the value of the protocol field set to 2 and the TTL field set to 1. The destination IP address of the datagram, however, depends on the type of message, as shown in figure 3.30.

Message Type	*IP Address*
General Query	224.0.0.1
Other Queries	Group address
Report	224.0.0.22

FIGURE 3.30: DESTINATION IP ADDRESSES

UNIT-III
NETWORK LAYER
OBJECTIVE QUESTIONS

1. The network layer concerns with
a) bits b) frames
c) packets d) none of the mentioned

2. Which one of the following is not a function of network layer?
a) routing b) inter-networking
c) congestion control **d) none of the mentioned**

3. The 4 byte IP address consists of
a) network address b) host address
c) both (a) and (b) d) none

4. In virtual circuit network each packet contains
a) full source and destination address **b) a short VC number**
c) both (a) and (b) d) none

5. Which one of the following routing algorithm can be used for network layer design?
a) shortest path algorithm b) distance vector routing
c) link state routing **d) all the above**

6. 6. Multidestination routing
a) is same as broadcast routing b) contains the list of all destinations
c) data is not sent by packets d) none

7. A subset of a network that includes all the routers but contains no loops is called
a) spanning tree b) spider structure
c) spider tree d) none

8. Which one of the following algorithm is not used for congestion control?
a) traffic aware routing b) admission control
c) load shedding **d) none of the mentioned**

9. The network layer protocol of internet is

a) Ethernet **b) internet protocol**

c) hypertext transfer protocol d) none of the mentioned

10. ICMP is primarily used for

a) error and diagnostic functions b) addressing

c) forwarding

d) none of the mentioned

11. The operation of subnet is controlled by _____.

a. Network Layer.

b. Data Link Layer

c. Data Layer

d. Transport Layer

12. _____are two popular examples of distance vector routing protocols.

a. OSPF and RIP **b. RIP and BGP**

c. BGP and OSPF d. BGP and SPF

13._____ deals with the issues of creating and maintaining routing tables.

a. Forwarding **b. Routing**

c. Directing d. None directing

14. During an adverse condition, the length of time for every device in the network to produce an accurate routing table is called the _____.

a. Accurate time b. integrated time

c. Convergence time d. Average time

15. A _____ routing table contains information entered manually.

a. Static b. Dynamic

c. Hierarchical d. Non static

16. Which of the following is/are the uses of static routing methods?

a. To manually define a default route. b. To provide more secure network environment.

c. To provide more efficient resource utilization. **d. All of the above**

17. A _____routing table is updated periodically using one of the dynamic routing protocols.

a. static **b. dynamic**

c. hierarchical d. non static

18.Which of the following is not the category of dynamic routing algorithm?
a. Distance vector protocols b. Link state protocols
c. Hybrid protocols **d. Automatic state protocols**

19. In _____ forwarding, the full IP address of a destination is given in the routing table.
a. next-hop b. network-specific
c. host-specific d. default

20. To build the routing table, _____algorithms allow routers to automatically discover and maintain awareness or the paths through the network.
a. Static routing **b. dynamic routing**
c. Hybrid routing d. automatic routing

21. In _____ forwarding, the mask and destination addresses are both 0.0.0.0 in the routing table.
a. next-hop b. network-specific
c. host-specific **d. default**

22).To build the routing table, _____method use preprogrammed definitions representing paths through the network.
a. Static routing b. dynamic routing
c. Hybrid routing d. automatic routing

23).In _____forwarding, the destination addresses is a network address in the routing table.
a. next-hop **b. network-specific**
c. host-specific d. default

24)._____ allows routers to exchange information within an AS.
a. Interior Gateway Protocol (IGP) b. Exterior Gateway Protocol (EGP)
c. Border Gateway Protocol (BGP) d. Static Gateway Protocol (SGP)

25. In _____forwarding, the routing table holds the address of just the next hop instead of complete route information.
a. next-hop b. network-specific
c. host-specific d. default

26. Which of the following is an example of Exterior Gateway Protocol?
a. Open Short Path First (OSPF) **b. Border Gateway Protocol (BGP)**
c. Routing Information Protocol (RIP) d. All of the above

27. A one-to-all communication between one source and all hosts on a network is classified as a____.

a. unicast b. multicast

c. broadcast d. point to point

28. _____allow the exchange of summary information between autonomous systems.

a. Interior Gateway Protocol (IGP) **b. Exterior Gateway Protocol (EGP)**

c. Border Gateway Protocol (BGP) d. Dynamic Gateway Protocol (DGP)

29).A robust routing protocol provides the ability to ..... build and manage the information in the IP routing table.

a. Dynamically b. Statically

c. Hierarchically d. All of the above

30.State True of False for definition of an autonomous system(AS).

i) An AS is defined as a physical portion of a larger IP network.

ii) An AS is normally comprised of an internetwork within an organization.

a. i-True, ii-True b. i-True, ii-False

c. i-False, ii-True d. i-False, ii-False

31. What are the parameters on which two networks differ.

a) Packet sized used b) use flow and error control technique

c) Connectionless control and security mechanism **d) all**

32. _____ are the limitations that cause different networks have different packet size.

a) hardware b) operating system

c) protocols **d) all**

33. Fragmentation means_____

a) adding of sma;ll packets to form large packets

b) breaking the large packet into small packets

c) forwarding packet through different networks

d) None

34. The header part of a fragment contains_____ number of fields

a) 2 **b)3** c)1 d)4

35.The header checksum is the IP header is used to verify _____.

a) only header b) only data c) both d) None

36. The higest IPV4 address in digital notation is

a) 255.0.0.0 b)255.255.0.0 **c)255.255.255.255**

d)255.255.255.255

37. Which class of IP address is used for used for more networks and less hosts.

a) class-A b) class-B **c)class-C** d) class-E

38. In IPV4 addressing, the digital notation address 222.255.255.255 belongs to
a) class-A b) class-B **c)class-C** d) class-E

39.classless inter domain routing contains_____.
a)32 bit IP address b)32 bit mask address **c)both** d)None

40.To indicate a sender that,it has no data,to the receiver bit is set
a)PSH b)RST **c)FIN** d) ACK

41.connecting different networks is called _____
a) intranet working **b)internetworking** c)multiple networking d)ALL

42.Bridges are used in _____ layer.
a) physical **b)MAC** c)network d)application

43.Which is a intranet working device
a) router b)gateway c)bridge **d)ALL**

44. Gateways are used at _____layer.
a) datalink layer b)network layer **c)application** d)ALL

45.The max length of the option load field in IP datagram is _____.
a)40 bytes b)80 bytes c)16 bytes d)any number of bytes multipleof 4

46. The length of the subnet mask is _____ bits .
a)16 bits **b)32bits** c)64bits d)any

47.Address resolution protocol is used to MAP the IP address on to the_____.
a) data link layer b)internet address c)network address d)port address

48.RARP is used to map the data link layer address onto _____address.
a)network b)port **c)IP** d)None

49.which are the following option are used in IPV4.
a) security b)timestamp c)source routing **d)ALL**

50.Which class of IP addressing provide more number of hosts in each network
a)class-A b)class-B c)class-c d)class-D

UNIT-III
Descriptive Questions

1. What are the Services provided by Network layer to the Transport layer ?

2. Discuss the functions of the communication subnet to provide datagram service.

3. What is meant by connection state information in a virtual circuit network?

4. Compare Virtual-Circuit and Datagram Subnets.

5. What is routing algorithm? What are the classifications of it?

6. What is the Optimality Principle?

7. With an example explain shortest path routing algorithm.

8. Explain flooding

9. Explain distance vector routing algorithm.

10. Explain count-to-infinity problem.
11. Write short notes on the following
    (a) IPV4 (b) IPV6
12. Write about Internet Control Protocols.

UNIT-IV

TRANSPORT LAYER:

The transport layer in the TCP/IP suite is located between the application layer and the network layer. It provides services to the application layer and receives services from the network layer.

The transport layer acts as a liaison between a client program and a server program, a process-to-process connection. The transport layer is the heart of the TCP/IP protocol suite; it is the end-to-end logical vehicle for transferring data from one point to another in the Internet.

Introduction:

The transport layer is located between the application layer and the network layer. It provides a process-to-process communication between two application layers, one at the local host and the other at the remote host.

Communication is provided using a logical connection, which means that the two application layers, which can be located in different parts of the globe, assume that there is an imaginary direct connection through which they can send and receive messages.

THE TRANSPORT SERVICE:

Services provided to the upper layers:

The ultimate goal of the transport layer is to provide efficient, reliable, and cost-effective data transmission service to its users, normally processes in the application layer. To achieve this, the transport layer makes use of the services provided by the network layer. The software and/or hardware within the transport layer that does the work is called the **transport entity**.

The transport entity can be located in the operating system kernel, in a library package bound into network applications, in a separate user process, or even on the network interface card. The first two options are most common on the Internet. The (logical) relationship of the network, transport, and application layers is illustrated in Fig. 4.1.

Just as there are two types of network service, ***connection-oriented*** *and* ***connectionless****, there are also two types of transport service*. The ***connection-oriented transport service*** is similar to the connection-oriented network service in many ways. In both cases, connections have three phases: establishment, data transfer, and release. Addressing and flow control are also similar in both layers.

Furthermore, the ***connectionless transport service*** is also very similar to the connectionless network service. However, note that it can be difficult to provide a connectionless transport service on top of a connection-oriented network service, since it is inefficient to set up a connection to send a single packet and then tear (*meaning run/rip/rush*) it down immediately afterwards.

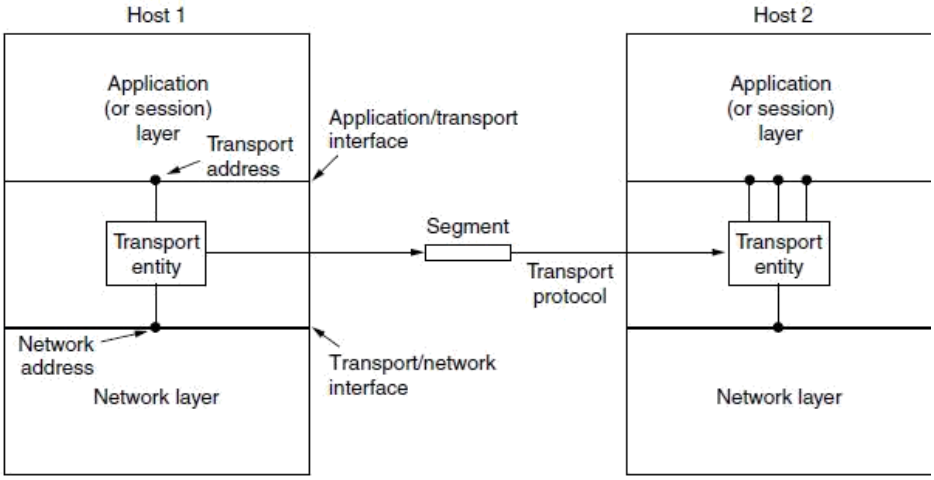


Figure 4.1: The network, transport, and application layers

Transport service primitives:

To allow users to access the transport service, the transport layer must provide some operations to application programs, that is, a transport service interface. Each transport service has its own interface.

The transport service is similar to the network service, but there are also some important differences. The main difference is that the network service is intended to model the service offered by real networks and all. Real networks can lose packets, so the network service is generally unreliable.

The connection-oriented transport service, in contrast, is reliable. Of course, real networks are not error-free, but that is precisely the purpose of the transport layer—to provide a reliable service on top of an unreliable network.

A second difference between the network service and transport service is whom the services are intended for. The network service is used only by the transport entities. Few users write their own transport entities, and thus few users or programs ever (*meaning always/forever/still*) see the bare network service.

Berkeley sockets: Let us now briefly inspect another set of transport primitives, the socket primitives as they are used for TCP. Sockets were first released as part of the Berkeley UNIX 4.2BSD software distribution in 1983. They quickly became popular.

The primitives are now widely used for Internet programming on many operating systems, especially UNIX-based systems, and there is a socket-style API for Windows called "winsock." The primitives are listed in Fig. 4.2.

Primitive	Meaning
SOCKET	Create a new communication endpoint
BIND	Associate a local address with a socket
LISTEN	Announce willingness to accept connections; give queue size
ACCEPT	Passively establish an incoming connection
CONNECT	Actively attempt to establish a connection
SEND	Send some data over the connection
RECEIVE	Receive some data from the connection
CLOSE	Release the connection

Figure 4.2: The socket primitives for TCP

Note: An Example of Socket Programming: An Internet File Server

ELEMENTS OF TRANSPORT PROTOCOLS:

The transport service is implemented by a **transport protocol** used between the two transport entities. In some ways, transport protocols resemble the data link protocols. Both have to deal with error control, sequencing, and flow control, among other issues.

However, significant differences between the two also exist. These differences are due to major dissimilarities between the environments in which the two protocols operate, as shown in Fig. 4.3.

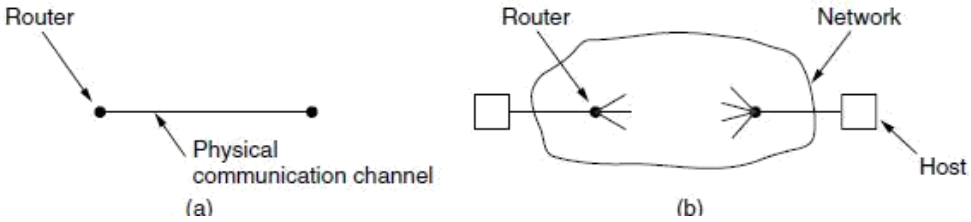


Figure 4.3: Environment of the (a) data link layer (b) transport layer

At the data link layer, two routers communicate directly via a physical channel, whether wired or wireless, whereas at the transport layer, this physical channel is replaced by the entire network.

For one thing, over point-to-point links such as wires or optical fiber, it is usually not necessary for a router to specify which router it wants to talk to—each outgoing line leads directly to a particular router. In the transport layer, explicit addressing of destinations is required.

For another thing, the process of establishing a connection over the wire of Fig. 4.3(a) is simple: the other end is always there (unless it has crashed, in which case it is not there). Either way, there is not much to do.

Even on wireless links, the process is not much different. Just sending a message is sufficient to have it reach all other destinations. If the message is not acknowledged due to an error, it can be resent. In the transport layer, initial connection establishment is complicated.

Addressing:

When an application (e.g., a user) process wishes to set up a connection to a remote application process, it must specify which one to connect to. (Connectionless transport has the same problem: to whom should each message be sent?) The method normally used is to define transport addresses to which processes can listen for connection requests.

In the Internet, these endpoints are called **ports**. We will use the generic term **TSAP** (**Transport Service Access Point**) to mean a specific endpoint in the transport layer. The analogous endpoints in the network layer (i.e., network layer addresses) are naturally called **NSAPs** (**Network Service Access Points**). IP addresses are examples of NSAPs.

Figure 4.4 illustrates the relationship between the NSAPs, the TSAPs, and a transport connection.

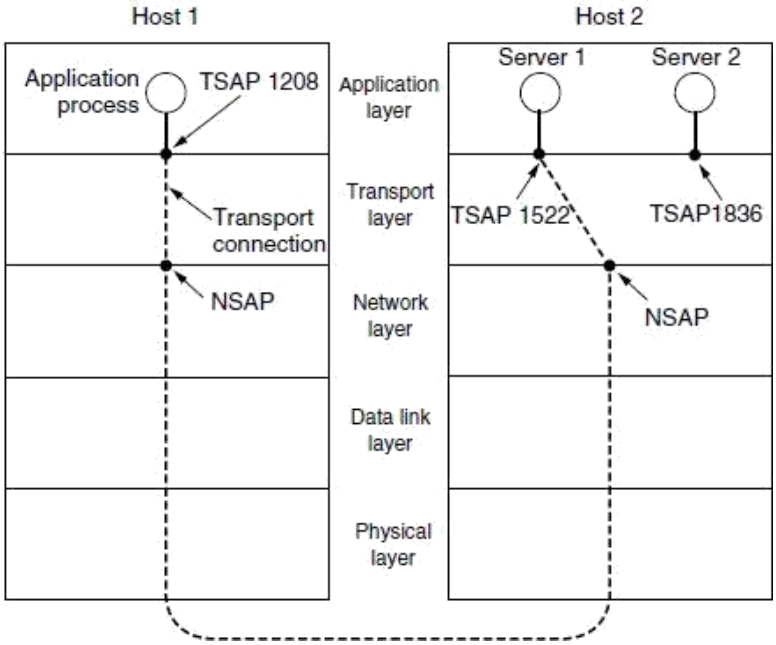


Figure 4.4: TSAPs, NSAPs, and Transport connections

Application processes, both clients and servers, can attach themselves to a local TSAP to establish a connection to a remote TSAP. These connections run through NSAPs on each host, as shown in figure 4.4.

A possible scenario for a transport connection is as follows:

1. A mail server process attaches itself to TSAP 1522 on host 2 to wait for an incoming call. A call such as our LISTEN might be used, for example.

2. An application process on host 1 wants to send an email message, so it attaches itself to TSAP 1208 and issues a CONNECT request.

    - The request specifies TSAP 1208 on host 1 as the source and TSAP 1522 on host 2 as the destination. This action ultimately results in a transport connection being established between the application process and the server.

3. The application process sends over the mail message.

4. The mail server responds to say that it will deliver the message.

5. The transport connection is released.

Connection Establishment:

Establishing a connection sounds easy, but it is actually surprisingly tricky. At first glance, it would seem sufficient for one transport entity to just send a CONNECTION REQUEST segment to the destination and wait for a CONNECTION ACCEPTED reply. The problem occurs when the network can lose, delay, corrupt, and duplicate packets. This behavior causes serious complications.

Imagine a network that is so congested that acknowledgements hardly ever get back in time and each packet times out and is retransmitted two or three times. Suppose that the network uses datagrams inside and that every packet follows a different route.

Some of the packets might get stuck in a traffic jam inside the network and take a long time to arrive. That is, they may be delayed in the network and pop out much later, when the sender thought that they had been lost.

The worst possible nightmare is as follows. A user establishes a connection with a bank, sends messages telling the bank to transfer a large amount of money to the account of a not-entirely-trustworthy person. Unfortunately, the packets decide to take the scenic route to the destination and go off exploring a remote corner of the network.

The sender then times out and sends them all again. This time the packets take the shortest route and are delivered quickly so the sender releases the connection.

Unfortunately, eventually the initial batch of packets finally come out of hiding and arrive at the destination in order, asking the bank to establish a new connection and transfer money (again). The bank has no way of telling that these are duplicates. It must assume that this is a second, independent transaction, and transfers the money again.

The crux (*meaning root*) of the problem is that the delayed duplicates are thought to be new packets. We cannot prevent packets from being duplicated and delayed. But if and when this happens, the packets must be rejected as duplicates and not processed as fresh packets.

The problem can be attacked in various ways, none of them very satisfactory. One way is to use throwaway transport addresses. In this approach, each time a transport address is needed, a new one is generated. When a connection is released, the address is discarded and never used again. Delayed duplicate packets then never find their way to a transport process and can do no damage.

Note: However, this approach makes it more difficult to connect with a process in the first place.

Another possibility is to give each connection a unique identifier (i.e., a sequence number incremented for each connection established) chosen by the initiating party and put in each segment, including the one requesting the connection.

After each connection is released, each transport entity can update a table listing obsolete connections as (peer transport entity, connection identifier) pairs. Whenever a connection request comes in, it can be checked against the table to see if it belongs to a previously released connection.

Unfortunately, this scheme has a basic flaw: it requires each transport entity to maintain a certain amount of history information indefinitely. This history must persist at both the source and destination machines. Otherwise, if a machine crashes and loses its memory, it will no longer know which connection identifiers have already been used by its peers.

Instead, we need to take a different tack to simplify the problem. Rather than allowing packets to live forever within the network, we devise a mechanism to kill off aged packets that are still hobbling about.

Packet lifetime can be restricted to a known maximum using one (or more) of the following techniques:

1. Restricted network design.
2. Putting a hop counter in each packet.
3. Timestamping each packet.

TCP uses three-way handshake to establish connections in the presence of delayed duplicate control segments as shown in figure 4.5.

Connection Release:

Releasing a connection is easier than establishing one. There are two styles of terminating a connection: ***asymmetric release*** and ***symmetric release***.

Asymmetric release is the way the telephone system works: when one party hangs up, the connection is broken.

Symmetric release treats the connection as two separate unidirectional connections and requires each one to be released separately.

Asymmetric release is abrupt and may result in data loss. Consider the scenario of Fig. 4.6. After the connection is established, host 1 sends a segment that arrives properly at host 2. Then host 1 sends another segment.

Unfortunately, host 2 issues a DISCONNECT before the second segment arrives. The result is that the connection is released and data are lost.

Symmetric release does the job when each process has a fixed amount of data to send and clearly knows when it has sent it. In other situations, determining that all the work has been done and the connection should be terminated is not so obvious.

One can envision a protocol in which host 1 says "I am done. Are you done too?" If host 2 responds: "I am done too. Goodbye, the connection can be safely released."

In practice, we can avoid this quandary (*meaning dilemma/difficulty*) by foregoing the need for agreement and pushing the problem up to the transport user, letting each side independently decide when it is done. This is an easier problem to solve.

Figure 4.7 illustrates four scenarios of releasing using a three-way handshake. While this protocol is not infallible, it is usually adequate. In Fig. 4.7(a), we see the normal case in which one of the users sends a DR (DISCONNECTION REQUEST) segment to initiate the connection release.

When it arrives, the recipient sends back a DR segment and starts a timer, just in case its DR is lost. When this DR arrives, the original sender sends back an ACK segment and releases the connection.

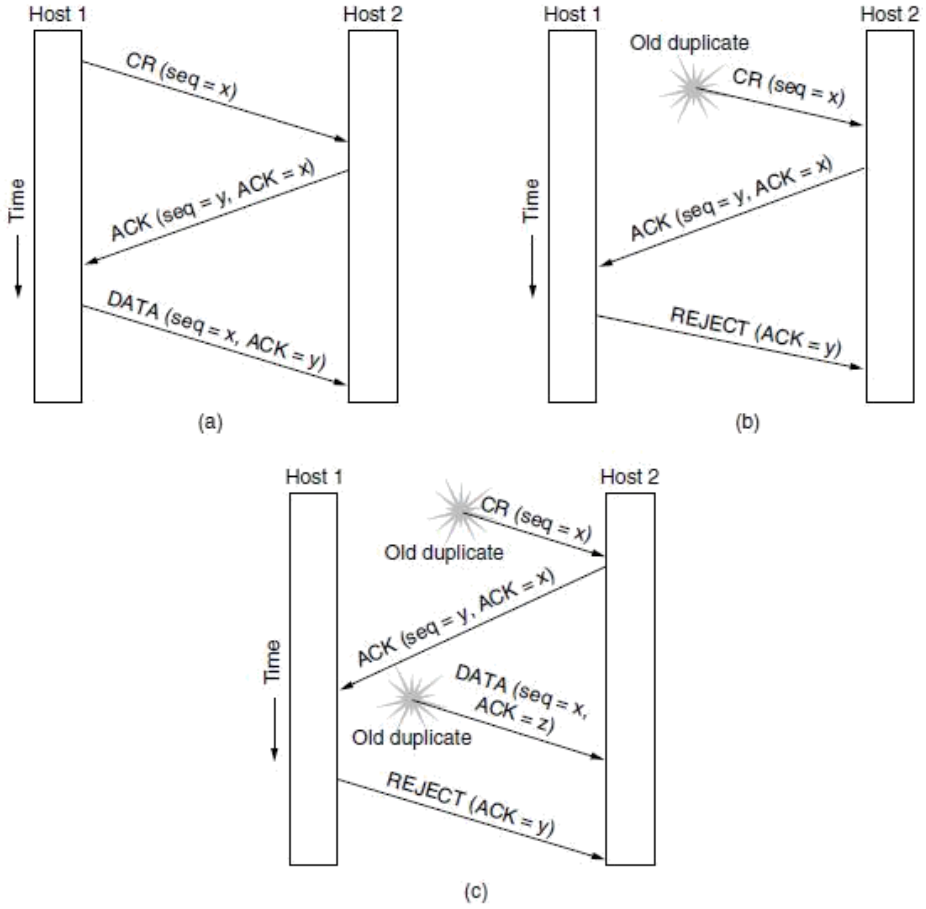


Figure 4.5: Three protocol scenarios for establishing a connection using a three-way handshake. CR denotes Connection Request. (a) normal operation. (b) old duplicate connection request appearing out of nowhere. (c) duplicate connection request and duplicate ack.

Finally, when the ACK segment arrives, the receiver also releases the connection. Releasing a connection means that the transport entity removes the information about the connection from its table of currently open connections and signals the connection's owner (the transport user) somehow.

If the final ACK segment is lost, as shown in Fig. 4.7(b), the situation is saved by the timer. When the timer expires, the connection is released anyway. Now consider the case of the second DR being lost.

The user initiating the disconnection will not receive the expected response, will time out, and will start all over again. In Fig. 4.7(c), we see how this works, assuming that the second time no segments are lost and all segments are delivered correctly and on time.

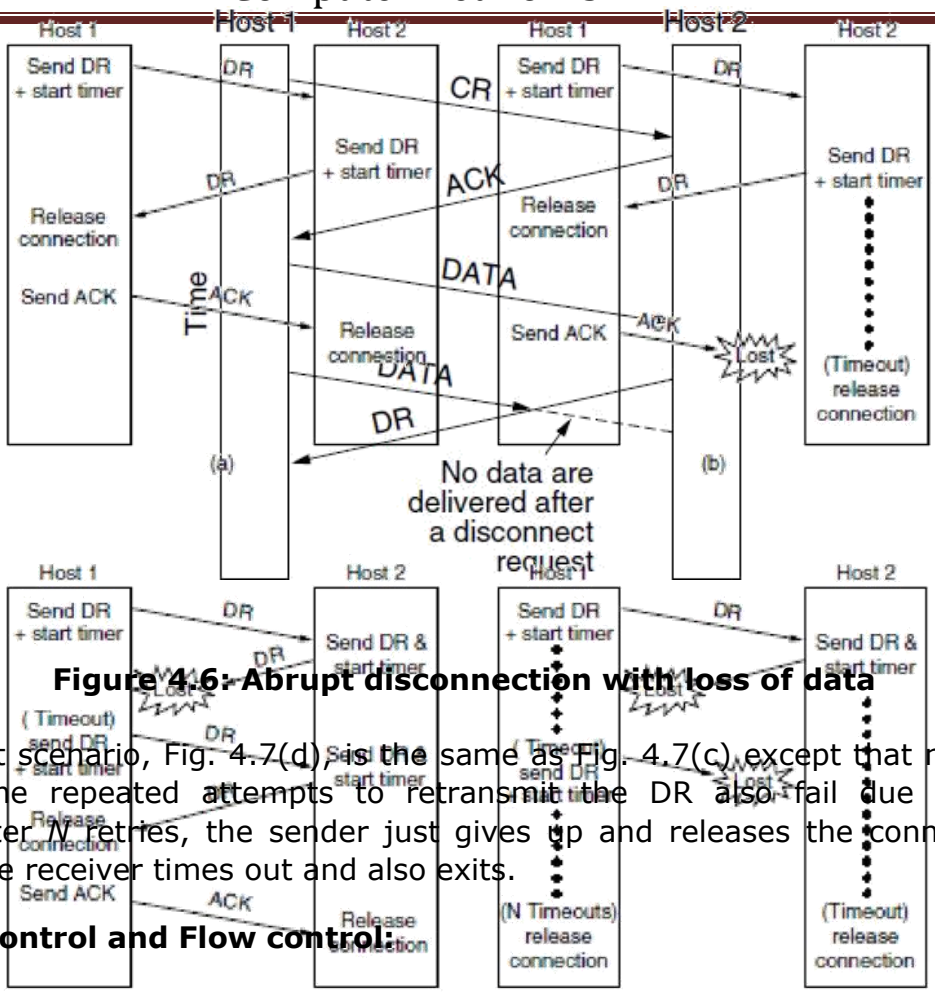


Figure 4.6: Abrupt disconnection with loss of data

Our last scenario, Fig. 4.7(d), is the same as Fig. 4.7(c) except that now we assume all the repeated attempts to retransmit the DR also fail due to lost segments. After N retries, the sender just gives up and releases the connection. Meanwhile, the receiver times out and also exits.

Error control and Flow control:

Error control is ensuring that the data is delivered with the desired level of reliability, usually that all of the data is delivered without any errors. Flow control is keeping a fast transmitter from overrunning a slow receiver.

MULTIPLEXING:

Multiplexing, or sharing several conversations over connections, virtual circuits, and physical links plays a role in several layers of the network architecture. In the transport layer, the need for multiplexing can arise in a number of ways. For example, if only one network address is available on a host, all transport connections on that machine have to use it.

When a segment comes in, some way is needed to tell which process to give it to. This situation, called **multiplexing**, is shown in Fig. 4.8(a). In this figure, four distinct transport connections all use the same network connection (e.g., IP address) to the remote host.

Multiplexing can also be useful in the transport layer for another reason. Suppose, for example, that a host has multiple network paths that it can use. If a user needs more bandwidth or more reliability than one of the network paths can provide, a way out is to have a connection that distributes the traffic among multiple network paths on a round-robin basis, as indicated in Fig. 4.8(b).

Figure 4.7: Four protocol scenarios for releasing a connection. (a) normal case of three-way handshake. (b) final ACK lost. (c) response lost. (d) response lost and subsequent DRs lost.

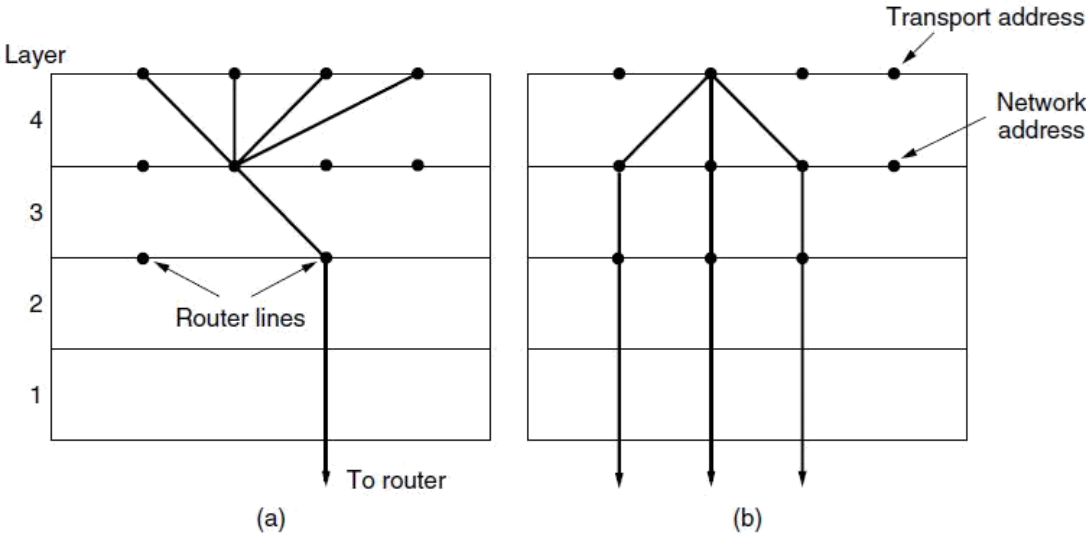


Figure 4.8: (A) Multiplexing (B) Inverse Multiplexing

CRASH RECOVERY:

If hosts and routers are subject to crashes or connections are long-lived (e.g., large software or media downloads), recovery from these crashes becomes an issue.

If the transport entity is entirely within the hosts, recovery from network and router crashes is straightforward. The transport entities expect lost segments all the time and know how to cope with them by using retransmissions.

A more troublesome problem is how to recover from host crashes. In particular, it may be desirable for clients to be able to continue working when servers crash and quickly reboot.

CONGESTION CONTROL:

If the transport entities on many machines send too many packets into the network too quickly, the network will become congested, with performance degraded as packets are delayed and lost.

Controlling congestion to avoid this problem is the combined responsibility of the network and transport layers. Congestion occurs at routers, so it is detected at the network layer.

However, congestion is ultimately caused by traffic sent into the network by the transport layer. The only effective way to control congestion is for the transport protocols to send packets into the network more slowly.

DESIRABLE BANDWIDTH ALLOCATION:

Before we describe how to regulate traffic, we must understand what we are trying to achieve by running a congestion control algorithm. That is, we must specify the state in which a good congestion control algorithm will operate the network.

The goal is more than to simply avoid congestion. It is to find a good allocation of bandwidth to the transport entities that are using the network. A good allocation will deliver good performance because it uses all the available bandwidth but avoids congestion, it will be fair across competing transport entities, and it will quickly track changes in traffic demands.

Efficiency and Power:

An efficient allocation of bandwidth across transport entities will use all of the network capacity that is available. However, it is not quite right to think that if there is a 100-Mbps link, five transport entities should get 20 Mbps each. They should usually get less than 20 Mbps for good performance.

Max-Min Fairness:

In the preceding discussion, we did not talk about how to divide bandwidth between different transport senders. This sounds like a simple question to answer—give all the senders an equal fraction of the bandwidth—but it involves several considerations.

Perhaps the first consideration is to ask what this problem has to do with congestion control.

A second consideration is what a fair portion means for flows in a network. It is simple enough if *N* flows use a single link, in which case they can all have 1*/N* of the bandwidth (although efficiency will dictate that they use slightly less if the traffic is bursty).

But what happens if the flows have different, but overlapping, network paths? For example, one flow may cross three links, and the other flows may cross one link. The three-link flow consumes more network resources. It might be fairer in some sense to give it less bandwidth than the one-link flows.

The form of fairness that is often desired for network usage is **max-min fairness**. An allocation is max-min fair if the bandwidth given to one flow cannot be increased without decreasing the bandwidth given to another flow with an allocation that is no larger.

Convergence:

A final criterion is that the congestion control algorithm converge quickly to a fair and efficient allocation of bandwidth. The discussion of the desirable operating point above assumes a static network environment.

However, connections are always coming and going in a network, and the bandwidth needed by a given connection will vary over time too. Because of the variation in demand, the ideal operating point for the network varies over time.

A good congestion control algorithm should rapidly converge to the ideal operating point, and it should track that point as it changes over time. If the convergence is too slow, the algorithm will never be close to the changing operating point. If the algorithm is not stable, it may fail to converge to the right point in some cases, or even oscillate around the right point.

Regulating the sending rate:

Now it is time to regulate the sending rates to obtain a desirable bandwidth allocation. The sending rate may be limited by two factors.

- The first is flow control, in the case that there is insufficient buffering at the receiver.
- The second is congestion, in the case that there is insufficient capacity in the network.

In Fig. 4.9, we see this problem illustrated hydraulically. In Fig. 4.9(a), we see a thick pipe leading to a small-capacity receiver. This is a flow-control limited situation. As long as the sender does not send more water than the bucket can contain, no water will be lost.

In Fig. 4.9(b), the limiting factor is not the bucket capacity, but the internal carrying capacity of the network. If too much water comes in too fast, it will back up and some will be lost (in this case, by overflowing the funnel).

The way that a transport protocol should regulate the sending rate depends on the form of the feedback returned by the network. Different network layers may return different kinds of feedback. The feedback may be explicit or implicit, and it may be precise or imprecise.

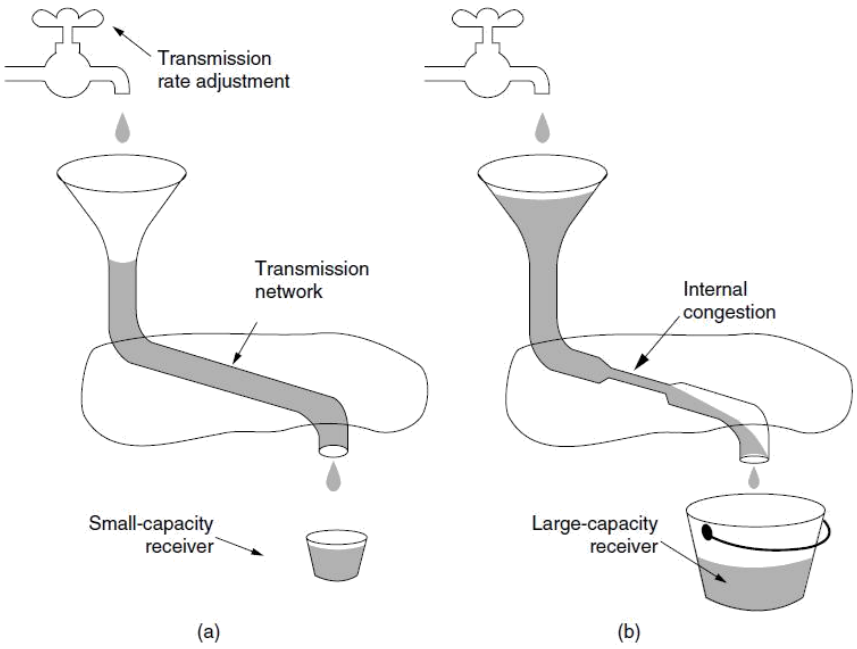


Figure 4.9: (a) a fast network feeding a low-capacity receiver. (b) a slow network feeding a high-capacity receiver.

Wireless issues:

Transport protocols such as TCP that implement congestion control should be independent of the underlying network and link layer technologies. That is a good theory, but in practice there are issues with wireless networks. The main issue is that packet loss is often used as a congestion signal, including by TCP.

Wireless networks lose packets all the time due to transmission errors. To function well, the only packet losses that the congestion control algorithm should observe are losses due to insufficient bandwidth, not losses due to transmission errors. One solution to this problem is to mask the wireless losses by using retransmissions over the wireless link.

THE INTERNET TRANSPORT PROTOCOLS:

UDP:

The Internet has two main protocols in the transport layer, a connectionless protocol and a connection-oriented one. The protocols complement each other.

The connectionless protocol is UDP. It does almost nothing beyond sending packets between applications, letting applications build their own protocols on top as needed.

The connection-oriented protocol is TCP. It does almost everything. It makes connections and adds reliability with retransmissions, along with flow control and congestion control, all on behalf of the applications that use it.

INTRODUCTION TO UDP:

The Internet protocol suite supports a connectionless transport protocol called **UDP** (**User Datagram Protocol**).

UDP provides a way for applications to send encapsulated IP datagrams without having to establish a connection. UDP is described in RFC 768.

UDP transmits **segments** consisting of an 8-byte header followed by the payload. The header is shown in Fig. 4.10. The two **ports** serve to identify the endpoints within the source and destination machines.

When a UDP packet arrives, its payload is handed to the process attached to the destination port. This attachment occurs when the BIND primitive or something similar is used.

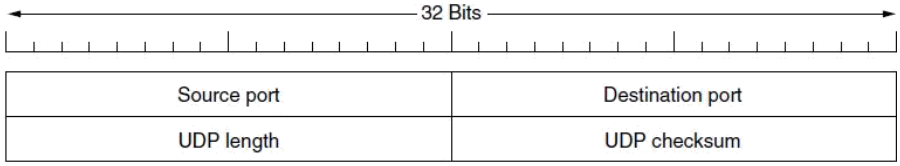


Figure 4.10: the UDP header

Think of ports as mailboxes that applications can rent to receive packets. In fact, the main value of UDP over just using raw IP is the addition of the source and destination ports.

Without the port fields, the transport layer would not know what to do with each incoming packet. With them, it delivers the embedded segment to the correct application.

The source port is primarily needed when a reply must be sent back to the source. By copying the *Source port* field from the incoming segment into the *Destination port* field of the outgoing segment, the process sending the reply can specify which process on the sending machine is to get it.

The *UDP length* field includes the 8-byte header and the data. The minimum length is 8 bytes, to cover the header. The maximum length is 65,515 bytes, which is lower than the largest number that will fit in 16 bits because of the size limit on IP packets.

An optional *Checksum* is also provided for extra reliability. It checksums the header, the data, and a conceptual IP pseudoheader. When performing this computation, the *Checksum* field is set to zero and the data field is padded out with an additional zero byte if its length is an odd number.

The checksum algorithm is simply to add up all the 16-bit words in one's complement and to take the one's complement of the sum.

Remote procedure call:

In a certain sense, sending a message to a remote host and getting a reply back is a lot like making a function call in a programming language. The idea behind RPC is to make a remote procedure call look as much as possible like a local one.

In the simplest form, to call a remote procedure, the client program must be bound with a small library procedure, called the **client stub**, that represents the server procedure in the client's address space.

Similarly, the server is bound with a procedure called the **server stub**. These procedures hide the fact that the procedure call from the client to the server is not local. The actual steps in making an RPC are shown in Fig. 4.12.

- Step 1 is the client calling the client stub. This call is a local procedure call, with the parameters pushed onto the stack in the normal way.
- Step 2 is the client stub packing the parameters into a message and making a system call to send the message. Packing the parameters is called **marshaling**.
- Step 3 is the operating system sending the message from the client machine to the server machine.

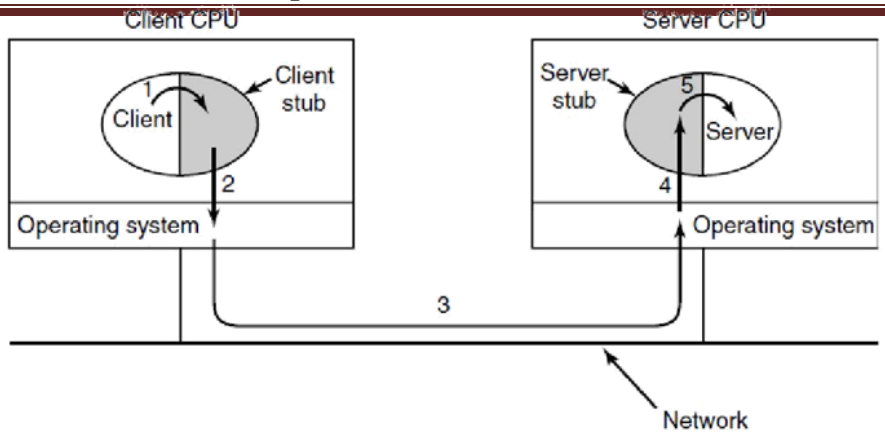


Figure 4.12: Steps in making a remote procedure call, the stubs are shaded

- Step 4 is the operating system passing the incoming packet to the server stub.
- Finally, step 5 is the server stub calling the server procedure with the unmarshaled parameters.

The reply traces the same path in the other direction.

The key item to note here is that the client procedure, written by the user, just makes a normal (i.e., local) procedure call to the client stub, which has the same name as the server procedure. Since the client procedure and client stub are in the same address space, the parameters are passed in the usual way.

Similarly, the server procedure is called by a procedure in its address space with the parameters it expects. To the server procedure, nothing is unusual.

Real-Time Transport Protocols

Client-server RPC is one area in which UDP is widely used. Another one is for real-time multimedia applications.

In particular, as Internet radio, Internet telephony, music-on-demand, videoconferencing, video-on-demand, and other multimedia applications became more commonplace, people have discovered that each application was reinventing more or less the same real-time transport protocol. Thus was **RTP** (**Real-time Transport Protocol**) born.

It is described in RFC 3550 and is now in widespread use for multimedia applications. There are two aspects of real-time transport . The first is the RTP protocol for transporting audio and video data in packets. The second is the processing that takes place, mostly at the receiver, to play out the audio and video at the right time.

RTP—The Real-Time Transport Protocol:

The basic function of RTP is to multiplex several real-time data streams onto a single stream of UDP packets. The UDP stream can be sent to a single destination (unicasting) or to multiple destinations (multicasting).

Because RTP just uses normal UDP, its packets are not treated specially by the routers unless some normal IP quality-of-service features are enabled. In particular, there are no special guarantees about delivery, and packets may be lost, delayed, corrupted, etc.

The RTP format contains several features to help receivers work with multimedia information. The RTP header is illustrated in Fig. 4.13. It consists of three 32-bit words and potentially some extensions.

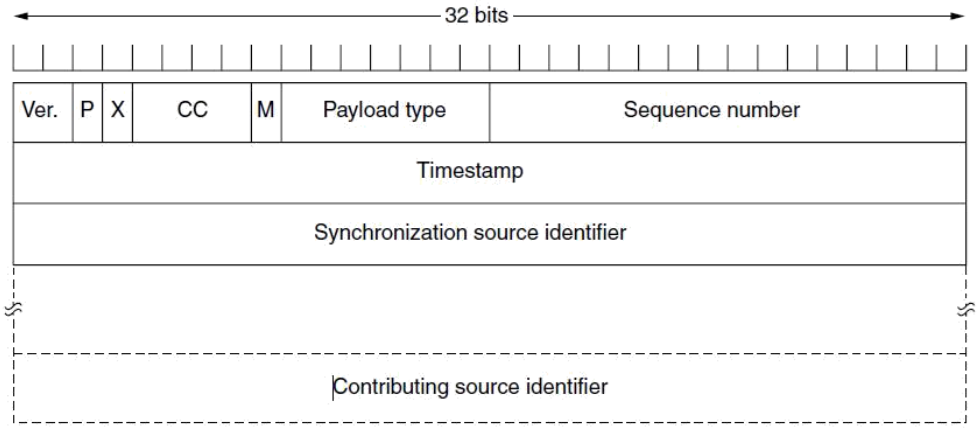


Figure 4.13: The RTP header

The first word contains the Version field, which is already at 2.

The *P* bit indicates that the packet has been padded to a multiple of 4 bytes.

The X bit indicates that an extension header is present.

The CC field tells how many contributing sources are present, from 0 to 15.

The M bit is an application-specific marker bit. It can be used to mark the start of a video frame, the start of a word in an audio channel, or something else that the application understands.

The Payload type field tells which encoding algorithm has been used (e.g., uncompressed 8-bit audio, MP3, etc.).

The Sequence number is just a counter that is incremented on each RTP packet sent. It is used to detect lost packets.

The Timestamp is produced by the stream's source to note when the first sample in the packet was made.

The Synchronization source identifier tells which stream the packet belongs

to. It is the method used to multiplex and demultiplex multiple data streams onto a single stream of UDP packets.

Finally, the Contributing source identifiers, if any, are used when mixers are present.

RTCP—The Real-time Transport Control Protocol

RTP has a little sister protocol (little sibling protocol?) called **RTCP** (**Realtime Transport Control Protocol**). It is defined along with RTP in RFC 3550 and handles feedback, synchronization, and the user interface. It does not transport any media samples.

THE INTERNET TRANSPORT PROTOCOLS:

TCP

UDP is a simple protocol and it has some very important uses, such as clientserver interactions and multimedia, but for most Internet applications, reliable, sequenced delivery is needed. UDP cannot provide this, so another protocol is required. It is called TCP and is the main workhorse of the Internet.

Introduction to TCP:

TCP (**Transmission Control Protocol**) was specifically designed to provide a reliable end-to-end byte stream over an unreliable internetwork. An internetwork differs from a single network because different parts may have wildly different topologies, bandwidths, delays, packet sizes, and other parameters.

TCP was designed to dynamically adapt to properties of the internetwork and to be robust in the face of many kinds of failures. TCP was formally defined in RFC 793 in September 1981.

As time went on, many improvements have been made, and various errors and inconsistencies have been fixed. To give you a sense of the extent of TCP, the important RFCs are now RFC 793 plus: clarifications and bug fixes in RFC 1122; extensions for high-performance in RFC 1323.

Selective acknowledgements in RFC 2018; congestion control in RFC 2581; repurposing of header fields for quality of service in RFC 2873; improved retransmission timers in RFC 2988; and explicit congestion notification in RFC 3168. The IP layer gives no guarantee that datagrams will be delivered properly, nor any indication of how fast datagrams may be sent.

It is up to TCP to send datagrams fast enough to make use of the capacity but not cause congestion, and to time out and retransmit any datagrams that are not delivered. Datagrams that do arrive may well do so in the wrong order; it is also up to TCP to reassemble them into messages in the proper sequence.

The TCP Service Model:

TCP service is obtained by both the sender and the receiver creating end points, called **sockets.** Each socket has a socket number (address) consisting of the IP address of the host and a 16-bit number local to that host, called a **port**. A port is the TCP name for a TSAP.

For TCP service to be obtained, a connection must be explicitly established between a socket on one machine and a socket on another machine. A socket may be used for multiple connections at the same time. In other words, two or more connections may terminate at the same socket.

Port numbers below 1024 are reserved for standard services that can usually only be started by privileged users (e.g., root in UNIX systems). They are called **well-known ports**.

For example, any process wishing to remotely retrieve mail from a host can connect to the destination host's port 143 to contact its IMAP daemon. The list of well-known ports is given at *www.iana.org*. Over 700 have been assigned. A few of the better-known ones are listed in Fig. 4.14.

Port	Protocol	Use
20, 21	FTP	File transfer
22	SSH	Remote login, replacement for Telnet
25	SMTP	Email
80	HTTP	World Wide Web
110	POP-3	Remote email access
143	IMAP	Remote email access
443	HTTPS	Secure Web (HTTP over SSL/TLS)
543	RTSP	Media player control
631	IPP	Printer sharing

Figure 4.14: Some assigned ports

All TCP connections are full duplex and point-to-point. Full duplex means that traffic can go in both directions at the same time. Point-to-point means that each connection has exactly two end points. TCP does not support multicasting or broadcasting.

A TCP connection is a byte stream, not a message stream. Message oundaries are not preserved end to end.

The TCP Protocol:

A key feature of TCP, and one that dominates the protocol design, is that every byte on a TCP connection has its own 32-bit sequence number. When the Internet began, the lines between routers were mostly 56-kbps leased lines, so a host blasting away at full speed took over 1 week to cycle through the sequence numbers.

The sending and receiving TCP entities exchange data in the form of segments. A **TCP segment** consists of a fixed 20-byte header (plus an optional part) followed by zero or more data bytes. The TCP software decides how big segments should be.

It can accumulate data from several writes into one segment or can split data from one write over multiple segments. Two limits restrict the segment size. First, each segment, including the TCP header, must fit in the 65,515- byte IP payload. Second, each link has an **MTU** (**Maximum Transfer Unit**).

Each segment must fit in the MTU at the sender and receiver so that it can be sent and received in a single, unfragmented packet. However, it is still possible for IP packets carrying TCP segments to be fragmented when passing over a network path for which some link has a small MTU.

If this happens, it degrades performance and causes other problems. Instead, modern TCP implementations perform **path MTU discovery** by using the technique outlined in RFC 1191. This technique uses ICMP error messages to find the smallest MTU for any link on the path. TCP then adjusts the segment size downwards to avoid fragmentation.

The basic protocol used by TCP entities is the sliding window protocol with a dynamic window size. When a sender transmits a segment, it also starts a timer. When the segment arrives at the destination, the receiving TCP entity sends back a segment (with data if any exist, and otherwise without) bearing an acknowledgement number equal to the next sequence number it expects to receive and the remaining window size.

If the sender's timer goes off before the acknowledgement is received, the sender transmits the segment again.

The TCP Segment Header:

Figure 4.15 shows the layout of a TCP segment. Every segment begins with a fixed-format, 20-byte header. The fixed header may be followed by header options.

After the options, if any, up to 65,535 − 20 − 20 = 65,495 data bytes may follow, where the first 20 refer to the IP header and the second to the TCP header.

Segments without any data are legal and are commonly used for acknowledgements and control messages.

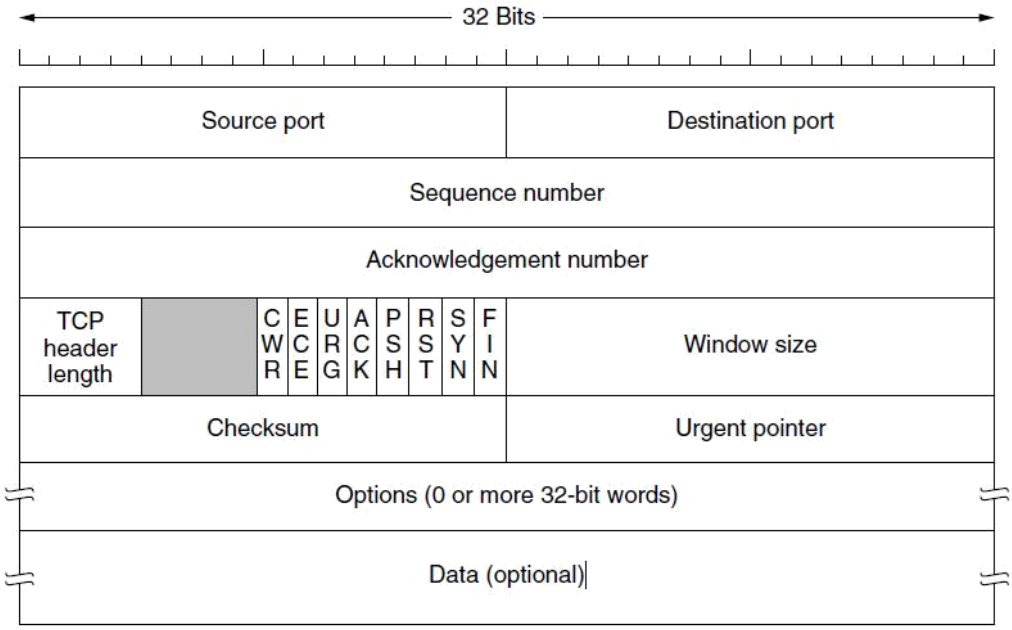


Figure 4.15: The TCP Header

The *Source port* and *Destination port* fields identify the local end points of the connection. The source and destination end points together identify the connection. This connection identifier is called a **5 tuple** because it consists of five pieces of information: the protocol (TCP), source IP and source port, and destination IP and destination port.

The *Sequence number* and *Acknowledgement number* fields perform their usual functions.

The *Sequence number* and *Acknowledgement number* fields perform their usual functions.

The *TCP header length* tells how many 32-bit words are contained in the TCP header. This information is needed because the *Options* field is of variable length, so the header is, too.

Now come eight 1-bit flags. *CWR* and *ECE* are used to signal congestion when ECN (Explicit Congestion Notification) is used. *CWR* is set to signal *Congestion Window Reduced* from the TCP sender to the TCP receiver so that it knows the sender has slowed down and can stop sending the *ECN-Echo*.

URG is set to 1 if the *Urgent pointer* is in use. The *Urgent pointer* is used to indicate a byte offset from the current sequence number at which urgent data are to be found.

The *ACK* bit is set to 1 to indicate that the *Acknowledgement number* is valid. This is the case for nearly all packets. If *ACK* is 0, the segment does not contain an acknowledgement, so the *Acknowledgement number* field is ignored.

The *PSH* bit indicates PUSHed data. The receiver is hereby kindly requested to deliver the data to the application upon arrival and not buffer it until a full buffer has been received (which it might otherwise do for efficiency).

The *RST* bit is used to abruptly reset a connection that has become confused due to a host crash or some other reason.

The *SYN* bit is used to establish connections. The *FIN* bit is used to release a connection.

The *Window size* field tells how many bytes may be sent starting at the byte acknowledged.

A *Checksum* is also provided for extra reliability. The *Options* field provides a way to add extra facilities not covered by the regular header.

TCP Connection Establishment:

Connections are established in TCP by means of the three-way handshake. To establish a connection, one side, say, the server, passively waits for an incoming connection by executing the LISTEN and ACCEPT primitives in that order, either specifying a specific source or nobody in particular.

The other side, say, the client, executes a CONNECT primitive, specifying the IP address and port to which it wants to connect, the maximum TCP segment size it is willing to accept, and optionally some user data (e.g., a password). The CONNECT primitive sends a TCP segment with the *SYN* bit on and *ACK* bit off and waits for a response.

When this segment arrives at the destination, the TCP entity there checks to see if there is a process that has done a LISTEN on the port given in the *Destination port* field. If not, it sends a reply with the *RST* bit on to reject the connection.

TCP Connection Release

Although TCP connections are full duplex, to understand how connections are released it is best to think of them as a pair of simplex connections. Each simplex connection is released independently of its sibling.

To release a connection, either party can send a TCP segment with the *FIN* bit set, which means that it has no more data to transmit. When the *FIN* is acknowledged, that direction is shut down for new data.

Data may continue to flow indefinitely in the other direction, however. When both directions have been shut down, the connection is released.

TCP Congestion Control:

The network layer detects congestion when queues grow large at routers and tries to manage it, if only by dropping packets. It is up to the transport layer to receive congestion feedback from the network layer and slow down the rate of traffic that it is sending into the network.

In the Internet, TCP plays the main role in controlling congestion, as well as the main role in reliable transport. That is why it is such a special protocol.

PERFORMANCE PROBLEMS IN COMPUTER NETWORKS

Some performance problems, such as congestion, are caused by temporary resource overloads. If more traffic suddenly arrives at a router than the router can handle, congestion will build up and performance will suffer.

Performance also degrades when there is a structural resource imbalance. For example, if a gigabit communication line is attached to a low-end PC, the poor host will not be able to process the incoming packets fast enough and some will be lost. These packets will eventually be retransmitted, adding delay, wasting bandwidth, and generally reducing performance.

Overloads can also be synchronously triggered. As an example, if a segment contains a bad parameter , in many cases the receiver will thoughtfully send back an error notification.

Another tuning issue is setting timeouts. When a segment is sent, a timer is set to guard against loss of the segment. If the timeout is set too short, unnecessary retransmissions will occur, clogging the wires. If the timeout is set too long, unnecessary delays will occur after a segment is lost.

NETWORK PERFORMANCE MEASUREMENT:

When a network performs poorly, its users often complain to the folks running it, demanding improvements. To improve the performance, the operators must first determine exactly what is going on. To find out what is really happening, the operators must make measurements.

Measurements can be made in different ways and at many locations (both in the protocol stack and physically). The most basic kind of measurement is to start a timer when beginning some activity and see how long that activity takes.

Other measurements are made with counters that record how often some event has happened (e.g., number of lost segments).

Measuring network performance and parameters has many potential pitfalls. We list a few of them here. Any systematic attempt to measure network performance should be careful to avoid these.

1) Make Sure That the Sample Size Is Large Enough

Do not measure the time to send one segment, but repeat the measurement, say, one million times and take the average.

2) Make Sure That the Samples Are Representative

Ideally, the whole sequence of one million measurements should be repeated at different times of the day and the week to see the effect of different network conditions on the measured quantity.

3) Caching Can Wreak Havoc with Measurements

Repeating a measurement many times will return an unexpectedly fast answer if the protocols use caching mechanisms.

4) Be Sure That Nothing Unexpected Is Going On during Your Tests

Making measurements at the same time that some user has decided to run a video conference over your network will often give different results than if there is no video conference.

5) Be Careful When Using a Coarse-Grained Clock

Computer clocks function by incrementing some counter at regular intervals.

6) Be Careful about Extrapolating the Results

Suppose that you make measurements with simulated network loads running from 0 (idle) to 0.4 (40% of capacity).

UNIT-V

INTRODUCTION TO APPLICATION LAYER:

INTRODUCTION:

The application layer provides services to the user. Communication is provided using a logical connection, which means that the two application layers assume that there is an imaginary direct connection through which they can send and receive messages.

Providing Services:

All communication networks that started before the Internet were designed to provide services to network users. Most of these networks, however, were originally designed to provide one specific service. For example, the telephone network was originally designed to provide voice service: to allow people all over the world to talk to each other. This network, however, was later used for some other services, such as facsimile (fax), enabled by users adding some extra hardware at both ends.

The Internet was originally designed for the same purpose: to provide service to users around the world. The layered architecture of the TCP/IP protocol suite, however, makes the Internet more flexible than other communication networks such as postal or telephone networks.

Each layer in the suite was originally made up of one or more protocols, but new protocols can be added or some protocols can be removed or replaced by the Internet authorities. However, if a protocol is added to each layer, it should be designed in such a way that it uses the services provided by one of the protocols at the lower layer.

If a protocol is removed from a layer, care should be taken to change the protocol at the next higher layer that supposedly uses the services of the removed protocol. The application layer, however, is somewhat different from other layers in that it is the highest layer in the suite.

The protocols in this layer do not provide services to any other protocol in the suite; they only receive services from the protocols in the transport layer. This means that protocols can be removed from this layer easily. New protocols can be also added to this layer as long as the new protocols can use the services provided by one of the transport-layer protocols.

Standard and Nonstandard Protocols:

To provide smooth operation of the Internet, the protocols used in the first four layers of the TCP/IP suite need to be standardized and documented.

Standard Application-Layer Protocols:

There are several application-layer protocols that have been standardized and documented by the Internet authority, and we are using them in our daily interaction with the Internet.

Each standard protocol is a pair of computer programs that interact with the user and the transport layer to provide a specific service to the user.

Nonstandard Application-Layer Protocols:

A programmer can create a nonstandard application-layer program if she can write two programs that provide service to the user by interacting with the transport layer.

Application-Layer Paradigms

It should be clear that to use the Internet we need two application programs to interact with each other: one running on a computer somewhere in the world, the other running on another computer somewhere else in the world. The two programs need to send messages to each other through the Internet infrastructure.

However, we have not discussed what the relationship should be between these programs.

Should both application programs be able to request services and provide services, or should the application programs just do one or the other?

Two paradigms have been developed during the lifetime of the Internet to answer this question: the client-server paradigm and the peer-to-peer paradigm.

Traditional Paradigm: Client-Server:

The traditional paradigm is called the **client-server paradigm**. It was the most popular paradigm until a few years ago. In this paradigm, the service provider is an application program, called the server process; it runs continuously, waiting for another application program, called the client process, to make a connection through the Internet and ask for service.

There are normally some server processes that can provide a specific type of service, but there are many clients that request service from any of these server processes. The server process must be running all the time; the client process is started when the client needs to receive service.

New Paradigm: Peer-to-Peer:

A new paradigm, called the **peer-to-peer paradigm** (often abbreviated *P2P*

paradigm) has emerged to respond to the needs of some new applications.

In this paradigm, there is no need for a server process to be running all the time and waiting for the client processes to connect. The responsibility is shared between peers.

A computer connected to the Internet can provide service at one time and receive service at another time. A computer can even provide and receive services at the same time.

CLIENT-SERVER PROGRAMMING:

In a client-server paradigm, communication at the application layer is between two running application programs called ***processes****:* a *client* and a *server*.

A client is a running program that initializes the communication by sending a request; a server is another application program that waits for a request from a client.

The server handles the request received from a client, prepares a result, and sends the result back to the client. This definition of a server implies that a server must be running when a request from a client arrives, but the client needs to be run only when it is needed.

This means that if we have two computers connected to each other somewhere, we can run a client process on one of them and the server on the other. However, we need to be careful that the server program is started before we start running the client program.

Application Programming Interface:

A client process communicate with a server process with the help of a computer program which is normally written in a computer language with a predefined set of instructions that tells the computer what to do.

A computer language has a set of instructions for mathematical operations, a set of instructions for string manipulation, a set of instructions for input/output access, and so on.

If we need a process to be able to communicate with another process, we need a new set of instructions to tell the lowest four layers of the TCP/IP suite to open the connection, send and receive data from the other end, and close the connection. A set of instructions of this kind is normally referred to as an **application programming interface (API)**.

An interface in programming is a set of instructions between two entities. In this case, one of the entities is the process at the application layer and the other is the *operating system* that encapsulates the first four layers of the TCP/IP protocol

suite.

Several APIs have been designed for communication. One of the most common one is: **socket interface**. The socket interface is a set of instructions that provide communication between the application layer and the operating system, as shown in Figure 5.1.

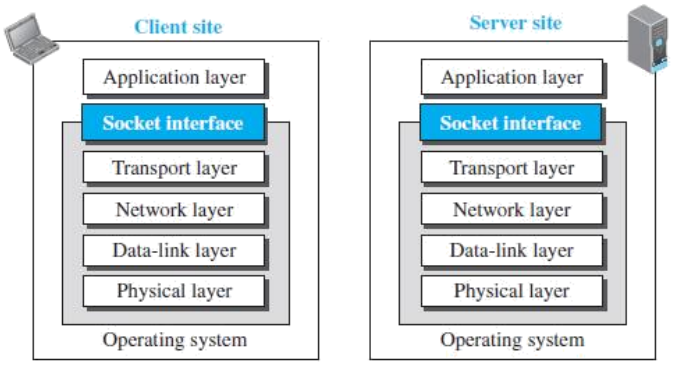


FIGURE 5.1: Position Of The Socket Interface

It is a set of instructions that can be used by a process to communicate with another process. The idea of sockets allows us to use the set of all instructions already designed in a programming language for other sources and sinks.

For example, in most computer languages, like C, C++, or Java, we have several instructions that can read and write data to other sources and sinks such as a keyboard (a source), a monitor (a sink), or a file (source and sink). We can use the same instructions to read from or write to sockets.

Sockets:

Although a socket is supposed to behave like a terminal or a file, it is not a physical entity like them; it is an abstraction. It is an object that is created and used by the application program.

Socket Addresses:

The interaction between a client and a server is two-way communication. In a two-way communication, we need a pair of addresses: local (sender) and remote (receiver). The local address in one direction is the remote address in the other direction and vice versa.

Since communication in the client-server paradigm is between two sockets, we need a pair of **socket addresses** for communication: a local socket address and a remote socket address. However, we need to define a socket address in terms of identifiers used in the TCP/IP protocol suite.

A socket address should first define the computer on which a client or a server is running. Socket address should be a combination of an IP address (32 bit)

and a port number (16 bit).

Since a socket defines the end-point of the communication, we can say that a socket is identified by a pair of socket addresses, a local and a remote.

Finding Socket Addresses: How can a client or a server find a pair of socket addresses for communication? The situation is different for each site.

Server Site: The server needs a local (server) and a remote (client) socket address for communication.

Local Socket Address The local (server) socket address is provided by the operating system. The operating system knows the IP address of the computer on which the server process is running. The port number of a server process, however, needs to be assigned.

If the server process is a standard one defined by the Internet authority, a port number is already assigned to it. For example, the assigned port number for a Hypertext Transfer Protocol (HTTP) is the integer 80, which cannot be used by any other process.

Remote Socket Address The remote socket address for a server is the socket address of the client that makes the connection. Since the server can serve many clients, it does not know beforehand the remote socket address for communication.

The server can find this socket address when a client tries to connect to the server. The client socket address, which is contained in the request packet sent to the server, becomes the remote socket address that is used for responding to the client.

Client Site: The client also needs a local (client) and a remote (server) socket address for communication.

Local Socket Address The local (client) socket address is also provided by the operating system. The operating system knows the IP address of the computer on which the client is running. The port number, however, is a 16-bit temporary integer that is assigned to a client process each time the process needs to start the communication.

The port number, however, needs to be assigned from a set of integers defined by the Internet authority and called the ephemeral (temporary) port numbers. The operating system, however, needs to guarantee that the new port number is not used by any other running client process.

Remote Socket Address Finding the remote (server) socket address for a client, however, needs more work. When a client process starts, it should know the socket address of the server it wants to connect to.

Using Services of the Transport Layer:

A pair of processes provide services to the users of the Internet, human or programs. A pair of processes, however, need to use the services provided by the transport layer for communication because there is no physical communication at the application layer.

WORLD WIDE WEB AND HTTP:

World Wide Web:

The idea of the Web was first proposed by Tim Berners-Lee in 1989. The Web today is a repository of information in which the documents, called ***web pages,*** are distributed all over the world and related documents are linked together.

The popularity and growth of the Web can be related to two terms in the above statement: *distributed* and *linked*. Distribution allows the growth of the Web. Each web server in the world can add a new web page to the repository and announce it to all Internet users without overloading a few servers.

Linking allows one web page to refer to another web page stored in another server somewhere else in the world. The linking of web pages was achieved using a concept called ***hypertext,*** which was introduced many years before the advent of the Internet.

The idea was to use a machine that automatically retrieved another document stored in the system when a link to it appeared in the document. The Web implemented this idea electronically to allow the linked document to be retrieved when the link was clicked by the user.

Today, the term *hypertext,* coined to mean linked text documents, has been changed to ***hypermedia,*** to show that a web pagecan be a text document, an image, an audio file, or a video file.

Architecture:

The WWW today is a distributed client-server service, in which a client using a browser can access a service using a server. However, the service provided is distributed over many locations called *sites.* Each site holds one or more web pages.

Each web page, however, can contain some links to other web pages in the same or other sites. In other words, a web page can be simple or composite. A simple web page has no links to other web pages; a composite web page has one or more links to other web pages. Each web page is a file with a name and address.

Web Client (Browser): A variety of vendors offer commercial **browsers** that interpret and display a web page, and all of them use nearly the same architecture. Each browser usually consists of three parts: a controller, client protocols, and interpreters (figure 5.2).



Figure 5.2: Browser

The controller receives input from the keyboard or the mouse and uses the client programs to access the document. After the document has been accessed, the controller uses one of the interpreters to display the document on the screen.

The client protocol can be one of the protocols described later, such as HTTP or FTP. The interpreter can be HTML, Java, or JavaScript, depending on the type of document. Some commercial browsers include Internet Explorer, Netscape Navigator, and Firefox.

Web Server: The web page is stored at the server. Each time a request arrives, the corresponding document is sent to the client. To improve efficiency, servers normally store requested files in a cache in memory; memory is faster to access than a disk.

A server can also become more efficient through multithreading or multiprocessing. In this case, a server can answer more than one request at a time. Some popular web servers include Apache and Microsoft Internet Information Server.

Uniform Resource Locator (URL):

A web page, as a file, needs to have a unique identifier to distinguish it from other web pages. To define a web page, we need three identifiers: *host, port,* and *path*.

However, before defining the web page, we need to tell the browser what clientserver application we want to use, which is called the *protocol*. This means we need four identifiers to define the web page.

The first is the type of vehicle to be used to fetch the web page; the last three make up the combination that defines the destination object (web page).

***Protocol*.** The first identifier is the abbreviation for the client-server program that we need in order to access the web page.

Although most of the time the protocol is HTTP (HyperText Transfer Protocol), we can also use other protocols such as FTP (File Transfer Protocol).

***Host*.** The host identifier can be the IP address of the server or the unique name given to the server. IP addresses can be defined in dotted decimal notation.

***Port*.** The port, a 16-bit integer, is normally predefined for the client-server application.

***Path*.** The path identifies the location and the name of the file in the underlying operating system. The format of this identifier normally depends on the operating system.

To combine these four pieces together, the **uniform resource locator (URL)** has been designed; it uses three different separators between the four pieces as shown below:

protocol://host/path	Used most of the time
protocol://host:port/path	Used when port number is needed

Web Documents:

The documents in the WWW can be grouped into three broad categories: static, dynamic, and active.

Static Documents:

Static documents are fixed-content documents that are created and stored in a server. The client can get a copy of the document only. In other words, the contents of the file are determined when the file is created, not when it is used.

Static documents are prepared using one of several languages: *HyperText Markup Language* (HTML), *Extensible Markup Language* (XML), *Extensible Style Language* (XSL), and *Extensible Hypertext Markup Language* (XHTML).

Dynamic Documents:

A **dynamic document** is created by a web server whenever a browser requests the document. When a request arrives, the web server runs an application program or a script that creates the dynamic document.

The server returns the result of the program or script as a response to the browser that requested the document. Because a fresh document is created for each request, the contents of a dynamic document may vary from one request to another. A very simple example of a dynamic document is the retrieval of the time and date from a server.

Active Documents:

For many applications, we need a program or a script to be run at the client site. These are called ***active documents*.** For example, suppose we want to run a program that creates animated graphics on the screen or a program that interacts with the user.

HyperText Transfer Protocol (HTTP):

The **HyperText Transfer Protocol (HTTP)** is used to define how the client-server programs can be written to retrieve web pages from the Web. An HTTP client sends a request; an HTTP server returns a response. The server uses the port number 80; the client uses a temporary port number. HTTP uses the services of TCP, which, as discussed before, is a connection-oriented and reliable protocol.

Nonpersistent versus Persistent Connections:

If the web pages, objects to be retrieved, are located on different servers, we do not have any other choice than to create a new TCP connection for retrieving each object. However, if some of the objects are located on the same server, we have two choices: to retrieve each object using a new TCP connection or to make a TCP connection and retrieve them all. The first method is referred to as a *nonpersistent connection,* the second as a *persistent connection*.

Nonpersistent Connections

In a **nonpersistent connection**, one TCP connection is made for each request/response.

The following lists the steps in this strategy:

1. The client opens a TCP connection and sends a request.

2. The server sends the response and closes the connection.

3. The client reads the data until it encounters an end-of-file marker; it then closes the connection.

Persistent Connections

HTTP version 1.1 specifies a **persistent connection** by default. In a persistent connection, the server leaves the connection open for more requests after sending a response.

The server can close the connection at the request of a client or if a time-out has been reached. The sender usually sends the length of the data with each response. However, there are some occasions when the sender does not know the length of the data.

This is the case when a document is created dynamically or actively. In these cases, the server informs the client that the length is not known and closes the connection after sending the data so the client knows that the end of the data has been reached. Time and resources are saved using persistent connections.

Only one set of buffers and variables needs to be set for the connection at each site. The round trip time for connection establishment and connection termination is saved.

Message Formats:

The HTTP protocol defines the format of the request and response messages. Each message is made of four sections. The first section in the request message is called the *request line;* the first section in the response message is called the *status line*.

The other three sections have the same names in the request and response messages. However, the similarities between these sections are only in the names; they may have different contents. We discuss each message type separately.

Request Message:

There are three fields in this line separated by one space and terminated by two characters (carriage return and line feed). The fields are called *method, URL*, and *version*.

The method field defines the request types. Several methods are defined like GET, PUT, HEAD, POST, TRACE, DELETE, etc. The URL defines the address and name of the corresponding web page. The version field gives the version of the protocol; the most current version of HTTP is 1.1.

Response Message:

A response message consists of a status line, header lines, a blank line, and sometimes a body. The first line in a response message is called the *status line*. There are three fields in this line separated by spaces and terminated by a carriage return and line feed.

The first field defines the version of HTTP protocol, currently 1.1. The status code field defines the status of the request. It consists of three digits. Whereas the codes in the 100 range are only informational, the codes in the 200 range indicate a successful request.

The codes in the 300 range redirect the client to another URL, and the codes in the 400 range indicate an error at the client site. Finally, the codes in the 500 range indicate an error at the server site.

The status phrase explains the status code in text form. After the status line, we can have zero or more *response header* lines. Each header line sends additional information from the server to the client.

Web Caching: Proxy Servers:

HTTP supports **proxy servers**. A proxy server is a computer that keeps copies of responses to recent requests. The HTTP client sends a request to the proxy server. The proxy server checks its cache.

If the response is not stored in the cache, the proxy server sends the request

to the corresponding server. Incoming responses are sent to the proxy server and stored for future requests from other clients.

The proxy server reduces the load on the original server, decreases traffic, and improves latency. However, to use the proxy server, the client must be configured to access the proxy instead of the target server.

HTTP Security:

HTTP per se does not provide security. HTTP can be run over the Secure Socket Layer (SSL). In this case, HTTP is referred to as HTTPS. HTTPS provides confidentiality, client and server authentication, and data integrity.

FTP:

File Transfer Protocol (FTP) is the standard protocol provided by TCP/IP for copying a file from one host to another. Although transferring files from one system to another seems simple and straightforward, some problems must be dealt with first.

Although we can transfer files using HTTP, FTP is a better choice to transfer large files or to transfer files using different formats. Figure 5.3 shows the basic model of FTP. The client has three components: the user interface, the client control process, and the client data transfer process. The server has two components: the server control process and the server data transfer process.

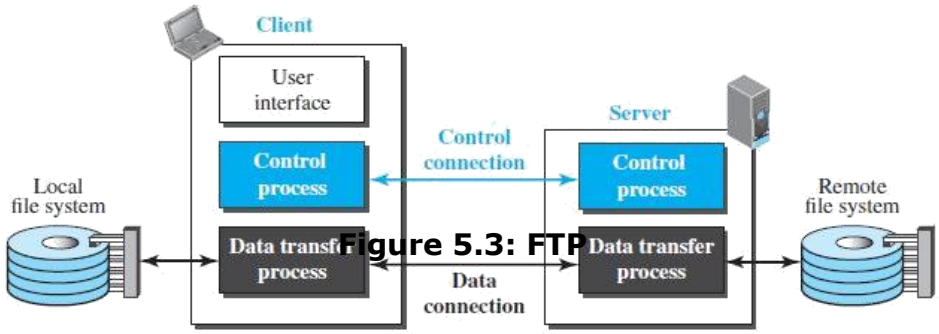


Figure 5.3: FTP

The control connection is made between the control processes. The data connection is made between the data transfer processes. Separation of commands and data transfer makes FTP more efficient. The control connection uses very simple rules of communication. We need to transfer only a line of command or a line of response at a time. The data connection, on the other hand, needs more complex rules due to the variety of data types transferred.

Two Connections

The two connections in FTP have different lifetimes. The control connection remains connected during the entire interactive FTP session. The data connection is opened and then closed for each file transfer activity.

FTP uses two well-known TCP ports: port 21 is used for the control connection, and port 20 is used for the data connection.

Control Connection:

During this control connection, commands are sent from the client to the server and responses are sent from the server to the client. Commands, which are sent from the FTP client control process, are in the form of ASCII uppercase, which may or may not be followed by an argument. Some of the most common commands are shown in table below:

Command	*Argument(s)*	*Description*
ABOR		Abort the previous command
CDUP		Change to parent directory
CWD	Directory name	Change to another directory
DELE	File name	Delete a file
LIST	Directory name	List subdirectories or files
MKD	Directory name	Create a new directory
PASS	User password	Password
PASV		Server chooses a port
PORT	Port identifier	Client chooses a port
PWD		Display name of current directory
QUIT		Log out of the system

Every FTP command generates at least one response. A response has two parts: a three-digit number followed by text. The numeric part defines the code; the text part defines needed parameters or further explanations. The first digit defines the status of the command. The second digit defines the area in which the status applies. The third digit provides additional information.

Code	Description	Code	Description
125	Data Connection Open	250	Request file action OK
150	File Status OK	331	User name OK; password is needed
200	Command OK	425	Cannot open data connection

ELECTRONIC MAIL:

Electronic mail (or e-mail) allows users to exchange messages. The nature of this application, however, is different from other applications discussed so far. In an application such as HTTP or FTP, the server program is running all the time, waiting for a request from a client. When the request arrives, the server provides the service. There is a request and there is a response.

In the case of electronic mail, the situation is different. First, e-mail is considered a one-way transaction. When Alice sends an email to Bob, she may expect a response, but this is not a mandate. Bob may or may not respond. If he does respond, it is another one-way transaction.

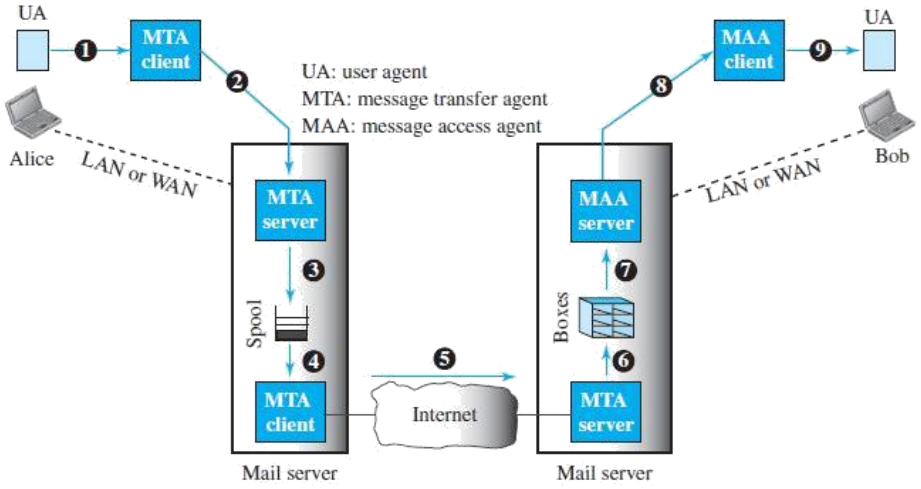
Second, it is neither feasible nor logical for Bob to run a server program and

wait until someone sends an e-mail to him. Bob may turn off his computer when he is not using it.

This means that the idea of client/server programming should be implemented in another way: using some intermediate computers (servers). The users run only client programs when they want and the intermediate servers apply the client/server paradigm

Architecture:

To explain the architecture of e-mail, we give a common scenario as shown in Figure 5.4.



In the common scenario, the sender and the receiver of the e-mail, Alice and Bob respectively, are connected via a LAN or a WAN to two mail servers. The administrator has created one mailbox for each user where the received messages are stored.

A *mailbox* is part of a server hard drive, a special file with permission restrictions. Only the owner of the mailbox has access to it. The administrator has also created a queue (spool) to store messages waiting to be sent.

A simple e-mail from Alice to Bob takes nine different steps. Alice and Bob use three different *agents*: a **user agent (UA),** a **message transfer agent (MTA),** and a **message access agent (MAA).** When Alice needs to send a message to Bob, she runs a UA program to prepare the message and send it to her mail server.

The mail server at her site uses a queue (spool) to store messages waiting to be sent. The message, however, needs to be sent through the Internet from Alice's site to Bob's site using an MTA. Here two message transfer agents are needed: one client and one server.

Like most client-server programs on the Internet, the server needs to run all the time because it does not know when a client will ask for a connection. The client, on the other hand, can be triggered by the system when there is a message

in the queue to be sent.

The user agent at the Bob site allows Bob to read the received message. Bob later uses an MAA client to retrieve the message from an MAA server running on the second server.

User Agent: The first component of an electronic mail system is the **user agent (UA).** It provides service to the user to make the process of sending and receiving a message easier.

A user agent is a software package (program) that composes, reads, replies to, and forwards messages. It also handles local mailboxes on the user computers.

Message Transfer Agent: SMTP: Based on the common scenario, we can say that the e-mail is one of those applications that needs three uses of client-server paradigms to accomplish its task. It is important that we distinguish these three when we are dealing with e-mail.

The formal protocol that defines the MTA client and server in the Internet is called ***Simple Mail Transfer Protocol (SMTP).*** SMTP is used two times, between the sender and the sender's mail server and between the two mail servers. SMTP simply defines how commands and responses must be sent back and forth.

Message Access Agent: POP and IMAP: The first and second stages of mail delivery use SMTP. However, SMTP is not involved in the third stage because SMTP is a *push* protocol; it pushes the message from the client to the server.On the other hand, the third stage needs a *pull* protocol; the client must pull messages from the server. The direction of the bulk data is from the server to the client. The third stage uses a message access agent.

Currently two message access protocols are available: Post Office Protocol, version 3 (POP3) and Internet Mail Access Protocol, version 4 (IMAP4).

POP3:

Post Office Protocol, version 3 (POP3) is simple but limited in functionality. The client POP3 software is installed on the recipient computer; the server POP3 software is installed on the mail server.

Mail access starts with the client when the user needs to download its e-mail from the mailbox on the mail server. The client opens a connection to the server on TCP port 110. It then sends its user name and password to access the mailbox. The user can then list and retrieve the mail messages, one by one.

POP3 has two modes: the *delete* mode and the *keep* mode. In the delete mode, the mail is deleted from the mailbox after each retrieval. In the keep mode, the mail remains in the mailbox after retrieval.

IMAP4:

Another mail access protocol is **Internet Mail Access Protocol, version 4 (IMAP4).** IMAP4 is similar to POP3, but it has more features; IMAP4 is more powerful and more complex.

POP3 is deficient in several ways. It does not allow the user to organize her mail on the server; the user cannot have different folders on the server. In addition, POP3 does not allow the user to partially check the contents of the mail before downloading.

IMAP4 provides the following extra functions:

- A user can check the e-mail header prior to downloading.
- A user can search the contents of the e-mail for a specific string of characters prior to downloading.
- A user can partially download e-mail. This is especially useful if bandwidth is limited and the e-mail contains multimedia with high bandwidth requirements.
- A user can create, delete, or rename mailboxes on the mail server.

TELNET:

A server program can provide a specific service to its corresponding client program. For example, the FTP server is designed to let the FTP client store or retrieve files on the server site. However, it is impossible to have a client/server pair for each type of service we need; the number of servers soon becomes intractable which is not scalable.

Another solution is to have a specific client/server program for a set of common scenarios, but to have some generic client/server programs that allow a user on the client site to log into the computer at the server site and use the services available there.

For example, if a student needs to use the Java compiler program at her university lab, there is no need for a Java compiler client and a Java compiler server. The student can use a client logging program to log into the university server and use the compiler program at the university. We refer to these generic client/server pairs as ***remote logging*** applications.

One of the original remote logging protocols is **TELNET,** which is an abbreviation for *TErminaL NETwork.* Although TELNET requires a logging name and password, it is vulnerable to hacking because it sends all data including the password in plaintext (not encrypted).

A hacker can eavesdrop and obtain the logging name and password. Because of this security issue, the use of TELNET has diminished in favor of another protocol, Secure Shell (SSH).

Although TELNET is almost replaced by SSH, we briefly discuss TELNET here for two reasons:

1. The simple plaintext architecture of TELNET allows us to explain the issues and challenges related to the concept of remote logging, which is also used in SSH when it serves as a remote logging protocol.

2. Network administrators often use TELNET for diagnostic and debugging purposes.

Local versus Remote Logging:

When a user logs into a local system, it is called *local logging*. As a user types at a terminal or at a workstation running a terminal emulator, the keystrokes are accepted by the terminal driver.

The terminal driver passes the characters to the operating system. The operating system, in turn, interprets the combination of characters and invokes the desired application program or utility.

However, when a user wants to access an application program or utility located on a remote machine, she performs *remote logging.* Here the TELNET client and server programs come into use. The user sends the keystrokes to the terminal driver where the local operating system accepts the characters but does not interpret them.

The characters are sent to the TELNET client, which transforms the characters into a universal character set called *Network Virtual Terminal* (NVT) characters and delivers them to the local TCP/IP stack.

The commands or text, in NVT form, travel through the Internet and arrive at the TCP/IP stack at the remote machine. Here the characters are delivered to the operating system and passed to the TELNET server, which changes the characters to the corresponding characters understandable by the remote computer.

However, the characters cannot be passed directly to the operating system because the remote operating system is not designed to receive characters from a TELNET server; it is designed to receive characters from a terminal driver.

The solution is to add a piece of software called a *pseudoterminal driver*, which pretends that the characters are coming from a terminal. The operating system then passes the characters to the appropriate application program.

NVT uses two sets of characters, one for data and one for control. Both are 8-bit bytes. For data, NVT normally uses what is called *NVT ASCII*. This is an 8-bit character set in which the seven lowest order bits are the same as US ASCII and the highest order bit is 0.

To send control characters between computers (from client to server or vice versa), NVT uses an 8-bit character set in which the highest order bit is set to 1.

Options: TELNET lets the client and server negotiate options before or during the use of the service.

User Interface:

The operating system (UNIX, for example) defines an interface with user-friendly commands. An example of such a set of commands can be found in Table beow:

Command Name	Meaning
open	Connect to a remote computer
close	Close the connections
display	Show the operating parameters
mode	Change to line or character mode
Quit	Exit TELNET
send	Send special characters

SECURE SHELL (SSH):

Although **Secure Shell (SSH)** is a secure application program that can be used today for several purposes such as remote logging and file transfer, it was originally designed to replace TELNET.

There are two versions of SSH: SSH-1 and SSH-2, which are totally incompatible. The first version, SSH-1, is now deprecated because of security flaws in it. In this section, we discuss only SSH-2.

Components: SSH is an application-layer protocol with three components.

SSH Transport-Layer Protocol (SSH-TRANS):

Since TCP is not a secured transport-layer protocol, SSH first uses a protocol that creates a secured channel on top of the TCP. This new layer is an independent protocol referred to as SSH-TRANS.

When the procedure implementing this protocol is called, the client and server first use the TCP protocol to establish an insecure connection. Then they exchange several security parameters to establish a secure channel on top of the TCP. The services provided by this protocol are:

1. Privacy or confidentiality of the message exchanged.
2. Data integrity, which means that it is guaranteed that the messages exchanged between the client and server are not changed by an intruder.
3. Server authentication, which means that the client is now sure that the server is the one that it claims to be.
4. Compression of the messages, which improves the efficiency of the system and makes attack more difficult.

SSH Authentication Protocol (SSH-AUTH):

After a secure channel is established between the client and the server and the server is authenticated for the client, SSH can call another procedure that can authenticate the client for the server. The client authentication process in SSH is very similar to what is done in Secure Socket Layer (SSL).

This layer defines a number of authentication tools similar to the ones used in SSL. Authentication starts with the client, which sends a request message to the server. The request includes the user name, server name, the method of authentication, and the required data. The server responds with either a success message, which confirms that the client is authenticated, or a failed message, which means that the process needs to be repeated with a new request message.

SSH Connection Protocol (SSH-CONN):

After the secured channel is established and both server and client are authenticated for each other, SSH can call a piece of software that implements the third protocol, SSHCONN.

One of the services provided by the SSH-CONN protocol is multiplexing. SSH-CONN takes the secure channel established by the two previous protocols and lets the client create multiple logical channels over it. Each channel can be used for a different purpose, such as remote logging, file transfer, and so on.

Applications:

Although SSH is often thought of as a replacement for TELNET, SSH is, in fact, a general-purpose protocol that provides a secure connection between a client and server.

SSH for Remote Logging:

Several free and commercial applications use SSH for remote logging. Among them, we can mention PuTTy, by Simon Tatham, which is a client SSH program that can be used for remote logging. Another application program is Tectia, which can be used on several platforms.

SSH for File Transfer:

One of the application programs that is built on top of SSH for file transfer is the *Secure File Transfer Program* (*sftp*). The *sftp* application program uses one of the channels provided by the SSH to transfer files. Another common application is called *Secure Copy* (*scp*). This application uses the same format as the UNIX copy command, *cp,* to copy files.

DOMAIN NAME SYSTEM (DNS):

Since the Internet is so huge today, a central directory system cannot hold all the mapping. In addition, if the central computer fails, the whole communication network will collapse.

A better solution is to distribute the information among many computers in the world. In this method, the host that needs mapping can contact the closest computer holding the needed information. This method is used by the **Domain Name System (DNS)**.

Figure 5.5 shows how TCP/IP uses a DNS client and a DNS server to map a name to an address. A user wants to use a file transfer client to access the corresponding file transfer server running on a remote host. The user knows only the file transfer server name, such as *afilesource.com.*

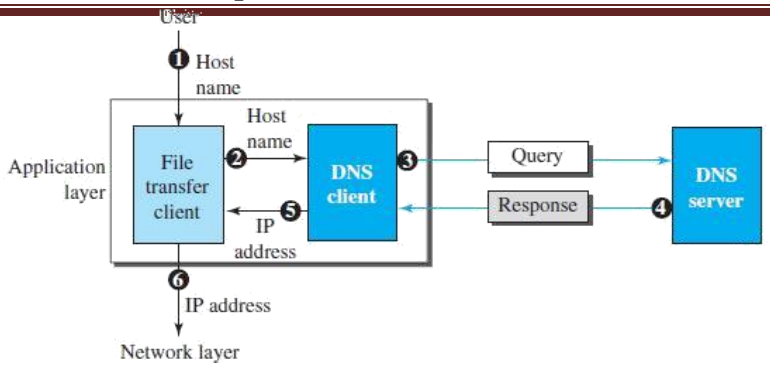


Figure 5.5: Purpose of DNS

Name Space:

A **name space** that maps each address to a unique name can be organized in two ways: flat or hierarchical. In a *flat name space,* a name is assigned to an address.

A name in this space is a sequence of characters without structure. The names may or may not have a common section; if they do, it has no meaning. The main disadvantage of a flat name space is that it cannot be used in a large system such as the Internet because it must be centrally controlled to avoid ambiguity and duplication.

In a *hierarchical name space,* each name is made of several parts. The first part can define the nature of the organization, the second part can define the name of an organization, the third part can define departments in the organization, and so on. In this case, the authority to assign and control the name spaces can be decentralized.

A central authority can assign the part of the name that defines the nature of the organization and the name of the organization. The responsibility for the rest of the name can be given to the organization itself.

The organization can add suffixes (or prefixes) to the name to define its host or resources. The management of the organization need not worry that the prefix chosen for a host is taken by another organization because, even if part of an address is the same, the whole address is different.

Domain Name Space:

To have a hierarchical name space, a **domain name space** was designed. In this design the names are defined in an inverted-tree structure with the root at the top. The tree can have only 128 levels: level 0 (root) to level 127 (see Figure 5.6).

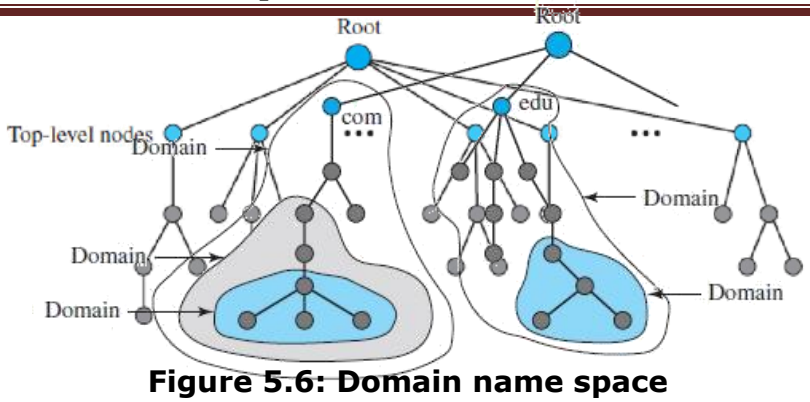


Figure 5.6: Domain name space

Label:

Each node in the tree has a **label**, which is a string with a maximum of 63 characters. The root label is a null string (empty string). DNS requires that children of a node (nodes that branch from the same node) have different labels, which guarantees the uniqueness of the domain names.

Domain Name:

Each node in the tree has a domain name. A full **domain name** is a sequence of labels separated by dots (**.**). The domain names are always read from the node up to the root.

The last label is the label of the root (null). This means that a full domain name always ends in a null label, which means the last character is a dot because the null string is nothing. Figure 5.7 shows some domain names.

Domain:

A **domain** is a subtree of the domain name space. The name of the domain is the name of the node at the top of the subtree. Figure 5.8 shows some domains. Note that a domain may itself be divided into domains.

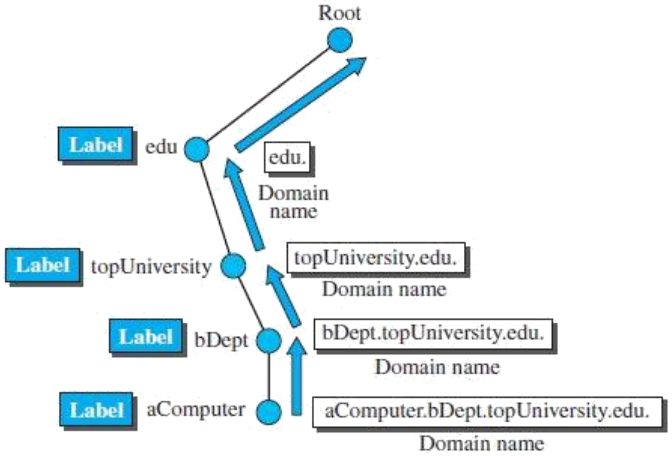


Figure 5.7: Domain names and labels

Figure 5.8: Domains

SNMP:

Several network management standards have been devised during the last few decades. The most important one is **Simple Network Management Protocol (SNMP),** used by the Internet.

SNMP is a framework for managing devices in an internet using the TCP/IP protocol suite. It provides a set of fundamental operations for monitoring and maintaining an internet. SNMP uses the concept of manager and agent. That is, a manager, usually a host, controls and monitors a set of agents, usually routers or servers (see Figure 5.9).

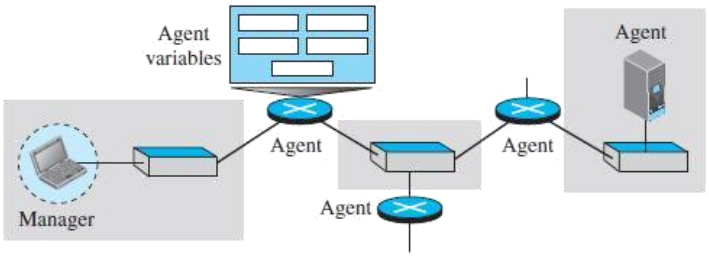


Figure 5.9: SNMP concept

SNMP is an application-level protocol in which a few manager stations control a set of agents. The protocol is designed at the application level so that it can monitor devices made by different manufacturers and installed on different physical networks.

In other words, SNMP frees management tasks from both the physical characteristics of the managed devices and the underlying networking technology. It can be used in a heterogeneous internet made of different LANs and WANs connected by routers made by different manufacturers.

Managers and Agents: A management station, called a *manager,* is a host that runs the SNMP client program. A managed station, called an *agent,* is a router (or a host) that runs the SNMP server program.

Management is achieved through simple interaction between a manager and an agent. The agent keeps performance information in a database. The manager has access to the values in the database.

For example, a router can store in appropriate variables the number of packets received and forwarded. The manager can fetch and compare the values of these two variables to see if the router is congested or not.

The manager can also make the router perform certain actions. For example, a router periodically checks the value of a reboot counter to see when it should reboot itself. It reboots itself, for example, if the value of the counter is 0. The manager can use this feature to reboot the agent remotely at any time. It simply sends a packet to force a 0 value in the counter.

Agents can also contribute to the management process. The server program running on the agent can check the environment and, if it notices something unusual, it can send a warning message (called a *Trap*) to the manager. In other words, management with SNMP is based on three basic ideas:

1. A manager checks an agent by requesting information that reflects the behavior of the agent.
2. A manager forces an agent to perform a task by resetting values in the agent database.
3. An agent contributes to the management process by warning the manager of an unusual situation.

Management Components: To do management tasks, SNMP uses two other protocols: **Structure of Management Information (SMI)** and **Management Information Base (MIB).**

Role of SNMP: SNMP has some very specific roles in network management. It defines the format of the packet to be sent from a manager to an agent and vice versa. It also interprets the result and creates statistics (often with the help of other management software).

Role of SMI: To use SNMP, we need rules for naming objects. This is particularly important because the objects in SNMP form a hierarchical structure. Part of a name can be inherited from the parent. We also need rules to define the types of objects.

Role of MIB: MIB creates a set of objects defined for each entity in a manner similar to that of a database (mostly metadata in a database, names & types without values).

Management is achieved through simple interaction between a manager and an agent. The agent keeps performance information in a database. The manager has access to the values in the database.

For example, a router can store in appropriate variables the number of packets received and forwarded. The manager can fetch and compare the values of these two variables to see if the router is congested or not.

The manager can also make the router perform certain actions. For example, a router periodically checks the value of a reboot counter to see when it should reboot itself. It reboots itself, for example, if the value of the counter is 0. The manager can use this feature to reboot the agent remotely at any time. It simply sends a packet to force a 0 value in the counter.

Agents can also contribute to the management process. The server program running on the agent can check the environment and, if it notices something unusual, it can send a warning message (called a *Trap*) to the manager. In other words, management with SNMP is based on three basic ideas:

1. A manager checks an agent by requesting information that reflects the behavior of the agent.
2. A manager forces an agent to perform a task by resetting values in the agent database.
3. An agent contributes to the management process by warning the manager of an unusual situation.

Management Components: To do management tasks, SNMP uses two other protocols: **Structure of Management Information (SMI)** and **Management Information Base (MIB).**

Role of SNMP: SNMP has some very specific roles in network management. It defines the format of the packet to be sent from a manager to an agent and vice versa. It also interprets the result and creates statistics (often with the help of other management software).

Role of SMI: To use SNMP, we need rules for naming objects. This is particularly important because the objects in SNMP form a hierarchical structure. Part of a name can be inherited from the parent. We also need rules to define the types of objects.

Role of MIB: MIB creates a set of objects defined for each entity in a manner similar to that of a database (mostly metadata in a database, names & types without values).